Česká zemědělská univerzita v Praze

Provozně ekonomická fakulta

Katedra informačního inženýrství

Bakalářská práce

Návrh a implementace webové aplikace komunitní půjčovny

Adam Stádník

© 2025 ČZU v Praze

Místo tohoto textu vložte PREDNI stranu zadání práce, které si můžete vyexportovat do PDF v IS.CZU.cz, pokud již máte schválené zadání i děkanem PEF.

!!!

Místo tohoto textu vložte ZADNÍ stranu zadání práce, které si můžete vyexportovat do PDF v IS.CZU.cz, pokud již máte schválené zadání i děkanem PEF.

V případě, že Vaše zadání je na více než 2 strany, vložte i další strany.

!!!

Čestné prohlášení

Prohlašuji, že svou bakalářskou práci "Návrh a implementace webové aplikace komunitní půjčovny" jsem vypracoval samostatně pod vedením vedoucího bakalářské práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou citovány v práci a uvedeny v seznamu použitých zdrojů na konci práce. Jako autor uvedené bakalářské práce dále prohlašuji, že jsem v souvislosti s jejím vytvořením neporušil autorská práva třetích osob.

V Praze dne datum odevzdání _____

Poděkování

Rád bych touto cestou poděkoval Ing. Josefu Pavlíčkovi, Ph.D., za jeho odborné vedení, cenné rady a trpělivost během konzultací. Jeho podpora a zpětná vazba mi významně pomohly při tvorbě této práce.

Návrh a implementace webové aplikace komunitní půjčovny

Abstrakt

Tato bakalářská práce se zabývá analýzou, návrhem a implementací webové aplikace zaměřené na komunitní půjčování předmětů. Cílem práce je vytvořit platformu, která uživatelům v blízkém okolí usnadní vzájemné sdílení a zapůjčování předmětů. Aplikace si klade za cíl podporovat principy udržitelnosti a přispívat k budování důvěry v rámci lokálních komunit.

Teoretická část práce se věnuje rešerši problematiky komunitního sdílení a analýze stávajících řešení. Podrobně také popisuje zvolený technologický stack, který tvoří webový framework Next.js pro vývoj v jazyce TypeScript a backendová platforma Supabase, jež zajišťuje SQL databázové služby, autentizaci uživatelů a real-time komunikaci.

Praktická část popisuje kompletní proces vývoje aplikace. Tento proces začíná analýzou požadavků a definováním cílové skupiny, pokračuje přes návrh uživatelského rozhraní (UI/UX) formou wireframů a prototypů a vrcholí implementací klientské i serverové logiky. Závěr práce je věnován testování funkčnosti, ověření použitelnosti a zhodnocení dosažených výsledků.

Klíčová slova: webová aplikace, komunita, TypeScript, SQL, Next.js, Supabase, UI, UX, Design, SEO

Design and Implementation of a Community Lending Web Application

Abstract

This bachelor's thesis deals with the analysis, design, and implementation of a web application focused on community-based item lending. The objective of the thesis is to create a platform that facilitates the mutual sharing and borrowing of items for users within a close proximity. The application aims to support principles of sustainability, reduce excessive consumption, and contribute to building trust within local communities.

The theoretical part of the thesis is dedicated to research on the subject of community sharing and an analysis of existing solutions. It also describes in detail the chosen technology stack, which consists of the Next.js web framework for development in the TypeScript language, and the Supabase backend platform, which provides SQL database services, user authentication, and real-time communication.

The practical part describes the complete application development process. This process begins with requirements analysis and target group definition, continues with the design of the user interface (UI/UX) through wireframes and prototypes, and culminates in the implementation of both client-side and server-side logic. The conclusion of the thesis is devoted to functionality testing, usability verification, and the evaluation of the achieved results.

Keywords: web application, community, TypeScript, SQL, Next.js, Supabase, UI, UX, Design, SEO

Obsah

1 Úvod **12**

2 Cíl práce a metodika **13**

2.1 Cíl práce 13

2.2 Metodika 13

3 Teoretická východiska **14**

3.1 Sdílená ekonomika 14

3.1.1 Principy P2P 14

3.1.2 Environmentální aspekty a udržitelnost 15

3.1.3 Analýza existujících platforem 16

3.2 Webové technologie 17

3.2.1 Protokol HTTP 18

3.2.2 Architektura REST 19

3.3 HTML 20

3.4 CSS 21

3.4.1 Tailwind CSS 22

3.5 TypeScript 22

3.5.1 JavaScript a ECMAScript 23

3.6 Framework Next.js 24

3.6.1 React 25

3.6.2 Renderování (SSR/CSR) 26

3.7 Supabase 27

3.7.1 Backend-as-a-Service 28

3.7.2 Autentizace uživatelů 29

3.8 Relační databáze 30

3.8.1 Relační model 30

3.8.2 PostgreSQL 31

3.8.3 Jazyk SQL 32

3.9 UI a UX 33

3.9.1 Uživatelská přívětivost 34

3.9.2 Wireframe a prototyp 35

3.10 Optimalizace a SEO 36

3.10.1 Metadata 36

3.10.2 Core Web Vitals 37

3.11 Správa verzí a nasazení 38

3.11.1 Systém GIT 39

3.11.2 Vercel 40

3.12 Shrnutí teoretických východisek pro praktickou část 41

4 Vlastní práce 43

4.1 Analýza a specifikace požadavků 43

4.1.1 Cílová skupina a persony 43

4.1.2 Funkční požadavky 44

4.1.3 Nefunkční požadavky 46

4.2 Návrh aplikace 48

4.2.1 Design systém 48

4.2.2 Wireframy a informační architektura 51

4.3 Technický návrh a databáze 56

4.3.1 Tabulky a relace 56

4.3.2 Architektura projektu 64

4.4 Implementace projektu 67

4.4.1 Inicializace projektu a konfigurace Supabase 67

4.4.2 Autentizace a správa uživatelů 68

4.4.3 Katalog předmětů a filtrování 70

4.4.4 Detail předmětu a správa inzerátu 72

4.4.5 Rezervační systém 73

4.4.6 Chatovací systém 76

4.4.7 Notifikace a hodnocení 78

4.4.8 Uživatelský profil a správa výpůjček 80

4.5 Zabezpečení a optimalizace 82

4.5.1 Row Level Security 82

4.5.2 SEO optimalizace 84

4.6 Testování 85

4.6.1 Testovací scénáře 85

4.6.2 Uživatelské testování 86

4.6.3 Měření výkonu a SEO 89

4.7 Nasazení aplikace 90

5 Zhodnocení výsledků 91

6 Závěr 94

7 Seznam použitých zdrojů 95

8 Seznam obrázků, tabulek, a zkratek 100

8.1 Seznam obrázků 100

8.2 Seznam tabulek 100

8.3 Seznam zkratek 100

Přílohy ..... Chyba! Záložka není definována.

1 Úvod

V dnešní konzumní společnosti je běžným jevem hromadění předmětů, které jejich vlastníci využívají pouze sporadicky. Ať už jde o nářadí, sportovní vybavení nebo domácí potřeby, mnoho z těchto věcí leží většinu času ladem, zatímco by mohly sloužit ostatním lidem v blízkém okolí. Tento neefektivní model spotřeby vede nejen k zbytečným finančním nákladům jednotlivců, ale také k nadměrné produkci odpadu a zatěžování životního prostředí. Odpovědí na tento problém je koncept sdílené ekonomiky, který v posledních letech mění přístup k vlastnictví a upřednostňuje dočasný přístup k věcem.

Tato bakalářská práce se zaměřuje na analýzu, návrh a implementaci webové aplikace komunitní půjčovny. Hlavním cílem je vytvořit moderní a uživatelsky přívětivou platformu, která odstraní bariéry v lokálním půjčování a umožní sousedům efektivně sdílet své vybavení. Aplikace si klade za cíl podporovat principy udržitelnosti, snižovat potřebu nákupu nových věcí a přispívat k budování důvěry v rámci lokálních komunit. Důraz je kladen na jednoduchost celého procesu, od vyhledávání předmětu přes rezervaci až po samotné předání, což má motivovat uživatele k aktivnímu zapojení do systému sdílení.

Teoretická část se věnuje rešerši problematiky komunitního sdílení a analýze stávajících řešení na trhu. Dále podrobně popisuje zvolený technologický stack, který tvoří webový framework Next.js pro vývoj v jazyce TypeScript a backendová platforma Supabase. Pozornost je věnována také souvisejícím technologiím, jako jsou relační databáze PostgreSQL, jazyk SQL a principy moderního UI/UX designu.

Praktická část popisuje kompletní proces vývoje aplikace. Tento proces začíná analýzou požadavků a definováním cílové skupiny, pokračuje přes návrh uživatelského rozhraní formou wireframů a prototypů a vrcholí implementací klientské i serverové logiky. Součástí praktické části je také návrh databázového modelu, řešení autentizace uživatelů a implementace klíčových funkcí, jako je katalog předmětů, rezervační systém či real-time komunikace. Závěr práce je věnován testování funkčnosti, ověření použitelnosti vytvořeného řešení a zhodnocení dosažených výsledků

2 Cíl práce a metodika

2.1 Cíl práce

Hlavním cílem bakalářské práce je návrh a implementace webové aplikace pro komunitní půjčování předmětů, která umožní efektivní sdílení vybavení v rámci lokálních komunit. Aplikace bude navržena s důrazem na intuitivní uživatelské rozhraní a podporu principů udržitelnosti.

Dílčím cílem práce je analýza problematiky sdílené ekonomiky a prozkoumání moderních technologií pro vývoj, konkrétně frameworku Next.js a platformy Supabase. Součástí cílů je také vytvoření UI/UX návrhu a následné ověření funkčnosti aplikace formou testování.

2.2 Metodika

Pro zpracování práce bude postup rozdělen do teoretické a praktické části. Teoretická fáze se zaměří na studium odborné literatury týkající se P2P modelů a analýzu existujících platforem. Následovat bude nastudování technologického stacku, zahrnující jazyk TypeScript, framework Next.js a relační databáze.

Praktická část začne specifikací požadavků a tvorbou wireframů a prototypů. Na základě návrhu proběhne implementace, která bude zahrnovat vytvoření databázového modelu, backendové logiky a klientského rozhraní. Závěr práce bude věnován testování funkčnosti a zhodnocení výsledků.

3 Teoretická východiska

3.1 Sdílená ekonomika

Sdílená ekonomika (anglicky Sharing Economy) je ekonomický model, který umožňuje lidem sdílet majetek a služby. Podle studie společnosti Deloitte se jedná o model, ve kterém uživatelé, kteří něco potřebují, mohou využít aktiva těch, kteří mají v danou chvíli k dispozici a nevyužívají je [1]. Tento fenomén je často označován také pojmy jako společná spotřeba (collaborative consumption), kolaborativní ekonomika nebo vzájemná výměna a řadí se mezi nejvýznamnější ekonomické trendy 21. století [2].

Základním principem je efektivnější využití zdrojů. Mnoho statků, jako jsou automobily, byty nebo nářadí, leží většinu času ladem. Sdílená ekonomika umožňuje tuto takzvanou „nevyužitou kapacitu“ (idling capacity) aktivovat a nabídnout ji ostatním [2]. V praxi se tak děje prostřednictvím digitálních platforem, které propojují nabídku s poptávkou a snižují transakční náklady. Někdy se pro tento model používá přesnější označení „přístupová ekonomika“ (access economy), protože uživatelé si statky nekupují, ale platí pouze za přístup k nim po dobu, kdy je potřebují [1].

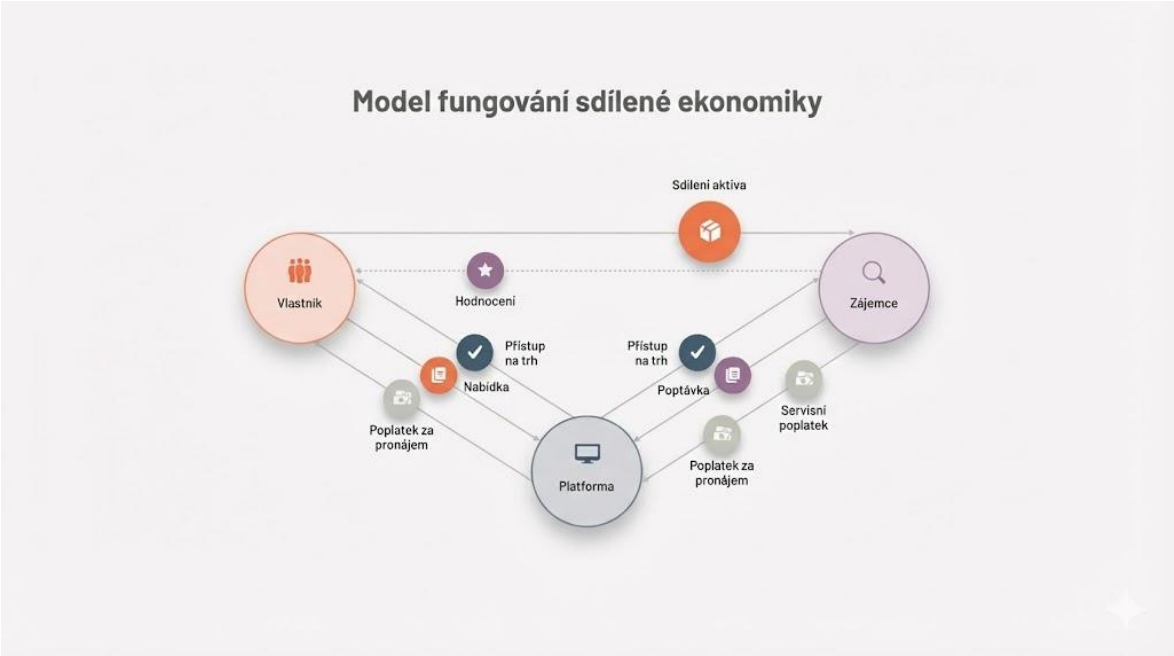
3.1.1 Principy P2P

Zkratka P2P vychází z anglického termínu Peer to Peer, který lze do češtiny přeložit jako „rovný s rovným“. Původ tohoto konceptu nalezneme v oblasti počítačových sítí, kde označuje architekturu, ve které jsou jednotliví účastníci (uzly sítě) rovnocenní a komunikují spolu přímo, bez nutnosti využití centrálního serveru pro distribuci úloh [4].

V ekonomickém modelu webových aplikací se tento princip projevuje přímým propojením dvou stran – uživatele, který službu či předmět nabízí, a uživatele, který ji poptává. Hlavní myšlenkou je vynechání tradičního prostředníka, jako jsou banky, klasické půjčovny nebo jiné centralizované instituce [3]. Díky odstranění tohoto mezičlánku dochází k zefektivnění procesu a často i k dosažení výhodnějších podmínek pro obě zúčastněné strany, ať už jde o nižší poplatky nebo flexibilitu [3].

Ačkoliv je podstata P2P založena na přímé interakci, v praxi je pro zajištění bezpečnosti a důvěry nezbytná existence digitální platformy. Tato platforma (webová

aplikace) však nevstupuje do vztahu jako poskytovatel služeb, ale plní roli administrátora [4]. Jejím úkolem je technicky zprostředkovat propojení nabídky s poptávkou, zajistit právní rámec a dohlížet na hladký průběh transakce [1].



Zdroj: Vlastní zpracování

3.1.2 Environmentální aspekty a udržitelnost

Jedním z hlavních motivů pro rozvoj sdílené ekonomiky je její pozitivní dopad na životní prostředí a udržitelnost. Koncept udržitelnosti nelze chápat úzce pouze jako ochranu přírody, ale jako komplexní systém stojící na třech pilířích: environmentálním, ekonomickém a sociálním [5]. Sdílená ekonomika tyto oblasti efektivně propojuje tím, že mění spotřebitelské chování směrem k odpovědnosti a šetrnosti.

V tradičním lineárním modelu ekonomiky často dochází k rychlému cyklu nákupu, krátkodobého využití a následného vyhození zboží. Oproti tomu sdílená ekonomika, úzce spjatá s principy cirkulární ekonomiky, usiluje o prodloužení životního cyklu výrobků a jejich maximální vytížení [1]. Pokud je jeden předmět, například nářadí v rámci komunitní půjčovny, využíván více uživateli, klesá celková poptávka po výrobě nových kusů. To vede k přímé úspoře primárních surovin, energie a vody, které by byly nutné pro výrobu a distribuci nových produktů [1].

Klíčovým faktorem pro environmentální přínos je aktivace takzvané „nevyužité kapacity“ (idling capacity). Mnoho předmětů v domácnostech není využíváno kontinuálně, ale leží většinu času ladem. Sdílení těchto již existujících aktiv zvyšuje jejich užitnou hodnotu bez nutnosti další produkce, čímž se efektivně snižuje množství generovaného odpadu a zmenšuje se uhlíková stopa spojená se spotřebou [2].

3.1.3 Analýza existujících platforem

Při analýze trhu sdílené ekonomiky v České republice lze pozorovat výrazný nepoměr mezi jednotlivými segmenty. Největšího rozvoje dosáhla oblast mobility a ubytování, kde se etablovaly globální platformy jako Airbnb nebo Uber, ale i silní lokální hráči. V oblasti sdílení automobilů (carsharing) fungují na českém trhu například služby Car4Way nebo HoppyGo. Právě HoppyGo je významným příkladem P2P modelu, kdy majitelé vozů (fyzické osoby) půjčují svá auta dalším uživatelům. Úspěch této platformy potvrzuje, že čeští uživatelé jsou ochotni sdílet i majetek vysoké hodnoty, pokud je proces zastřešen důvěryhodnou aplikací s vyřešeným pojištěním a identifikací uživatelů [1].

Dalším segmentem, který v tuzemsku prokázal životaschopnost principu „lidé lidem“, jsou P2P finance. Dominantním hráčem je zde platforma Zonky, která umožňuje lidem půjčovat peníze přímo žadatelům o úvěr bez nutnosti tradiční bankovní instituce. Tato služba prolomila psychologickou bariéru nedůvěry a ukázala, že při správném nastavení risk managementu a transparentním prostředí je P2P model funkční a vyhledávanou alternativou [3].

V přímé konkurenci pro navrhovanou aplikaci, tedy v oblasti půjčování drobného majetku a vybavení (nářadí, sportovní spotřeby, elektronika), je však situace odlišná. Na českém trhu se v minulosti objevila platforma SharyGo, která cílila přesně na tento segment. Ačkoliv služba získala mediální pozornost, narazila na bariéry s logistikou předání věcí a nastavením udržitelného obchodního modelu u transakcí s nižší hodnotou [2]. Kromě čistě digitálních řešení existují také takzvané Knihovny věcí (například Library of Things v Praze nebo Brně), které fungují na principu fyzického depozitáře. Tyto instituce sice efektivně podporují udržitelnost [5], jejich nevýhodou je však lokální omezení a nutnost fyzické návštěvy v provozní době.

Většina reálných výpůjček mezi lidmi se tak v současnosti odehrává mimo specializované aplikace, především na sociálních sítích. Největší neformální konkurencí jsou lokální sousedské skupiny na Facebooku nebo služba Facebook Marketplace. Tyto nástroje sice disponují obrovskou uživatelskou základnou, ale nejsou pro půjčování uzpůsobeny. Chybí zde rezervační systém, automatizované generování smluv či správa kaucí. Celý proces je tak založen pouze na neformální domluvě, což přináší rizika a administrativní zátěž [4]. Analýza trhu tedy ukazuje, že zatímco sdílení aut (HoppyGo) či peněz (Zonky) má svá silná řešení, oblast komunitního půjčování věcí postrádá moderní nástroj, který by kombinoval bezpečnost s uživatelskou přívětivostí.

3.2 Webové technologie

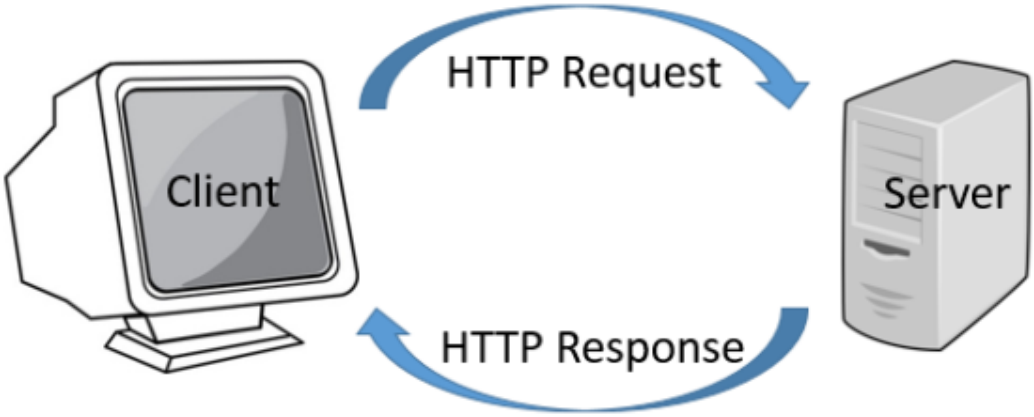
Oblast webových technologií prošla v posledních desetiletích dynamickým vývojem, který zásadně změnil způsob, jakým lidé využívají internet. Původní podoba webu, často označovaná jako Web 1.0, byla charakteristická statickými stránkami. Uživatelé v této éře obsah pouze pasivně konzumovali a možností interakce byly minimální. Jednalo se primárně o propojené dokumenty bez složitější aplikační logiky [6].

S nástupem Webu 2.0 se situace změnila a internet se stal platformou pro spolupráci a sdílení. Stránky se staly dynamickými a umožnily uživatelům obsah nejen číst, ale také jej vytvářet a upravovat. Tento posun vedl ke vzniku sociálních sítí, blogů a komplexních webových aplikací, jaké známe dnes [6]. Současné moderní webové technologie se tak již nesoustředí pouze na zobrazení textu, ale na poskytování plnohodnotného uživatelského zážitku, který je srovnatelný s desktopovými programy.

Z technického hlediska je fungování webu založeno na architektuře klient-server. Tato architektura rozděluje systém na dvě nezávislé části, které spolu komunikují přes síť. Na jedné straně stojí klient (webový prohlížeč), který iniciuje komunikaci odesláním požadavku. Na druhé straně se nachází server, který tento požadavek zpracuje – například vyhledá informace v databázi – a odešle klientovi zpět odpověď ve formě HTML kódu, stylů nebo dat [7]. Aby si obě strany rozuměly, musí tato komunikace probíhat podle přesně stanovených pravidel a protokolů, z nichž nejvýznamnější je protokol HTTP [7].

3.2.1 Protokol HTTP

Tento protokol, HTTP (Hypertext Transfer Protocol), je základním stavebním kamenem komunikace na síti World Wide Web. Jedná se o protokol aplikační vrstvy, který definuje soubor pravidel pro výměnu zpráv mezi klientem (např. webovým prohlížečem) a serverem [8]. Celá komunikace probíhá striktně na principu dotaz-odpověď. Klient nejprve sestaví a odešle požadavek (request) na konkrétní zdroj a server následně tento požadavek zpracuje a odešle zpět odpověď (response), která obsahuje požadovaná data, hlavičky a stavový kód [7].



Zdroj: https://www.innovationm.com/blog/http-protocol/

Klíčovou charakteristikou protokolu HTTP je jeho bezstavovost. To v praxi znamená, že server si mezi jednotlivými požadavky neuchovává žádné informace o předchozí komunikaci s daným klientem. Každý dotaz je serverem chápán jako zcela nový a nezávislý na těch předchozích [6]. Tento design zjednodušuje architekturu serveru, avšak pro potřeby moderních webových aplikací, které vyžadují například udržení přihlášení uživatele, musí být bezstavovost obcházena pomocí dodatečných mechanismů, jako jsou cookies nebo sessions.

Pro specifikaci zamýšlené akce se zdrojem využívá protokol HTTP různé metody, často označované jako slovesa. Mezi nejčastěji používané patří metoda GET, která slouží k získání dat ze serveru (například načtení HTML stránky), a metoda POST, jež je určena pro odesílání dat směrem na server (například při odeslání kontaktního formuláře) [8]. Vývoj protokolu neustále pokračuje; moderní verze jako HTTP/2 přinášejí významná vylepšení

v oblasti výkonu, například kompresi hlaviček nebo tzv. multiplexing, který umožňuje přenášet více požadavků v rámci jednoho spojení [8].

Nedílnou součástí každé odpovědi serveru je stavový kód, který informuje klienta o výsledku zpracování požadavků. Tyto trojciferné kódy jsou rozděleny do logických skupin podle první číslice. Kódy začínající dvojkou (např. 200 OK) značí úspěšné vyřízení, zatímco trojkové kódy indikují přesměrování. Pro vývojáře jsou klíčové kódy chybové: řada 4xx signalizuje chybu na straně klienta (například známá chyba 404 Not Found) a řada 5xx označuje selhání na straně serveru [8]. Správná implementace těchto kódů je nezbytná pro korektní fungování REST API rozhraní.

3.2.2 Architektura REST

Zkratka REST označuje Representational State Transfer. Jedná se o architekturu softwarového rozhraní pro distribuované prostředí, která je orientována na data, nikoli na volání procedur, jako je tomu u starších standardů typu RPC nebo SOAP [10]. Hlavním cílem REST je definovat strukturu API tak, aby byla srozumitelná a univerzálně použitelná pro různé aplikace a konzumenty dat. Aby bylo možné lépe pochopit principy této architektury, lze ji rozdělit do čtyř úrovní tak, jak je definoval Leonard Richardson:

1. **Nultá úroveň (Transport)**: Jedná se o samotný přesun dat. Ačkoliv se REST teoreticky neváže na jeden konkrétní protokol, v drtivé většině případů se využívá protokol HTTP, který je celosvětově rozšířený a řeší mnoho technických aspektů přenosu [10].
2. **První úroveň (Zdroje)**: Tato úroveň zavádí pojem zdroj (resource). Místo toho, aby všechny požadavky směřovaly na jeden centrální bod, má každý datový objekt (například článek, uživatel) svou vlastní unikátní adresu URL. Například požadavek na */articles* vrátí seznam článků, zatímco */articles/1* zpřístupní konkrétní článek [10].
3. **Druhá úroveň (HTTP slovesa)**: Na této úrovni se začínají aktivně využívat metody protokolu HTTP (tzv. HTTP Verbs) k určení akce, která se má se zdrojem provést. Metoda GET slouží pro čtení, POST pro vytváření, PUT pro úpravy a DELETE pro mazání dat [10].
4. **Třetí úroveň (HATEOAS)**: Nejvyšší stupeň REST API, kde odpověď serveru obsahuje nejen data, ale i hypertextové odkazy na další související data, což klientovi umožňuje dynamicky procházet rozhraním [10].

V praxi je moderní REST API často založeno na formátu JSON, ve kterém jsou na jednotlivých koncových bodech publikovány informace [9]. Důležitou součástí komunikace jsou návratové stavové kódy, které informují klienta o výsledku operace. Například kód 200 značí úspěšné volání, kód 400 indikuje neplatný požadavek (chyba validace parametrů) a kód 500 upozorňuje na neočekávanou chybu na straně serveru [9]. Správně navržené REST API je tak klíčovým prvkem pro propojení backendové logiky s klientskou aplikací.

3.3 HTML

Zkratka HTML (HyperText Markup Language) označuje standardní značkovací jazyk určený pro tvorbu dokumentů zobrazovaných ve webových prohlížečích. Nejedná se o programovací jazyk v pravém slova smyslu, neboť neobsahuje logické řídící struktury, ale slouží primárně k definici struktury a významu obsahu na webové stránce [11]. Technologie HTML je obvykle využívána v kombinaci s kaskádovými styly (CSS) pro vzhled a skriptovacím jazykem Javascript pro funkcionalitu.

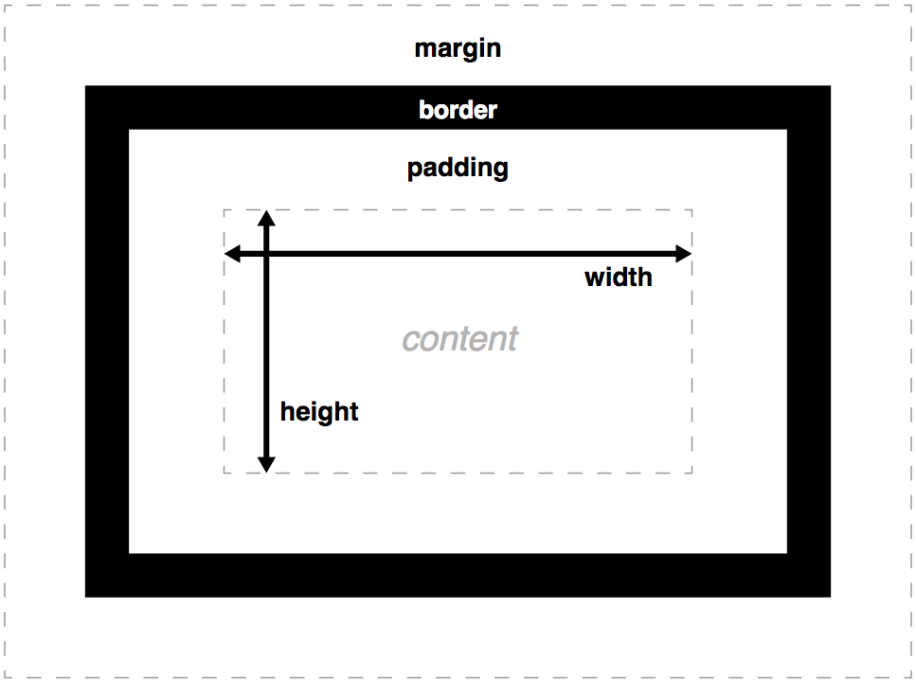
Základní stavební jednotkou jazyka jsou tzv. elementy. Ty jsou v kódu vyjádřeny pomocí značek (tagů), které se zapisují do lomených závorek. Standardní element se skládá z otevírací značky, obsahu a uzavírací značky, přičemž uzavírací značka obsahuje lomítko [12]. Příkladem může být odstavec textu ohraničený značkami * * a * *. Elementy mohou být do sebe libovolně vnořovány, čímž vzniká hierarchická struktura stránky [11].

Součástí otevírací značky mohou být také atributy, které elementu dodávají dodatečné informace nebo konfiguraci. Atributy se zapisují ve formátu *název="hodnota"*. Typickým využitím je například atribut class pro stylování nebo atribut src u obrázku, který definuje cestu k souboru [11].

Webový prohlížeč při načtení HTML dokumentu nezobrazuje samotné značky, ale interpretuje je a na jejich základě vykreslí výslednou stránku. Interně prohlížeč převádí HTML kód do struktury zvané DOM (Document Object Model), což je stromová reprezentace dokumentu, se kterou mohou dále pracovat skripty [11]. Moderní verze jazyka, HTML5, klade velký důraz na sémantiku. To znamená využívání elementů, které přesně popisují význam obsahu (např. *<header>, <article>, <footer>*) namísto generických kontejnerů, což je klíčové pro přístupnost a SEO [11].

3.4 CSS

Zkratka CSS (Cascading Style Sheets) označuje jazyk kaskádových stylů, který slouží k popisu prezentace dokumentu napsaného ve značkovacím jazyce, nejčastěji HTML. Nejedná se o programovací jazyk ani o značkovací jazyk, ale o tzv. jazyk šablon stylů (style sheet language) [13]. Jeho hlavním účelem je striktní oddělení obsahu (sémantiky a struktury) od formy (vzhledu), což umožňuje výrazně snadnější údržbu kódu a sdílení



stejného designu napříč více stránkami webu [13].

Zdroj: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_model

Zápis stylu se skládá z pravidel. Každé pravidlo začíná selektorem, který určuje, na které HTML elementy se má styl aplikovat (například na všechny nadpisy *h1*). Následuje blok deklarací uzavřený ve složených závorkách, kde jsou definovány konkrétní dvojice vlastnost: hodnota (například color: blue pro změnu barvy písma) [14].

Termín „kaskádové“ odkazuje na způsob, jakým prohlížeč určuje prioritu aplikovaných stylů v případě, že na jeden element cílí více definic současně. Výsledný vzhled je vypočítán na základě pravidel specificity selektorů, dědičnosti a pořadí v kódu [13]. V praxi se styly nejčastěji umisťují do externích souborů s příponou *.css*, které se na HTML dokument napojují v hlavičce stránky. Tento přístup je efektivnější než přímý zápis

stylů do HTML kódu, protože umožňuje změnit vzhled celého webu úpravou jediného souboru [14].

3.4.1 Tailwind CSS

Tailwind CSS je moderní „utility-first“ CSS framework, který slouží k rychlé tvorbě uživatelských rozhraní přímo v HTML kódu. Na rozdíl od tradičních frameworků, které často nabízejí sadu předpřipravených komponent (tlačítek, navigačních lišt), poskytuje Tailwind pouze nízkoúrovňové nástroje – tzv. utility classes [15]. Tyto třídy, jako například *flex, pt-4* nebo *text-center*, reprezentují konkrétní CSS vlastnosti a umožňují vývojáři skládat design elementu bez nutnosti psát vlastní CSS kód v externím souboru [16].

Hlavní předností tohoto přístupu je vysoká efektivita vývoje a snadná udržitelnost kódu, protože odpadá nutnost vymýšlet názvy vlastních tříd pro každý element. Framework je navíc plně konfigurovatelný; pomocí konfiguračního souboru lze definovat vlastní barevnou paletu, fonty či velikosti mezer, čímž lze snadno vytvořit unikátní design systém na míru konkrétní aplikaci [16].

3.5 TypeScript

TypeScript je open-source programovací jazyk vyvíjený a udržovaný společností Microsoft. Jedná se o striktní syntaktickou nadmnožinu (superset) jazyka JavaScript, což znamená, že jakýkoliv validní kód napsaný v JavaScriptu je zároveň validním kódem v TypeScriptu. Hlavním přínosem tohoto jazyka je přidání volitelného statického typování do jinak dynamického prostředí JavaScriptu [19]. Protože webové prohlížeče a běhová prostředí (jako Node.js) neumí TypeScript přímo vykonávat, musí být tento kód před spuštěním převeden (tzv. transpilován) do standardního JavaScriptu (ECMAScript) pomocí kompilátoru [19].

Zásadním rozdílem oproti čistému JavaScriptu je systém typové kontroly. JavaScript je dynamicky typovaný jazyk, kde se typy proměnných určují až za běhu programu (runtime). To často vede k chybám, které jsou odhaleny až v momentě, kdy aplikaci používá uživatel [18]. TypeScript tento problém řeší zavedením statické analýzy kódu. Vývojář může explicitně definovat datové typy (např. *string, number, boolean*) pro proměnné, parametry funkcí i návratové hodnoty. Pokud dojde k neshodě (například pokus o sečtení čísla a

textového řetězce), kompilátor upozorní na chybu již během psaní kódu (compile-time), nikoliv až při jeho spuštění [18].

Z technického hlediska nabízí TypeScript pokročilé prvky objektově orientovaného programování (OOP), které usnadňují vývoj rozsáhlých a škálovatelných aplikací. Klíčovým konceptem jsou rozhraní (Interfaces), která definují „smlouvu“ (contract) pro strukturu objektů. Rozhraní určuje, jaké vlastnosti a metody musí daný objekt obsahovat, aniž by definovalo jejich konkrétní implementaci [19]. Dále jazyk podporuje generické typy (Generics), které umožňují vytvářet znovupoužitelné komponenty schopné pracovat s různými datovými typy při zachování typové bezpečnosti [19].

Použití TypeScriptu výrazně zlepšuje vývojářskou zkušenost (Developer Experience). Díky statické analýze mohou moderní IDE (jako Visual Studio Code) poskytovat pokročilé funkce, jako je inteligentní napovídání kódu (IntelliSense), automatické doplňování, bezpečný refaktoring a snadná navigace v definicích [17]. Ačkoliv počáteční konfigurace a psaní typových definic může mírně zvýšit časovou náročnost na začátku projektu, z dlouhodobého hlediska vede k vyšší stabilitě kódu a snazší údržbě [18].

3.5.1 JavaScript a ECMAScript

JavaScript (často zkracováno jako JS) je vysokoúrovňový, dynamicky typovaný programovací jazyk, který spolu s HTML a CSS tvoří tři základní pilíře moderního webu. Původně byl navržen jako jednoduchý skriptovací jazyk pro oživení webových stránek v prohlížeči, avšak dnes se jedná o plnohodnotný multi-paradigmatický jazyk podporující objektově orientované, imperativní i funkcionální styly programování [20]. Díky své univerzálnosti a rozšíření již není omezen pouze na klientskou část aplikací (Frontend), ale prostřednictvím prostředí, jako je Node.js, se masivně využívá i na serverové straně (Backend) či při vývoji desktopových a mobilních aplikací [20].

Pro formální ukotvení jazyka je klíčový pojem ECMAScript. Zatímco JavaScript je název samotného programovacího jazyka (Implementace), ECMAScript je oficiální specifikace (Standard), kterou definuje organizace Ecma International, konkrétně technická komise TC39 [22]. Tato specifikace (označována jako ECMA-262) určuje syntaxi, typy, objekty a základní pravidla jazyka, která musí dodržovat tvůrci webových prohlížečů a běhových prostředí. Významným milníkem byla verze ECMAScript 2015 (známá jako

ES6), která přinesla zásadní modernizaci syntaxe (třídy, moduly, arrow funkce). Od té doby vychází nová verze standardu s vylepšeními pravidelně každý rok [22].

Z technického hlediska je JavaScript často označován jako interpretovaný jazyk, moderní enginy (jako je V8 v prohlížeči Chrome nebo SpiderMonkey ve Firefoxu) však využívají technologii JIT kompilace. To znamená, že zdrojový kód je za běhu analyzován a kompilován do strojového kódu, což výrazně zvyšuje výkon aplikací [21]. Na rozdíl od nízkoúrovňových jazyků je JavaScript spouštěn v bezpečném prostředí (Sandbox), které zamezuje přímému přístupu k paměti nebo operačnímu systému zařízení uživatele, čímž je zajištěna bezpečnost při prohlížení webových stránek [21].

3.6 Framework Next.js

Next.js je komplexní open-source webový framework postavený na knihovně React, který je vyvíjen a udržován společností Vercel. Zatímco samotný React je knihovna určená primárně pro tvorbu uživatelských rozhraní na straně klienta, Next.js rozšiřuje jeho možnosti o nástroje nezbytné pro tvorbu plnohodnotných produkčních aplikací. Poskytuje strukturované prostředí, které řeší běžné vývojářské výzvy, jako je optimalizace výkonu, routování nebo konfigurace kompilace, aniž by vývojář musel tyto aspekty nastavovat manuálně. [23].

Klíčovou vlastností frameworku je jeho flexibilita v oblasti renderování obsahu (vykreslování stránky). Na rozdíl od standardních Single Page Applications, které spoléhají na vykreslování v prohlížeči uživatele (Client-Side Rendering), umožňuje Next.js využívat vykreslování na straně serveru (Server-Side Rendering) a generování statických stránek (Static Site Generation) [25]. Při SSR je HTML kód stránky vygenerován na serveru pro každý požadavek zvlášť, zatímco u SSG dochází k generování obsahu již v době sestavování aplikace (build time). Tyto metody zásadně zlepšují SEO, protože vyhledávací roboti dostávají již hotový HTML obsah, a zároveň zrychlují první načtení stránky (First Contentful Paint) [25].

Z hlediska architektury využívá Next.js intuitivní systém směrování založený na souborovém systému (file-system based routing). V moderních verzích frameworku byl představen tzv. App Router, který je postaven na nejnovějších funkcích Reactu, konkrétně na React Server Components [24]. Tato architektura umožňuje spouštět komponenty přímo na serveru, což snižuje objem JavaScriptu odesílaného do prohlížeče uživatele. Framework

dále automaticky optimalizuje obrázky, fonty a skripty, čímž zajišťuje vysoké skóre v metrikách Core Web Vitals, které jsou klíčové pro uživatelskou přívětivost i hodnocení vyhledávači [24].

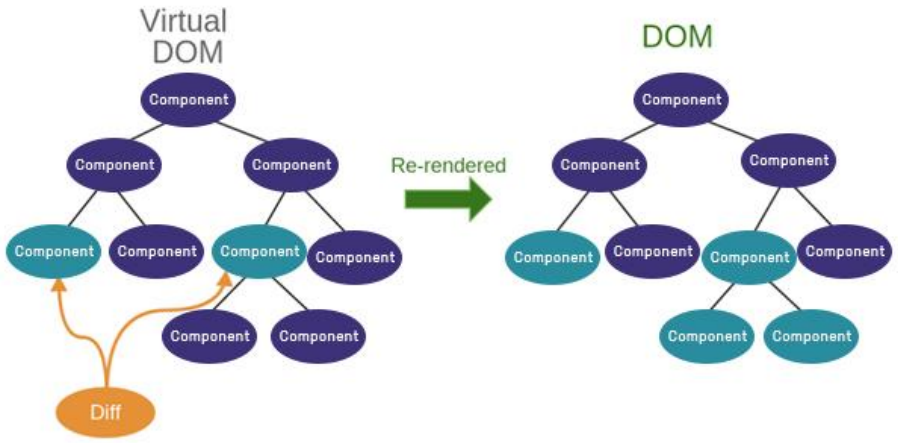
3.6.1 React

React je deklarativní, efektivní a flexibilní JavaScriptová knihovna určená pro vytváření uživatelských rozhraní (UI). Byla vyvinuta společností Facebook (nyní Meta) a v současnosti patří mezi nejrozšířenější nástroje pro vývoj frontendových aplikací. Na rozdíl od komplexních frameworků (jako je Angular) se React profiluje primárně jako knihovna zaměřená na „View“ vrstvu aplikace, což vývojářům poskytuje volnost při výběru dalších nástrojů pro směrování nebo správu stavu [26].

Základním stavebním kamenem Reactu jsou komponenty. Jedná se o malé, izolované a znovupoužitelné části kódu, které v sobě zapouzdřují vlastní logiku i vzhled. Z těchto komponent se následně skládají složitější uživatelská rozhraní. Pro zápis struktury komponent se využívá syntaxe zvaná JSX (JavaScript XML), která umožňuje psát kód připomínající HTML přímo uvnitř JavaScriptu. To sice vyžaduje transpilaci kódu (např. Babel), ale výrazně zpřehledňuje propojení logiky s vizuální stránkou [27].

Z technického hlediska je klíčovou vlastností Reactu využití tzv. Virtuálního DOMu. Přímá manipulace s reálným DOMem prohlížeče je výpočetně náročná a pomalá. React proto udržuje v paměti lehkou kopii DOMu. Jakmile dojde ke změně stavu aplikace, React vytvoří nový virtuální strom, porovná jej s předchozím (proces zvaný „diffing“) a následně provede v reálném DOMu pouze nezbytně nutné změny (proces „reconciliation“). Tento přístup zajišťuje vysoký výkon i u složitých a dynamických aplikací [26].

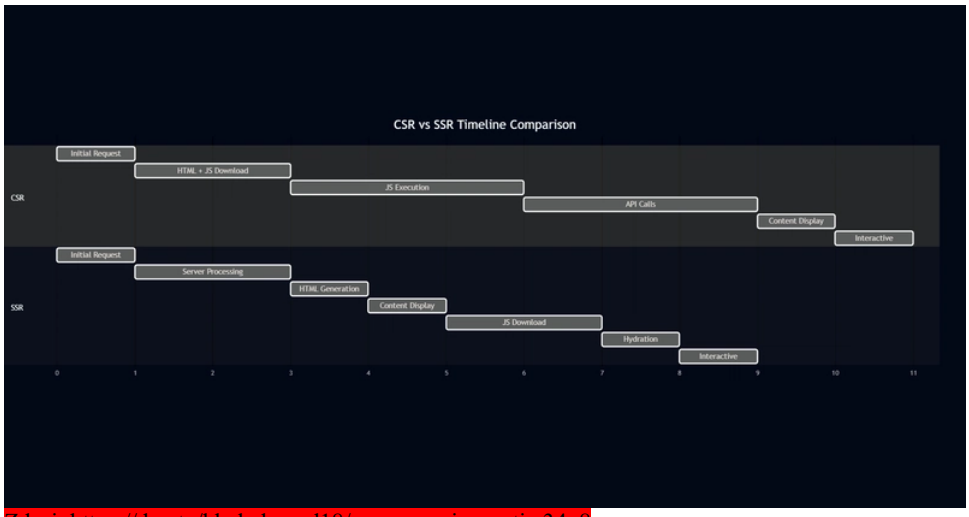
Důležitým aspektem je také jednosměrný tok dat (one-way data flow). Data v Reactu proudí od rodičovských komponent k potomkům pomocí tzv. „props“ (vlastností). To činí aplikaci předvídatelnou a usnadňuje ladění chyb, protože je vždy jasné, odkud data pocházejí a jakým způsobem ovlivňují vykreslování [27].



Zdroj: https://cashapona.com/2019/04/10/the-significance-of-virtual-dom-in-reactjs/

3.6.2 Renderování (SSR/CSR)

Pojem renderování v kontextu webových aplikací označuje proces, při kterém je kód aplikace (šablony a data) převeden na výsledný HTML dokument zobrazitelný v prohlížeči. Existují dva základní přístupy k tomuto procesu, které se liší tím, kde k výpočtu vizuální podoby dochází: na straně klienta (Client-Side Rendering) nebo na straně serveru (Server-Side Rendering). Volba mezi těmito metodami má zásadní dopad na výkon aplikace, uživatelskou zkušenost a optimalizaci pro vyhledávače [28].



Zdroj: https://dev.to/khaledsaeed18/csr-vs-ssr-in-nextjs-34o8

Client-Side Rendering (CSR) je metoda typická pro moderní jednostránkové aplikace (SPA). Při prvním požadavku obdrží prohlížeč od serveru pouze minimální HTML kostru (často prázdný element <*div*>) a odkaz na JavaScriptový soubor. Veškeré vykreslování obsahu a logiky probíhá až následně v prohlížeči uživatele pomocí JavaScriptu, který manipuluje s DOMem [28]. Hlavní výhodou je rychlá interaktivita po prvním načtení a snížení zátěže serveru. Nevýhodou je pomalejší prvotní zobrazení (uživatel musí čekat na stažení a spuštění skriptů) a potenciální problémy s indexací obsahu vyhledávacími roboty, kteří nemusí JavaScript správně interpretovat.

Server-Side Rendering (SSR) naproti tomu přenáší výpočetní zátěž na server. Při požadavku na stránku server vygeneruje kompletní HTML dokument včetně obsahu a odešle jej do prohlížeče. Uživatel tak vidí obsah stránky prakticky okamžitě, aniž by musel čekat na stažení JavaScriptu [29]. Tento přístup je klíčový pro SEO, protože vyhledávače dostávají plně čitelný textový obsah. Nevýhodou může být vyšší zátěž serveru (musí generovat stránku pro každý požadavek) a mírně delší doba odezvy serveru (Time to First Byte), než se HTML vygeneruje [29]. Moderní frameworky jako Next.js často využívají hybridní přístup, kdy se stránka poprvé vykreslí pomocí SSR pro rychlé zobrazení a následně se „oživí“ (proces zvaný „hydratace“) na klientovi pro plnou interaktivitu.

3.7 Supabase

Supabase je moderní open-source platforma typu Backend-as-a-Service (BaaS), která poskytuje vývojářům kompletní sadu nástrojů pro tvorbu backendové infrastruktury bez nutnosti spravovat vlastní servery. Často je prezentována jako otevřená alternativa k populární službě Google Firebase, avšak se zásadním architektonickým rozdílem: zatímco Firebase využívá proprietární NoSQL databázi, Supabase je postavena na standardní relační databázi PostgreSQL [30]. To vývojářům umožňuje využívat silnou typovou kontrolu, složité SQL dotazy a relace mezi daty, což jsou vlastnosti klíčové pro škálovatelné podnikové aplikace.

Z technického hlediska funguje Supabase jako sada integrovaných open-source nástrojů. Jádrem je zmíněná databáze PostgreSQL. Platforma využívá nástroj PostgREST, který automaticky analyzuje databázové schéma a okamžitě generuje dokumentované a

bezpečné RESTful API pro komunikaci s frontendem. To eliminuje nutnost manuálního psaní CRUD (Create, Read, Update, Delete) endpointů a výrazně urychluje vývoj [31].

Ekosystém Supabase dále zahrnuje modul pro autentizaci (Auth), který podporuje přihlašování e-mailem, magickými odkazy i skrze poskytovatele třetích stran (Google, GitHub, Apple). Bezpečnost dat je řešena přímo na úrovni databáze pomocí politik Row Level Security. Tyto politiky určují, který uživatel má právo číst nebo zapisovat konkrétní řádky v tabulce, což zajišťuje konzistentní zabezpečení bez ohledu na to, odkud k datům přistupujeme [30]. Mezi další funkce patří úložiště souborů (Storage), Edge Functions pro spouštění serverového kódu a Realtime engine, který umožňuje aplikacím odebírat změny v databázi v reálném čase [31].

3.7.1 Backend-as-a-Service

BaaS je cloudový model služeb, který vývojářům umožňuje outsourcovat veškeré zákulisní aspekty webové nebo mobilní aplikace, aby se mohli soustředit výhradně na psaní a údržbu frontendu. Poskytovatelé BaaS nabízejí předem vytvořený software pro činnosti, které probíhají na serverech, jako je správa databází, autentizace uživatelů, vzdálené aktualizace, push notifikace a cloudové úložiště [32].

Tento model lze přirovnat k filmovému režisérovi, který se díky existenci specializovaných týmů pro osvětlení, stavbu kulis a kostýmy může soustředit pouze na samotnou akci na scéně. Stejně tak se vývojář v BaaS modelu nemusí starat o správu serverů, virtuálních strojů nebo kontejnerů, což mu umožňuje rychleji vytvářet a spouštět aplikace. Komunikace s těmito službami probíhá prostřednictvím aplikačních rozhraní (API) a vývojových sad (SDK) nabízených dodavatelem [32].

Mezi hlavní výhody BaaS patří:

1. **Zrychlení vývoje**: Odpadá nutnost budovat vlastní backendovou infrastrukturu od základu. [32]
2. **Snížení nákladů**: Vývojáři platí pouze za využité prostředky a šetří čas, který by jinak věnovali údržbě serverů. [32]

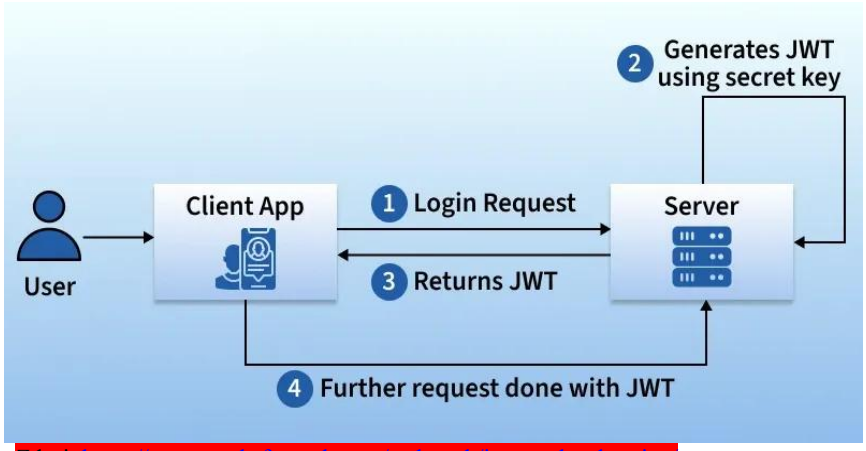
3. **Soustředění na UX**: Veškerá energie vývojového týmu může směřovat k tomu, co koncový uživatel vidí a zažívá. [32]

BaaS se liší od modelu Platform-as-a-Service (PaaS) tím, že poskytuje již hotovou serverovou logiku (např. autentizaci), zatímco PaaS nabízí pouze platformu, na které si vývojář musí tuto logiku sám naprogramovat. Od čistě serverless architektury se pak liší tím, že aplikace postavené na BaaS nemusí být nutně řízeny událostmi (event-driven) a automatické škálování závisí na konkrétním poskytovateli a jeho implementaci [32].

3.7.2 Autentizace uživatelů

Autentizace je proces ověření identity uživatele (kdo je), zatímco „autorizace určuje, k jakým zdrojům má tento ověřený uživatel přístup (co může dělat). V tradičních webových aplikacích musí vývojáři často budovat vlastní autentizační servery, spravovat relace (sessions) a bezpečně ukládat hesla. Supabase tento proces zjednodušuje pomocí integrované služby Supabase Auth, která je postavena na open-source serveru GoTrue [33].

Supabase Auth funguje na principu JSON Web Tokenů (JWT). Když se uživatel úspěšně přihlásí (např. e-mailem a heslem, magickým odkazem nebo přes OAuth poskytovatele jako Google či GitHub), služba mu vydá přístupový token (Access Token). Tento token je následně odesílán s každým požadavkem na databázi. Na rozdíl od běžných řešení, kde API server musí ověřovat každého uživatele, Supabase předává tento token přímo do databáze PostgreSQL. Databáze sama pak token dekóduje a identifikuje uživatele, což výrazně snižuje latenci a zjednodušuje architekturu backendu [33].



Zdroj: https://www.geeksforgeeks.org/web-tech/json-web-token-jwt/

Klíčovým prvkem zabezpečení v ekosystému Supabase je propojení autentizace s mechanismem Row Level Security (RLS). RLS umožňuje definovat bezpečnostní politiky přímo v SQL, které určují přístup k datům na úrovni jednotlivých řádků. Typická politika může například povolit čtení dat pouze v případě, že ID uživatele v tokenu (JWT) se shoduje s ID vlastníka záznamu v tabulce (*auth.uid() = user_id*). Tím je zajištěno, že i kdyby útočník obešel klientskou aplikaci a přistupoval k API přímo, databáze mu nevydá žádná data, ke kterým nemá oprávnění [34].

3.8 Relační databáze

Relační databáze je typ databáze, který ukládá a zpřístupňuje data, jež mají mezi sebou definované vztahy. Tento model, který v roce 1970 představil E. F. Codd, organizuje data do tabulek skládajících se z řádků a sloupců [36]. Každý řádek v tabulce představuje záznam s unikátním identifikátorem, zvaným primární klíč (Primary Key), zatímco sloupce obsahují jednotlivé atributy dat. Vztahy mezi tabulkami jsou následně realizovány pomocí cizích klíčů (Foreign Keys), což umožňuje efektivní propojování logických vazeb napříč celou databází [36].

Pro komunikaci s relačními databázemi se standardně využívá jazyk SQL (Structured Query Language), který slouží k vytváření, ukládání, aktualizaci a načítání dat. Zásadní vlastnosti relačních systémů je důraz na integritu a přesnost dat. Ta je v podnikovém prostředí často zajištěna dodržováním tzv. ACID principů (Atomicita, Konzistence, Izolace, Trvanlivost). Tyto vlastnosti garantují, že databázové transakce jsou zpracovány spolehlivě a že data zůstanou konzistentní i v případě výpadku systému nebo jiné chyby [35].

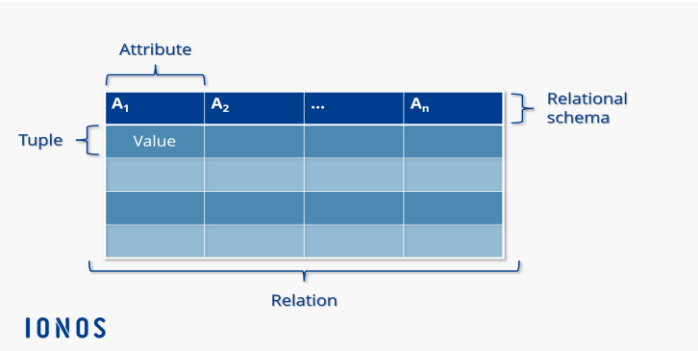
3.8.1 Relační model

Relační model dat, který tvoří teoretický základ relačních databází, vychází z matematické teorie množin. V tomto pojetí nejsou data vnímána jako fyzické soubory, ale jako abstrakce zvané relace. Každá relace se skládá ze záhlaví (schématu), které popisuje strukturu dat, a těla, které obsahuje samotná data ve formě n-tic [37].

Z formálního hlediska je relace definována jako podmnožina kartézského součinu domén (množin povolených hodnot). Základními stavebními kameny modelu jsou:

1. **Relace (Relation)**: Celá tabulka reprezentující určitou entitu (např. Student)

2. **Atribut (Attribute)**: Sloupec tabulky, který představuje konkrétní vlastnost (např. Jméno, Datum narození). Každý atribut má definovanou doménu hodnot.
3. **N-tice (Tuple)**: Řádek tabulky, který představuje jeden konkrétní záznam (instanci entity) [37].



Zdroj: https://www.ionos.ca/digitalguide/hosting/technical-matters/relational-databases

Důležitou vlastností relačního modelu je, že na pořadí řádků ani sloupců v relaci nezáleží (jelikož jde o množinu, nikoliv o seřazený seznam). Každá hodnota v buňce tabulky musí být atomická, tedy dále nedělitelná [37].

3.8.2 PostgreSQL

PostgreSQL je pokročilý open-source objektově-relační databázový systém (ORDBMS), který je vyvíjen globální komunitou více než 35 let. Jeho kořeny sahají do roku 1986 na univerzitu v Berkeley, kde profesor Michael Stonebraker zahájil projekt POSTGRES jako nástupce dřívějšího systému INGRES [38]. V roce 1994 byla do systému přidána podpora jazyka SQL, čímž vznikl základ pro dnešní podobu PostgreSQL [38].

Na rozdíl od čistě relačních databází podporuje PostgreSQL tzv. objektově-relační přístup. To znamená, že kromě tradičních tabulek a vztahů dokáže pracovat i s komplexními datovými typy, jako je JSON (JavaScript Object Notation), a umožňuje uživatelům definovat vlastní datové typy a funkce [38]. Díky této flexibilitě je PostgreSQL často označován za „nejpokročilejší open-source databázi na světě“ [39].

Klíčovou vlastností architektury PostgreSQL je implementace souběžného řízení přístupu pomocí mechanismu MVCC (Multiversion Concurrency Control). V praxi to znamená, že čtení dat neblokuje jejich zápis a naopak. Každá transakce vidí konzistentní snímek databáze z doby, kdy začala, což zajišťuje vysoký výkon i v prostředích s velkým

množstvím současně přístupujících uživatelů [38]. Systém je plně kompatibilní s principy ACID a nabízí širokou podporu pro různé programovací jazyky včetně Pythonu, Javy či C++ [39].

3.8.3 Jazyk SQL

SQL je standardizovaný doménově specifický jazyk určený pro správu dat v relačních databázových systémech (RDBMS) a pro zpracování datových toků. Původně byl vyvinut v 70. letech 20. století výzkumníky společnosti IBM pod názvem SEQUEL, později byl standardizován organizacemi ANSI (1986) a ISO (1987) [40].

Na rozdíl od většiny procedurálních programovacích jazyků je SQL jazykem deklarativním. To znamená, že uživatel specifikuje „co“ chce z databáze získat (např. seznam všech uživatelů starších 18 let), aniž by musel popisovat přesný algoritmus „jak“ má systém data fyzicky vyhledat a načíst. Optimalizaci provádění dotazu zajišťuje samotný databázový stroj [41].

Příkazy jazyka SQL se dělí do několika hlavních podskupin podle jejich účelu:

1. **Data Manipulation Language (DML)**: Slouží k manipulaci se samotnými daty. Nejčastěji používanými příkazy jsou *SELECT* (výběr dat), *INSERT* (vložení), *UPDATE* (aktualizace) a *DELETE* (smazání) [40].
2. **Data Definition Language (DDL)**: Používá se k definování a úpravě struktury databáze (schématu). Patří sem příkazy pro vytváření tabulek (*CREATE)*, jejich změnu (*ALTER)* nebo odstranění (*DROP)* [40].
3. **Data Control Language (DCL)**: Zajišťuje správu přístupových práv a zabezpečení. Pomocí příkazů *GRANT* a *REVOKE* administrátor určuje, kteří uživatelé mají oprávnění číst nebo měnit konkrétní data [40].
4. **Transaction Control Language (TCL)**: Slouží k řízení transakcí v databázi. Klíčovými příkazy jsou COMMIT (potvrzení transakce), ROLLBACK (vrácení změn) a SAVEPOINT (vytvoření záchytného bodu v rámci transakce) [40].

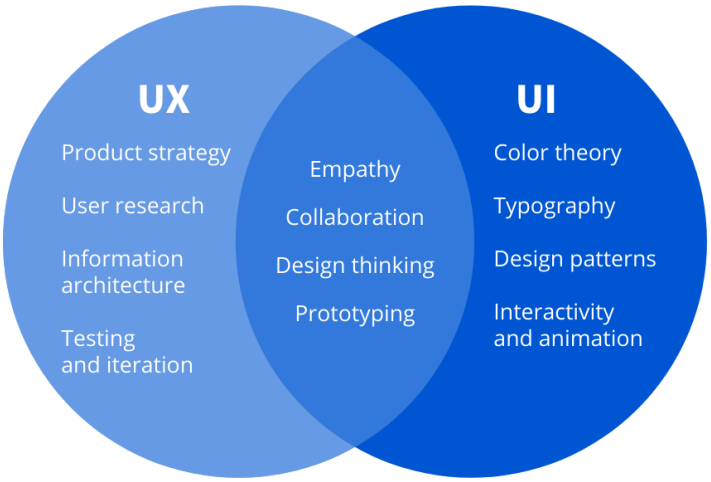
3.9 UI a UX

V procesu vývoje moderních softwarových řešení hrají klíčovou roli dvě disciplíny: User Experience (UX) a User Interface (UI). Ačkoliv jsou tyto termíny laickou veřejností často zaměňovány nebo vnímány jako synonyma, v odborné praxi představují odlišné, avšak vzájemně se doplňující aspekty designového procesu. UX design, neboli návrh uživatelské zkušenosti, je komplexní analytický obor, který se zaměřuje na celkový pocit a efektivitu uživatele při interakci s produktem. Tento termín v 90. letech zpopularizoval kognitivní vědec Don Norman ve společnosti Apple, který zdůrazňoval, že UX nezahrnuje jen samotné rozhraní aplikace, ale veškeré aspekty interakce zákazníka se společností, jejími službami a produkty [43].

Cílem UX designu není pouze vytvořit produkt, který je bezchybný, ale zajistit, aby proces jeho používání byl pro člověka logický a kognitivně nenáročný. Tvorba kvalitního UX je proces založený na datech, který začíná dlouho před napsáním prvního řádku kódu. Zahrnuje uživatelský výzkum, tvorbu person, uživatelských cest (User Journeys) a návrh informační architektury. Výstupem práce UX designéra jsou často tzv. drátěné modely (wireframy), které schematicky znázorňují rozložení prvků na stránce bez grafiky, aby se pozornost soustředila výhradně na funkčnost a logiku ovládání [43].

Na tyto základy navazuje User Interface (UI) design, který se soustředí na vizuální a interaktivní stránku produktu. UI designér transformuje drátěné modely do finální grafické podoby, která musí být nejen estetická, ale především intuitivní. Klíčovou roli zde hraje vizuální hierarchie, která pomocí velikosti, barev a kontrastu navádí oko uživatele k nejdůležitějším prvkům, jako jsou tlačítka pro akci (CTA). UI design rovněž řeší typografii, konzistenci ikon a zpětnou vazbu systému – například animaci, která uživateli potvrdí, že jeho akce byla úspěšná [42]. Nedílnou součástí moderního UI je také přístupnost (Accessibility), která zajišťuje, že aplikace bude čitelná a ovladatelná i pro uživatele se zrakovým postižením [44].

Vztah mezi těmito obory je symbiotický. Rozdíl lze ilustrovat na analogii s lidským tělem: pokud je kód kostrou a svaly, pak UX představuje orgány a nervovou soustavu, které tělu dávají život a funkčnost, zatímco UI je kůže a vnější vzhled, který určuje, jak nás vnímají ostatní. Skvěle vypadající aplikace (vynikající UI) s nefunkčním ovládáním a zmatečnou navigací (špatné UX) bude uživatele frustrovat a rychle ji opustí. Naopak funkční, ale vizuálně odpudivá aplikace může vzbudit nedůvěru v profesionalitu služby. Pro úspěch digitálního produktu je tedy nezbytné, aby obě disciplíny fungovaly v rovnováze. [44]



Zdroj: https://www.coursera.org/articles/ui-vs-ux-design

3.9.1 Uživatelská přívětivost

Uživatelská přívětivost, v odborné literatuře častěji označovaná jako použitelnost (Usability), není pouze subjektivním pocitem, že se aplikace „dobře ovládá“, ale měřitelnou kvalitativní vlastností. Dle mezinárodní definice se jedná o míru, s jakou mohou specifičtí uživatelé dosáhnout specifických cílů s účinností, efektivitou a spokojeností v konkrétním kontextu používání [46]. Použitelnost tedy neexistuje ve vakuu – produkt, který je použitelný pro jednoho uživatele v klidné kanceláři, může být zcela nepoužitelný pro jiného uživatele v terénu [46].

Pro hodnocení uživatelské přívětivosti se často využívá model složený ze tří klíčových faktorů:

1. **Účinnost (Effectiveness)**: Hodnotí přesnost a úplnost, s jakou uživatelé dosahují svých cílů. Jednoduše řečeno, zda systém umožní uživateli úkol vůbec dokončit a zda je výsledek správný [45].
2. **Efektivita (Efficiency)**: Vztahuje se k množství zdrojů vynaložených na dosažení cíle. Zdroje mohou představovat čas, mentální úsilí nebo počet kliknutí myší. Vysoce efektivní systém umožňuje uživateli dosáhnout cíle s minimální námahou [45].
3. **Spokojenost (Satisfaction)**: Subjektivní hodnocení uživatele, které odráží míru pohodlí a absenci frustrace při práci se systémem. Pozitivní postoj k používání produktu je klíčový pro jeho dlouhodobé přijetí [45].

Kromě těchto hlavních pilířů zahrnuje koncept použitelnosti i další aspekty, jako je učitelnost (jak rychle se nový uživatel v systému zorientuje) a zapamatovatelnost (zda je uživatel schopen systém efektivně používat i po delší pauze) [46].

3.9.2 Wireframe a prototyp

V procesu návrhu uživatelského rozhraní se využívají různé metody vizualizace, které se liší mírou detailu a interaktivity. Dvěma nejzásadnějšími fázemi jsou tvorba drátěných modelů (wireframů) a následné prototypování.

Wireframe (Drátěný model) je nízkoúrovňový (low-fidelity) statický nákres, který slouží jako kostra nebo „modrotisk“ budoucí aplikace. Jeho hlavním účelem je definovat strukturu stránky, rozložení prvků a hierarchii obsahu bez rušivých vlivů grafického designu, jako jsou barvy, obrázky nebo typografie [47]. Wireframe se soustředí na otázky „co“ a „kde“ bude umístěno, což umožňuje týmu a stakeholderům diskutovat o funkčnosti a logice ovládání v rané fázi vývoje, kdy jsou změny ještě levné a snadné [47].

Na rozdíl od statického wireframu je prototyp simulací finálního produktu, která zahrnuje interaktivitu. Prototypy mohou mít různou míru věrnosti (od jednoduchých proklikávacích modelů až po plně funkční simulace nerozeznatelné od hotové aplikace) a slouží především k testování a validaci nápadů [48]. Hlavním cílem prototypování je ověřit uživatelské toky (flows), odhalit chyby v použitelnosti a získat zpětnou vazbu od reálných

uživatelů ještě předtím, než se začne psát samotný kód. Tím se výrazně snižuje riziko nákladných úprav v pozdějších fázích vývoje [48].

Zatímco wireframe je tedy o struktuře a informacích, prototyp je o zážitku a chování aplikace. V praxi vývojáři často nejprve vytvoří wireframy pro schválení struktury a následně je převedou do interaktivního prototypu pro uživatelské testování [48].

3.10 Optimalizace a SEO

Optimalizace webových stránek je proces řízených úprav a experimentů, jehož cílem je zlepšit schopnost webu plnit stanovené obchodní cíle, například zvyšováním počtu konverzí či doby strávené na stránce [49]. Nedílnou součástí tohoto procesu je SEO, tedy optimalizace pro vyhledávače. Jedná se o soubor metod a strategií, které mají za cíl zviditelnit webovou stránku v organických (neplacených) výsledcích vyhledávání na klíčová slova relevantní pro daný obor. Rozdíl oproti placené reklamě (PPC) spočívá v dlouhodobé udržitelnosti návštěvnosti, která přetrvává i bez přímých finančních investic za každé kliknutí [51].

Podle oficiální dokumentace společnosti Google není cílem moderního SEO snaha manipulovat s řadícími algoritmy, ale spíše usnadnit vyhledávacím robotům průchod webem (crawling) a pochopení jeho obsahu (indexing) [50]. Kvalitní optimalizace stojí na třech pilířech: technické bezchybnosti kódu, tvorbě hodnotného obsahu, který odpovídá na dotazy uživatelů, a budování autority prostřednictvím zpětných odkazů z jiných důvěryhodných webů [51]. V současnosti se navíc SEO silně prolíná s uživatelskou přívětivostí (UX), neboť vyhledávače upřednostňují weby, které jsou nejen informačně bohaté, ale také rychlé, bezpečné a přehledné [50].

3.10.1 Metadata

Metadata jsou strukturovaná data, která neslouží přímo uživatelům prohlížejícím stránku, ale jsou určena primárně pro vyhledávače a webové prohlížeče. V kódu HTML jsou umístěna v sekci *<head>* a zjednodušeně lze říct, že jde o „data o datech“. Vyhledávačům, jako je Google či Seznam, pomáhají pochopit obsah stránky, správně jej zaindexovat a prezentovat ve výsledcích vyhledávání [53].

Mezi nejdůležitější metadata z pohledu SEO patří:

1. **Titulek (Title)**: Ačkoliv technicky jde o HTML element, je považován za nejdůležitější metadatum pro SEO. Zobrazuje se jako hlavní nadpis ve výsledcích vyhledávání a v záložce prohlížeče. Je klíčovým faktorem pro hodnocení relevance stránky vyhledávačem [53].
2. **Popisek (Meta Description)**: Krátký text, který shrnuje obsah stránky. Google potvrdil, že tento tag nepoužívá jako přímý hodnotící faktor pro řazení výsledků, má však zásadní vliv na míru prokliku (CTR). Kvalitní popisek funguje jako reklamní text, který má uživatele přesvědčit k návštěvě webu [52].
3. **Meta Robots**: Tento tag dává vyhledávačům instrukce, jak mají se stránkou nakládat. Pomocí hodnot jako *noindex* nebo *nofollow* lze zakázat indexaci konkrétní stránky nebo sledování odkazů, což je klíčové například pro skrytí administrativních sekcí webu [52].
4. **Canonical**: Slouží k vyřešení problémů s duplicitním obsahem. Pokud existuje více URL se stejným obsahem, kanonický odkaz řekne vyhledávači, která verze je ta „hlavní“, kterou má zobrazovat ve výsledcích [53].

Kromě standardních tagů podporuje Google i specifické značky, například *googlebot* (instrukce pouze pro robota Google) nebo *nositelinkssearchbox* (zákaz zobrazení vyhledávacího pole ve snippetech) [52].

3.10.2 Core Web Vitals

Core Web Vitals je iniciativa společnosti Google, jejímž cílem je poskytnout jednotné vedení pro signály kvality, které jsou nezbytné pro zajištění skvělé uživatelské zkušenosti na webu. Jedná se o podmnožinu metrik Web Vitals, která se vztahuje na všechny webové stránky a je měřena pomocí nástrojů Google (např. Search Console nebo PageSpeed Insights) na základě reálných dat od uživatelů prohlížeče Chrome (Chrome User Experience Report) [54].

V současné době se Core Web Vitals skládají ze tří specifických metrik, z nichž každá reprezentuje odlišný aspekt uživatelského zážitku:

1. **LCP (Largest Contentful Paint)**: Měří výkon načítání. Konkrétně sleduje čas, za který se vykreslí největší viditelný prvek v okně prohlížeče (obvykle hlavní obrázek nebo blok textu). Pro dobrý uživatelský zážitek by mělo k LCP dojít do 2,5 sekundy od začátku načítání stránky [54].
2. **INP (Interaction to Next Paint)**: Měří interaktivitu a odezvu stránky. Sleduje prodlevu mezi interakcí uživatele (např. kliknutí myši, stisk klávesy) a okamžikem, kdy prohlížeč vykreslí vizuální odezvu na tuto akci. Tato metrika v březnu 2024 nahradila starší metriku FID (First Input Delay). Pro dobrou odezvu by hodnota INP měla být 200 milisekund nebo méně [54].
3. **CLS (Cumulative Layout Shift)**: Měří vizuální stabilitu. Kvantifikuje, jak moc se prvky na stránce neočekávaně posouvají během načítání (např. reklama posune text, který uživatel právě čte). Pro zajištění dobré stability by stránky měly udržovat skóre CLS na hodnotě 0,1 nebo nižší [54].

Splnění limitů těchto metrik je důležité nejen pro komfort uživatelů, ale také přímo ovlivňuje hodnocení stránky ve vyhledávači Google, který Core Web Vitals používá jako jeden z hodnotících faktorů (ranking signals) [54].

3.11 Správa verzí a nasazení

V procesu vývoje softwaru hrají nezastupitelnou roli nástroje pro organizaci kódu a jeho doručení uživatelům. Základním pilířem je správa verzí (version control), což je systém pro řízení změn v dokumentech, počítačových programech a jiných souborech informací. Tyto systémy pomáhají softwarovým týmům spravovat změny ve zdrojovém kódu v průběhu času. Sledují každou individuální úpravu každého souboru, což umožňuje vývojářům v případě chyby vrátit se k jakékoliv předchozí verzi a porovnávat změny. V moderních distribuovaných systémech správy verzí má navíc každý člen týmu na svém počítači kompletní kopii celé historie projektu, nikoliv jen aktuální soubory [55].

Na proces vývoje a verzování navazuje nasazení (deployment). V kontextu softwarového a webového vývoje se jedná o proces přesunu změn nebo aktualizací z jednoho prostředí do druhého – nejčastěji z lokálního vývojového prostředí do tzv. živého (live) nebo produkčního prostředí. Nasazení je klíčovým krokem, který činí aplikaci, webovou stránku nebo softwarový modul dostupným pro koncové uživatele. Bez tohoto procesu by kód zůstal pouze v interním prostředí vývojářů a neplnil by svůj účel pro veřejnost [56].

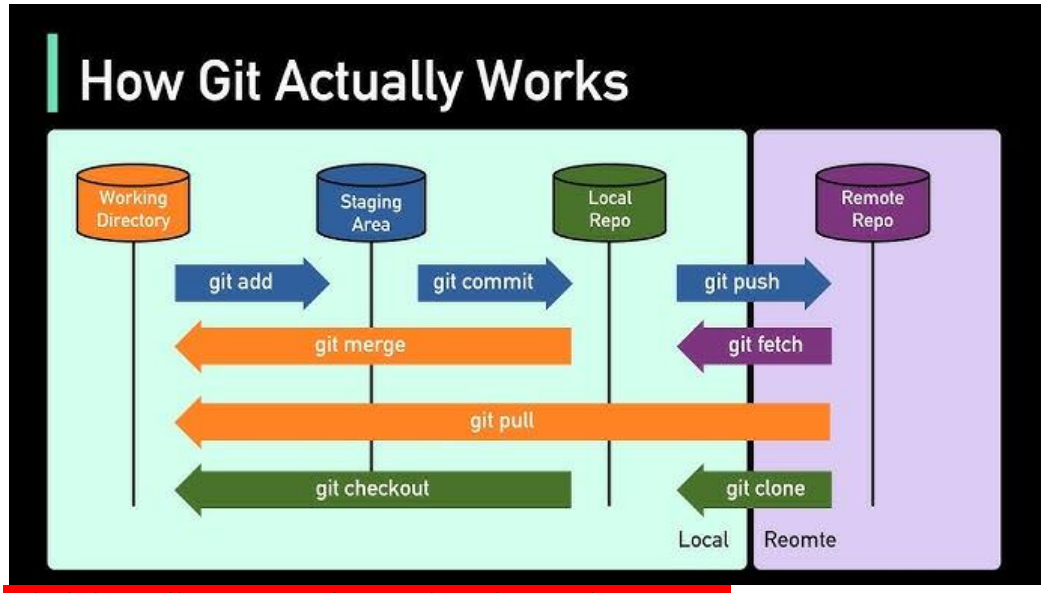
3.11.1 Systém GIT

Git je bezplatný a open source distribuovaný systém správy verzí, který byl navržen tak, aby zvládal vše od malých až po velmi rozsáhlé projekty s důrazem na rychlost a efektivitu. Tento systém vytvořil v roce 2005 Linus Torvalds pro potřeby vývoje jádra operačního systému Linux. Na rozdíl od starších centralizovaných systémů se Git vyznačuje svou schopností podporovat nelineární vývoj prostřednictvím tisíců paralelních větví, které mohou fungovat zcela nezávisle [57].

Hlavní rozdíl mezi Gitem a jinými systémy (jako je Subversion nebo CVS) spočívá v způsobu, jakým nahlíží na data. Zatímco tradiční systémy ukládají informace jako seznam změn v souborech v průběhu času, Git přistupuje k datům jako k sérii snímků (snapshot) malého souborového systému. Pokud se soubor nezměnil, Git jej znovu neukládá, ale pouze odkáže na předchozí identický snímek. Díky tomu, že je systém distribuovaný, má každý vývojář na svém lokálním počítači kompletní kopii celé historie projektu, což umožňuje provádět většinu operací offline a s vysokou rychlostí [58].

Git pracuje se třemi hlavními stavy, ve kterých se mohou soubory nacházet:

1. **Modified (Změněno)**: Soubor byl upraven, ale změna nebyla zatím potvrzena do databáze [57].
2. **Staged (Připraveno)**: Změněný soubor byl označen v aktuální verzi, aby byl zahrnut do příštího snímku (commitu) [57].
3. **Committed (Uloženo)**: Data jsou bezpečně uložena v lokální databázi [57].



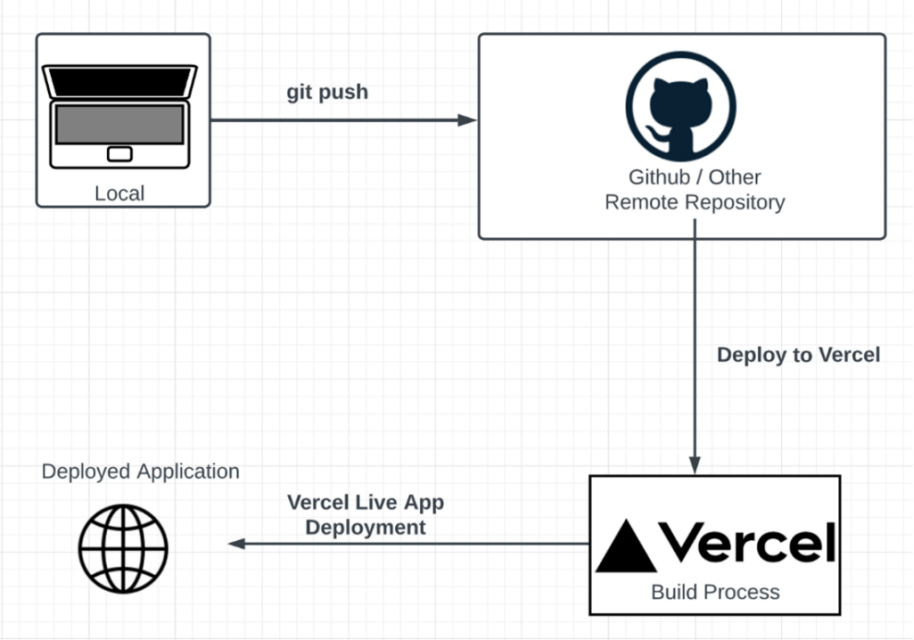
Zdroj: https://www.youtube.com/watch?v=e9lnsKot_SQ

3.11.2 **Vercel**

Vercel je cloudová platforma typu PaaS (Platform as a Service), která se specializuje na hosting frontendových frameworků a statických webových stránek. Společnost je zároveň tvůrcem populárního React frameworku Next.js, což zajišťuje maximální kompatibilitu a optimalizaci pro moderní webové aplikace. Cílem platformy je poskytnout vývojářům ekosystém, který eliminuje nutnost složité konfigurace serverů a infrastruktury, čímž jim umožňuje soustředit se primárně na tvorbu kódu a uživatelského zážitku [59].

Klíčovou vlastností Vercelu je jeho úzká integrace se systémy správy verzí (jako je GitHub, GitLab nebo BitBucket). Tato integrace umožňuje plně automatizovaný proces nasazení (CI/CD). V praxi to funguje tak, že jakmile vývojář odešle změnu (push) do repozitáře, Vercel automaticky spustí proces sestavení aplikace (build) a jejího nasazení. Unikátní funkcí jsou tzv. „Preview Deployments“, kdy Vercel vygeneruje živou URL adresu pro každou jednotlivou změnu v kódu (commit). To umožňuje týmu testovat nové funkce v reálném prostředí ještě předtím, než jsou sloučeny do hlavní produkční verze [60].

Z hlediska infrastruktury využívá Vercel globální síť Edge Network. To znamená, že obsah aplikace je automaticky distribuován na servery po celém světě a uživateli je doručován z geograficky nejbližšího bodu, což radikálně snižuje latenci a zvyšuje rychlost načítání. Platforma také podporuje tzv. serverless funkce, které umožňují spouštět backendovou logiku bez nutnosti spravovat vlastní server [59].



Zdroj: https://www.mitrais.com/news-updates/how-to-deploy-next-js-with-vercel/

3.12 Shrnutí teoretických východisek pro praktickou část

Tato kapitola uzavírá teoretickou část práce a definuje klíčové poznatky, které jsou základem pro následný návrh a implementace aplikace. Z rešerše trhu a technologických standardů vyplývají následující závěry:

1. **Důvěra a lokalita**: Analýza konkurence ukázala, že logistická náročnost byla pro projekty s celorepublikovým dosahem (např. SharyGo) kritická. Praktická část se proto zaměří na lokální komunitní model, kde mapa sousedství a systém hodnocení minimalizují bariéru nedůvěry.
2. **Specializace oproti sociálním sítím**: Rešerše potvrdila, že nejčastější platformou pro sdílení jsou dnes skupiny na Facebooku. Ty však postrádají strukturovanost dat. Praktická část tedy musí nabídnout nástroje, které FB nemá: přehledný katalog kategorií, filtrování podle dostupnosti a jasné parametry výpůjčky.

3. **Absence rezervačního systému**: Jako hlavní technický deficit stávajících komunitních řešení byla identifikována absence interaktivního kalendáře. V praktické části bude proto kladen důraz na automatizaci rezervací, aby uživatelé nemuseli dostupnost věci složitě zjišťovat v komentářích či chatu.
4. **Uživatelská přívětivost (Mobile-First)**: Jelikož ke sdílení dochází často "v terénu", teoretické principy UI/UX budou v praktické části aplikovány na tvorbu rozhraní optimalizovaného pro mobilní zařízení, umožňující bleskové vložení inzerátu (vyfocení a nahrání předmětu přímo na místě).

4 Vlastní práce

Cílem praktické části je návrh a implementace webové aplikace „SousedePůjč“, která slouží jako komunitní platforma pro sdílení předmětů. Řešení je koncipováno jako moderní responzivní webová aplikace využívající framework Next.js a cloudovou infrastrukturu Supabase.

Architektura aplikace je postavena na principu Server-Side Rendering (SSR), což ve spojení s globální distribuční sítí (Vercel Edge Network) zajišťuje vysokou rychlost načítání, optimalizaci pro vyhledávače (SEO) a škálovatelnost.

4.1 Analýza a specifikace požadavků

Analytická fáze projektu přímo vychází ze závěrů teoretické rešerše, kde byly identifikovány klíčové nedostatky současných řešení. Hlavním úkolem analýzy je definovat sadu funkcí, která eliminuje logistické a administrativní bariéry bránící rozvoji lokálního sdílení.

Při návrhu byly zohledněny tři hlavní požadavky vyplývající z teoretické části:

1. **Lokalita**: Aplikace musí prioritizovat zobrazení předmětů v bezprostřední blízkosti uživatele, aby se předešlo logistickým problémům, které byly kritické pro dřívější platformy.
2. **Struktura a pořádek**: Na rozdíl od sociálních sítí (Facebook Marketplace) musí systém nabídnout jasnou kategorizaci a filtrování.
3. **Automatizace procesu**: Nutnost implementace rezervačního systému pro odstranění zdlouhavé komunikace o dostupnosti.

4.1.1 Cílová skupina a persony

Pro definici uživatelských rolí a design uživatelského rozhraní byly stanoveny dvě primární persony. Vzhledem k P2P (Peer-to-Peer) charakteru aplikace se předpokládá, že jeden registrovaný uživatel může dynamicky střídat roli vlastníka i vypůjčitele.

Persona A: „Vlastník“ (Petr, 45 let)

- **Charakteristika**: Petr vlastní rodinný dům se zahradou a dílnou. Disponuje vybavením (např. vertikutátor, motorová pila, přívěsný vozík), které využívá pouze nárazově. Tento majetek představuje tzv. nevyužitou kapacitu.
- **Motivace**: Chce pomoci komunitě a zefektivnit využití svých věcí, ale odmítá trávit čas složitou administrativou nebo dohadováním termínů v chatu.
- **Klíčová potřeba**: Jednoduché rozhraní pro správu inzerátů a kalendář, který automaticky hlídá dostupnost jeho vybavení.

Persona B: „Vypůjčitel“ (Jana, 28 let)

- **Charakteristika**: Bydlí v městském bytě s omezenými skladovacími prostory. Potřebuje věci spíše jednorázově (např. tepovač na koberce, velký cestovní kufr).
- **Motivace**: Ekonomická úspora (nechce kupovat drahou věc na jedno použití) a ekologické smýšlení.
- **Klíčová potřeba**: Potřebuje rychle najít předmět v docházkové vzdálenosti (do 1 km) a ihned provést rezervaci bez nutnosti čekat na schválení v diskuzním vlákně.

4.1.2 Funkční požadavky

Na základě analýzy potřeb cílových skupin a identifikovaných nedostatků konkurenčních řešení byly stanoveny následující klíčové funkční požadavky. Tyto funkce tvoří základní rozsah aplikace (MVP – Minimum Viable Product).

Modul autentizace a správy profilu

- **Popis funkce**: Aplikace musí umožnit registraci a bezpečné přihlášení uživatelů. Každý uživatel bude mít veřejný profil zobrazující jeho jméno, avatar a historii aktivit.
- **Zdůvodnění z rešerše**: Analýza P2P trhu potvrdila, že anonymita je hlavní bariérou sdílení. Ověřené profily jsou nezbytné pro budování důvěry mezi sousedy a prevenci podvodů.

Modul interaktivní mapy a geolokace

- **Popis funkce**: Aplikace bude integrovat mapové rozhraní, které uživatelům vizualizuje dostupné předměty v jejich bezprostředním okolí. Uživatelé budou moci filtrovat inzeráty podle vzdálenosti od své aktuální polohy.
- **Zdůvodnění z rešerše**: Tento požadavek přímo reaguje na selhání platformy SharyGo, kde byla logistická náročnost identifikována jako kritický faktor. Uživatelé preferují předměty v docházkové vzdálenosti.

Strukturovaný katalog a správa inzerce

- **Popis funkce**: Uživatel v roli Vlastníka musí mít možnost vložit předmět s definovanými atributy: název, detailní popis, kategorie (např. Nářadí, Sport, Zahrada), cena za den a fotografie.
- **Zdůvodnění z rešerše**: Řeší problém chaosu a nepřehlednosti inzerce na sociálních sítích (Facebook Marketplace), kde chybí strukturovaná data a možnost efektivní kategorizace.

Rezervační systém (Booking)

- **Popis funkce**: U každého předmětu bude k dispozici kalendář dostupnosti. Vypůjčitel zvolí termín (od-do) a systém automaticky ověří dostupnost v databázi a vytvoří rezervaci. Vlastník má přehled o nadcházejících výpůjčkách.
- **Zdůvodnění z rešerše**: Odstraňuje nutnost zdlouhavé a neefektivní komunikace o volných termínech v chatu, což bylo v teoretické části identifikováno jako hlavní administrativní zátěž.

Systém hodnocení (Reputace)

- **Popis funkce**: Po ukončení výpůjčky umožní systém oběma stranám (Vlastník i Vypůjčitel) ohodnotit průběh transakce.
- **Zdůvodnění z rešerše**: Slouží jako nástroj sociální kontroly a motivace ke slušnému chování, což je základní princip fungování úspěšných platforem sdílené ekonomiky.

4.1.3 Nefunkční požadavky

Nefunkční požadavky definují technická omezení a kvalitativní parametry systému. Pro aplikaci „SousedePůjč“ byly s ohledem na zvolený technologický stack (Next.js + Supabase) a uživatelský komfort stanoveny následující priority.

Výkon a rychlost načítání (Performance)

- **Specifikace**: Aplikace musí splňovat metriky Core Web Vitals definované společností Google. Zejména hodnota LCP (Largest Contentful Paint) musí být na mobilních zařízeních pod 2,5 sekundy.
- **Technické řešení**: K dosažení těchto hodnot bude využito Server-Side Renderingu (SSR) v rámci frameworku Next.js, který generuje HTML na serveru a minimalizuje práci prohlížeče.
- **Zdůvodnění**: V teoretické části byl identifikován přímý vztah mezi rychlostí načítání a mírou opuštění stránky (bounce rate). Pomalá aplikace by odradila uživatele, kteří vyhledávají předměty v terénu.

Responzivita a Mobile-First přístup

- **Specifikace**: Uživatelské rozhraní musí být plně přizpůsobitelné různým úhlopříčkám displejů (od mobilních telefonů po desktopy). Prioritou je vertikální zobrazení na mobilních zařízeních.
- **Technické řešení**: Implementace pomocí CSS frameworku Tailwind CSS, který umožňuje rychlý vývoj responzivních komponent.
- **Zdůvodnění**: Scénáře použití (Use Cases) ukazují, že k samotnému procesu výpůjčky a kontrole rezervace dochází často mimo domov (např. v garáži, na ulici), kde má uživatel k dispozici pouze telefon.

Bezpečnost dat (Security)

- **Specifikace**: Systém musí zajistit, že uživatel může editovat pouze své vlastní inzeráty a osobní údaje. Data ostatních uživatelů musí být chráněna proti neoprávněnému zápisu.
- **Technické řešení**: Využití Row Level Security (RLS) na úrovni databáze PostgreSQL v platformě Supabase. Bezpečnostní pravidla jsou definována přímo v databázové vrstvě, nikoliv jen v aplikaci.

- **Zdůvodnění**: Ochrana osobních údajů a majetkových informací je klíčová pro budování důvěry v P2P systému.

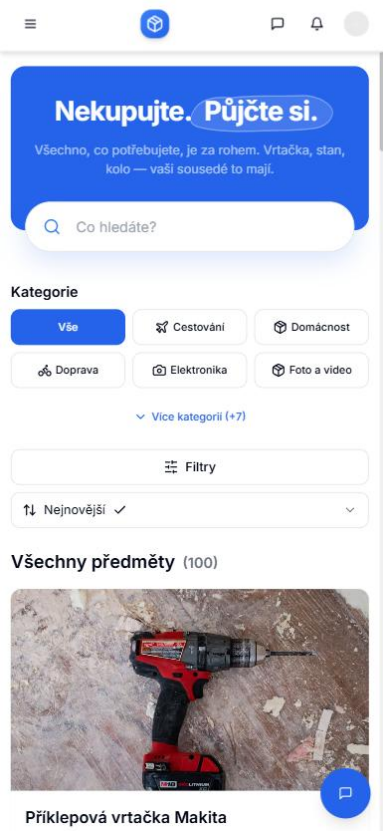
Dostupnost a škálovatelnost

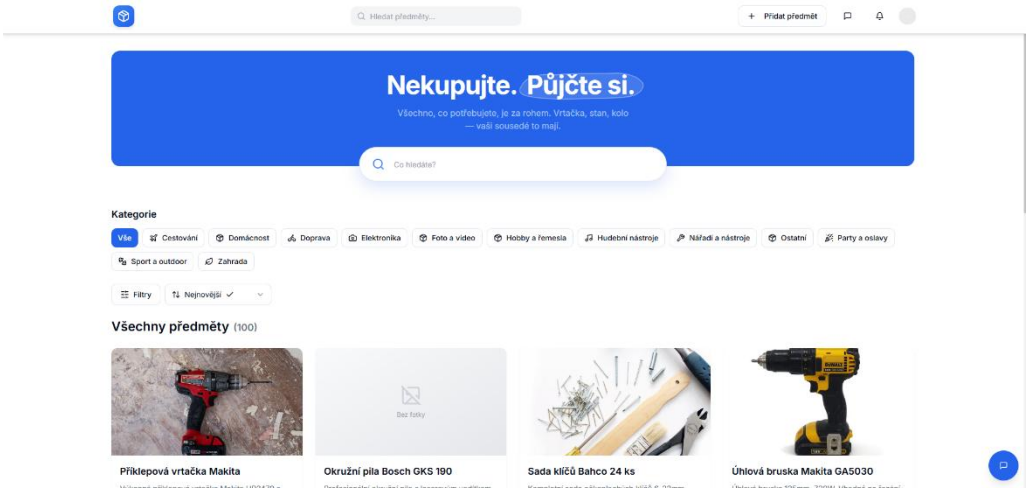
- **Specifikace**: Aplikace musí být dostupná 24/7 s minimálními výpadky. Architektura musí umožňovat nárůst počtu uživatelů bez nutnosti přepisování kódu.
- **Technické řešení**: Nasazení na serverless infrastruktuře Vercel (Frontend) a cloudové databázi Supabase (Backend), které automaticky škálují výkon podle aktuální zátěže.
- **Zdůvodnění**: V teoretické části byla analyzována rizika spojená s provozem vlastního serveru (náročná údržba, výpadky). Cloudové řešení eliminuje nutnost správy hardwaru a zajišťuje, že aplikace bude stabilní i při náhlém zvýšení návštěvnosti (např. při sdílení v komunitě).

4.2 Návrh aplikace

Fáze návrhu uživatelského rozhraní (UI) a uživatelské zkušenosti (UX) přímo vychází z funkčních požadavků definovaných v analytické části. Cílem nebylo pouze vytvořit esteticky příjemné prostředí, ale především navrhnout systém, který je intuitivní, důvěryhodný a použitelný v terénu.

Při návrhu byly aplikovány principy Mobile-First, jelikož scénáře použití (Use Cases) předpokládají, že k interakci s aplikací (hledání věci, potvrzení převzetí) dochází často mimo domov prostřednictvím mobilního telefonu.





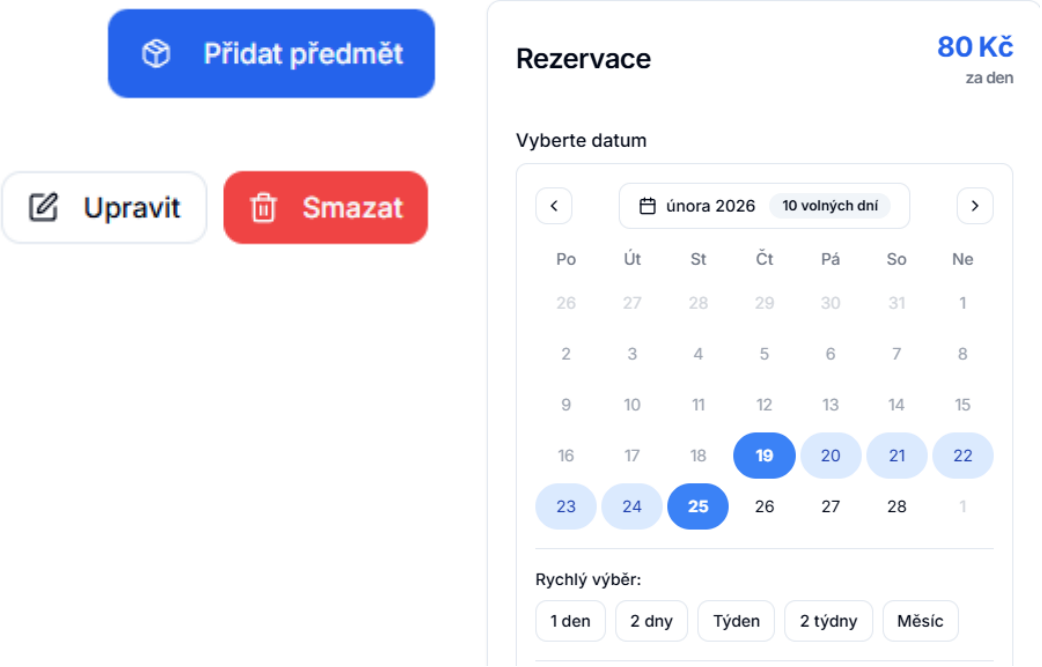
4.2.1 Design systém

Pro zajištění konzistence uživatelského rozhraní (UI) a udržitelnosti kódu byl vytvořen modulární design systém. Technologickým základem je framework Tailwind CSS, který umožňuje aplikovat styly prostřednictvím utility tříd přímo v JSX struktuře, čímž eliminuje nutnost udržovat rozsáhlé externí stylové soubory.

Definice barev a sémantika

Barevná paletika není definována pevnými hexadecimálními hodnotami v komponentách, ale využívá systém CSS proměnných definovaných v souboru *globals.css*. Tento přístup umožňuje centrální správu témat a případnou snadnou implementaci tmavého režimu.

- Primární barva (*--primary #2563eb*): Určena pro hlavní akční prvky (CTA tlačítka, aktivní stavy navigace)
- Sekundární a akcentní barvy (--secondary #f1f5f9, --muted #f1f5f9): Použity pro pozadí karet, neaktivní prvky a oddělovače obsahu, aby byla zachována vizuální hierarchie.
- Destruktivní akce (--destructive #ef4444): Vyhrazena pro kritická operace (např. smazání inzerátu, zamítnutí rezervace), což uživateli vizuálně signalizuje nevratnost kroku



Typografie a ikonografie

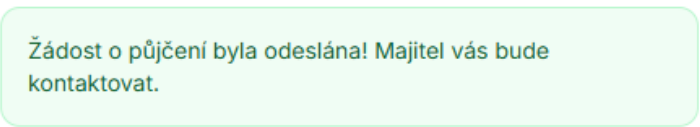
Typografický systém je postaven na variabilním bezpatkovém písmu, které zajišťuje čitelnost jak na mobilních zařízeních, tak na desktopu. Hierarchie nadpisů (*h1* až *h4*) a textu (*body*, *caption*) je řízena pomocí tříd Tailwindu (*text-xl*, *font-bold),* což zaručuje konzistentní řádkování a velikostí napříč celou aplikací.

Pro vizuální komunikaci funkcí (např. filtr, mapa, uživatel) je integrována knihovna Lucide React. Tyto SVG ikony jsou renderovány jako lehké komponenty, což minimalizuje dopad na výkon aplikace oproti klasickým bitmapovým ikonám.

Komponenty zpětné vazby (UI Feedback)

Aplikace implementuje princip okamžité zpětné vazby na akce uživatele. Pro tyto účely byla zvolena knihovna Sonner (Toast notifications).

- Funkce: Po provedeni asynchronní operace (např. odeslání požadavku na rezervaci do databáze Supabase) se uživateli zobrazí neblokující notifikace v rohu obrazovky.
- Výhody: Na rozdíl od modálních oken tento prvek nepřerušuje práci uživatele (Non-blocking UI), ale jasně informuje o výsledku operace (Úspěch/Chyba).

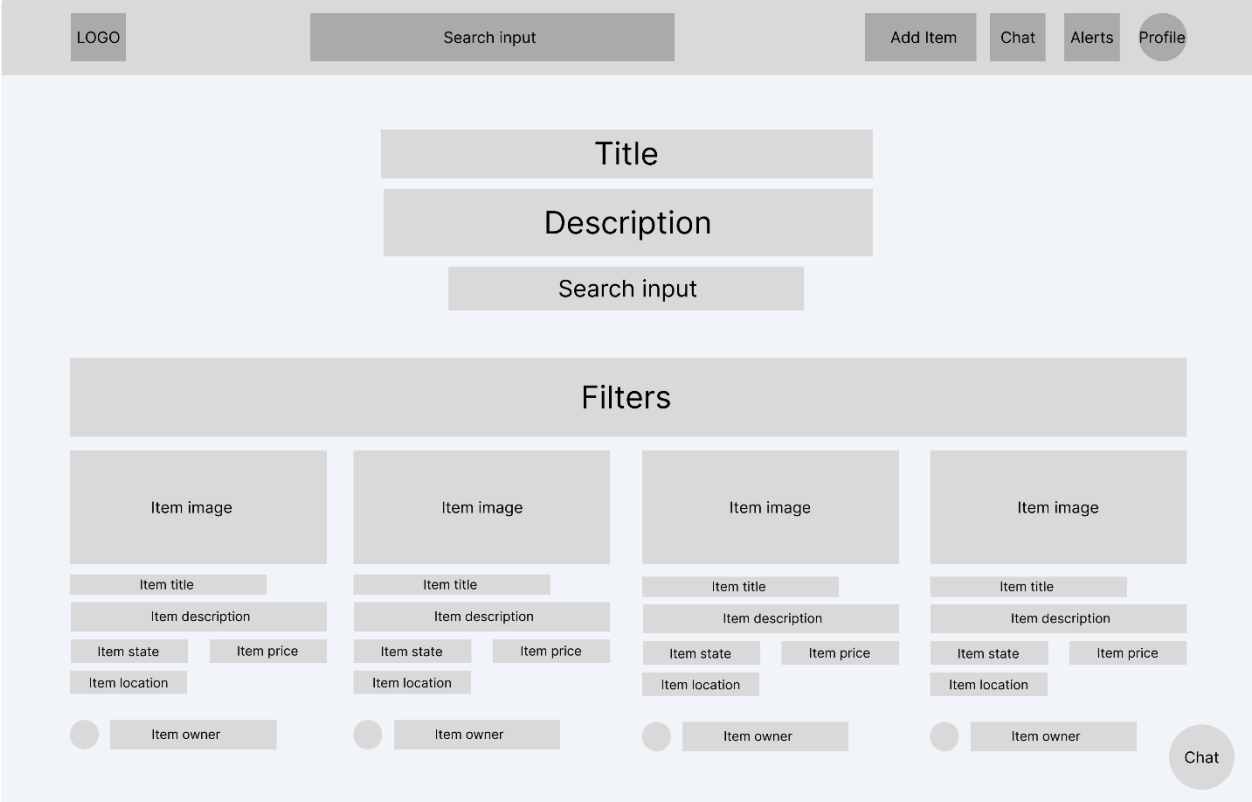


4.2.2 Wireframy a informační architektura

Před samotnou implementací uživatelského rozhraní byly vytvořeny drátěné modely (wireframy) klíčových obrazovek. Tento krok sloužil k přesnému rozvržení hlavních prvků a k definování uživatelské cesty napříč systémem. Návrh probíhal s ohledem na dříve definovaný Mobile-First přístup, a proto byly informační architektura i samotné rozhraní primárně koncipovány pro vertikální zobrazení na mobilních zařízeních. V rámci tohoto návrhu bylo zpracováno pět stěžejních obrazovek, které reprezentují hlavní průchod aplikací.

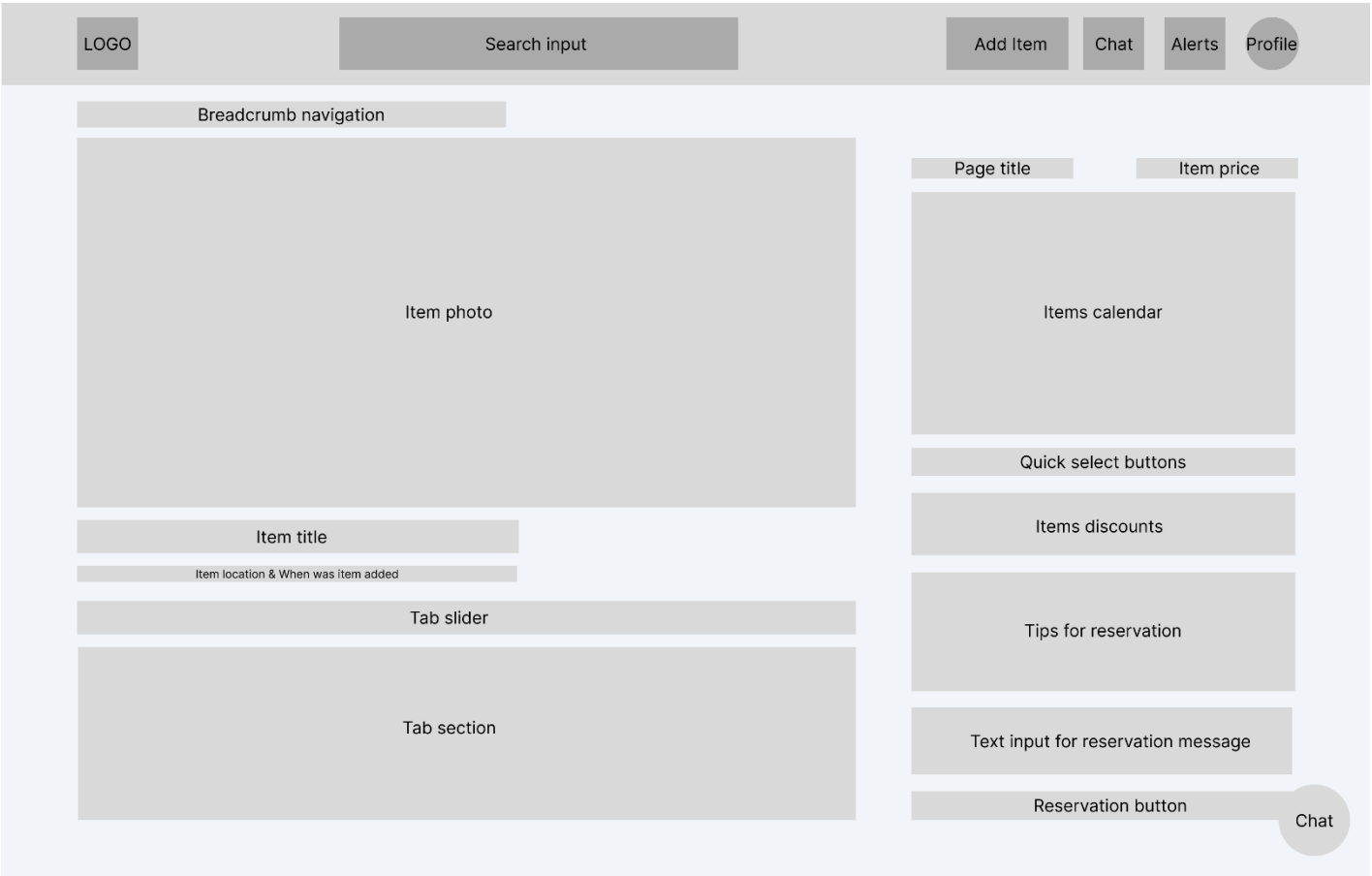
Domovská stránka a vyhledávání

Prvním navrženým modelem je domovská stránka, která funguje jako hlavní rozcestník celé aplikace. V její horní části, označované jako Hero sekce, je umístěn dominantní textový panel určený pro fulltextové vyhledávání v názvech inzerátů. Hned pod tímto prvkem se nachází rychlá navigace rozdělená do základních kategorií, jako je například nářadí nebo sport. Samotný výpis inzerátů je řešen formou responzivní mřížky karet, přičemž každá z nich dynamicky zobrazuje klíčová data z databáze, konkrétně fotografii, název, cenu za den a lokalitu. Celkové rozvržení má za cíl minimalizovat kognitivní zátěž uživatele a umožnit mu rychlý přechod na detail vybraného předmětu.



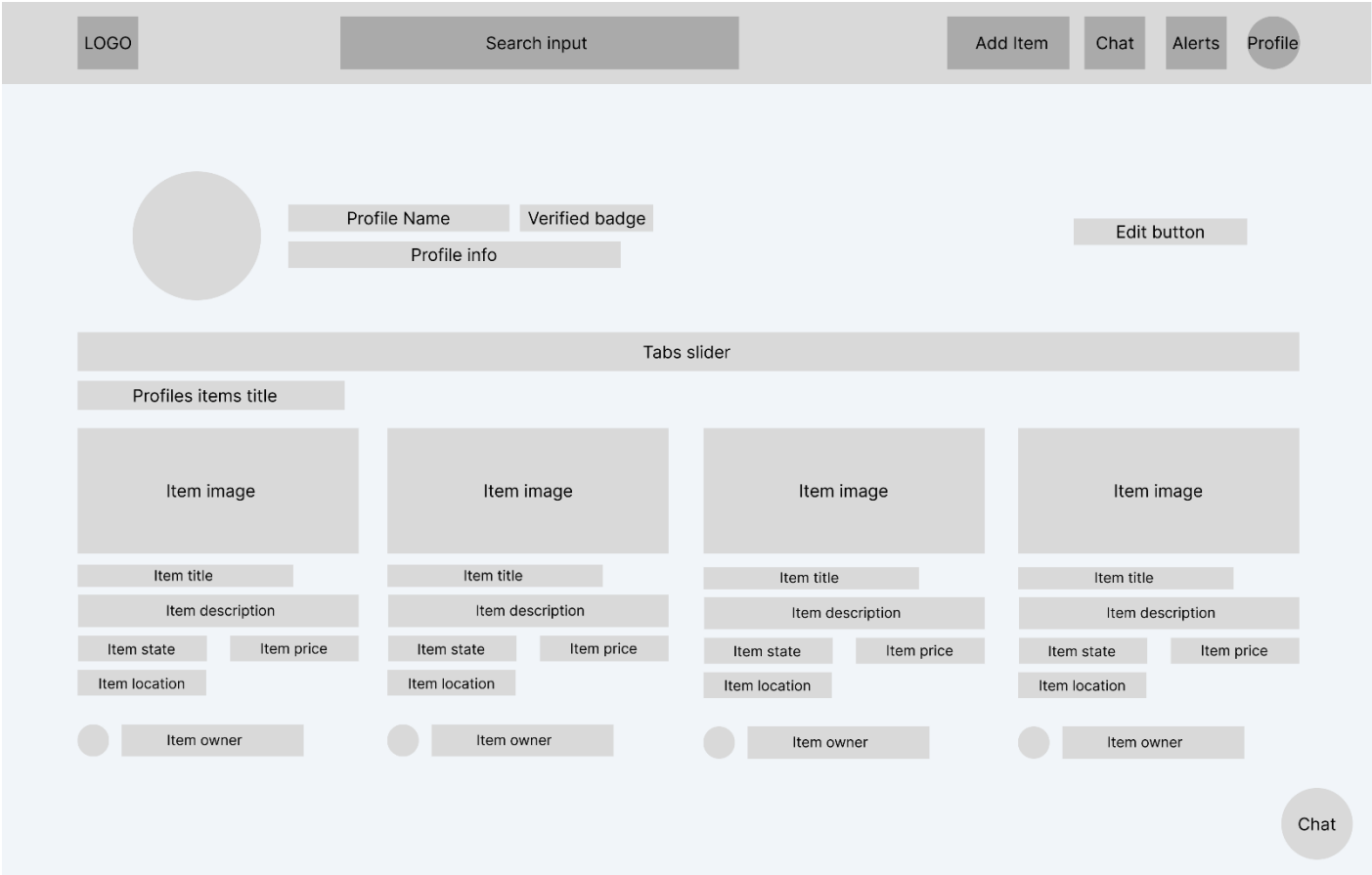
Detail předmětu a rezervace

Na domovskou stránku úzce navazuje detail předmětu, který představuje kritický bod pro konverzní proces, tedy samotné vytvoření žádosti o výpůjčku. Stránka je logicky segmentována tak, aby horní část prezentovala vizuální podobu předmětu, zatímco střední část obsahuje textový popis a modul uživatelského profilu vlastníka. Tento prvek hraje důležitou roli při budování důvěry uvnitř P2P systému. V dolní polovině obrazovky je integrována interaktivní mapa od poskytovatele Mapbox, která vizualizuje přibližnou polohu předmětu, čímž chrání soukromí majitele před potvrzením transakce. Nejdůležitějším interakčním prvkem je zde rezervační kalendář. Ten je algoritmicky omezen tak, aby neumožňoval výběr historických dat nebo termínů překrývajících se se schválenými rezervacemi. Jakmile uživatel zvolí požadovaný termín, systém okamžitě kalkuluje a zobrazuje celkovou cenu výpůjčky.



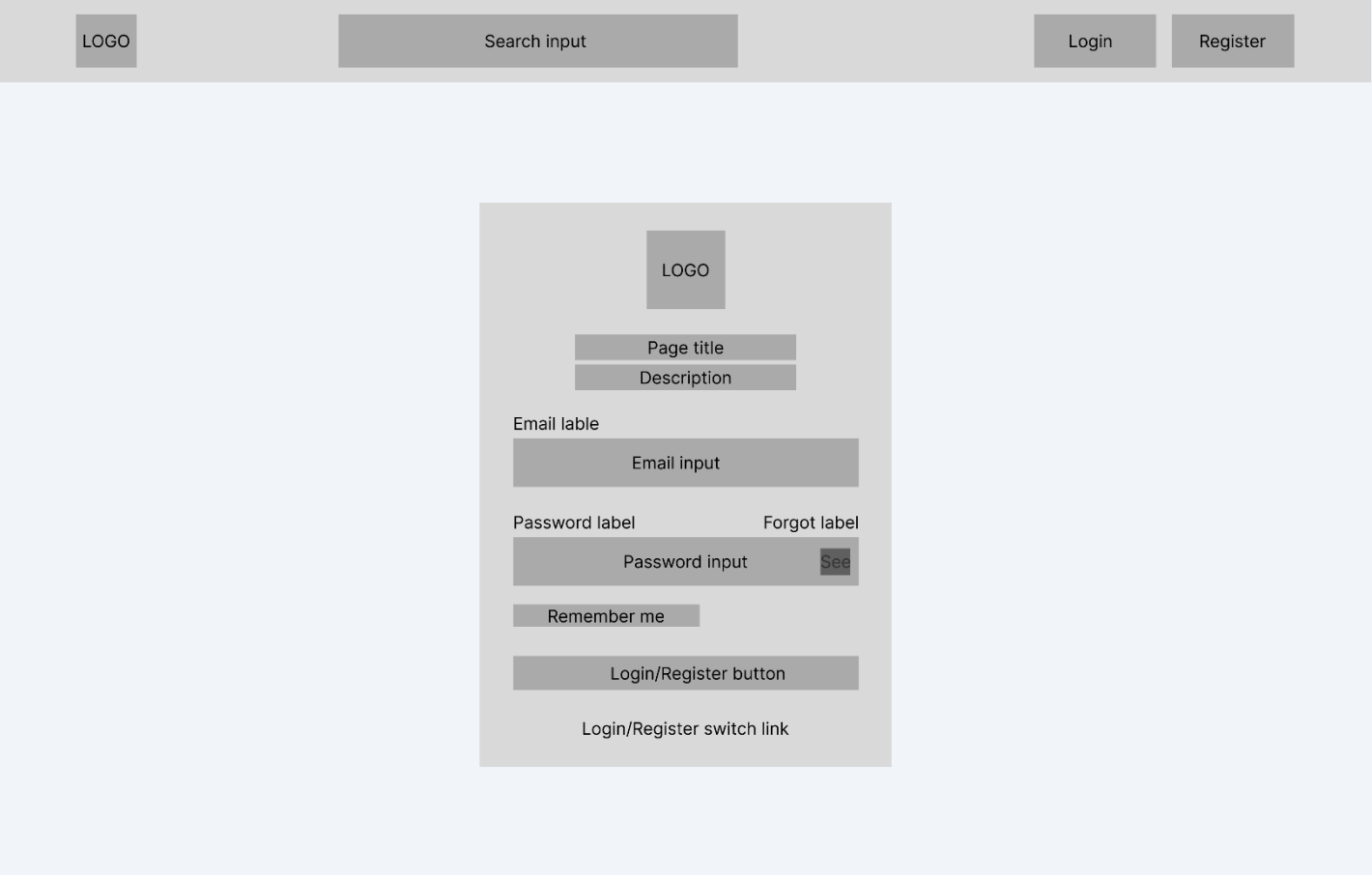
Uživatelský profil

Třetím navrženým modelem je komplexní uživatelský profil, který funguje jako centrální uzel pro správu osobního účtu a veškerých uživatelových aktivit v rámci platformy. Tato obrazovka je logicky segmentována do několika provázaných sekcí. V horní části je kladen důraz na samotnou identifikaci uživatele, což představuje klíčový prvek pro budování a udržení transparentnosti i důvěryhodnosti v komunitním ekosystému. Na tuto identifikační část plynule navazují sekce vyhrazené pro správu vlastních inzerátů a ucelený přehled všech příchozích i odchozích rezervací. Jednotlivé záznamy o výpůjčkách jsou zde prezentovány formou strukturovaného seznamu s barevně odlišenými statusy, které uživateli na první pohled indikují, zda je konkrétní žádost čekající, schválená, nebo naopak zamítnutá. Z prostředí svého profilu může uživatel provádět veškeré základní databázové operace nad nabízenými předměty. V roli vlastníka zde má rovněž k dispozici zřetelná akční tlačítka pro okamžité a bezpečné rozhodování o přijatých poptávkách, na jejichž stisk systém reaguje vizuální zpětnou vazbou pomocí vyskakovacích notifikací.



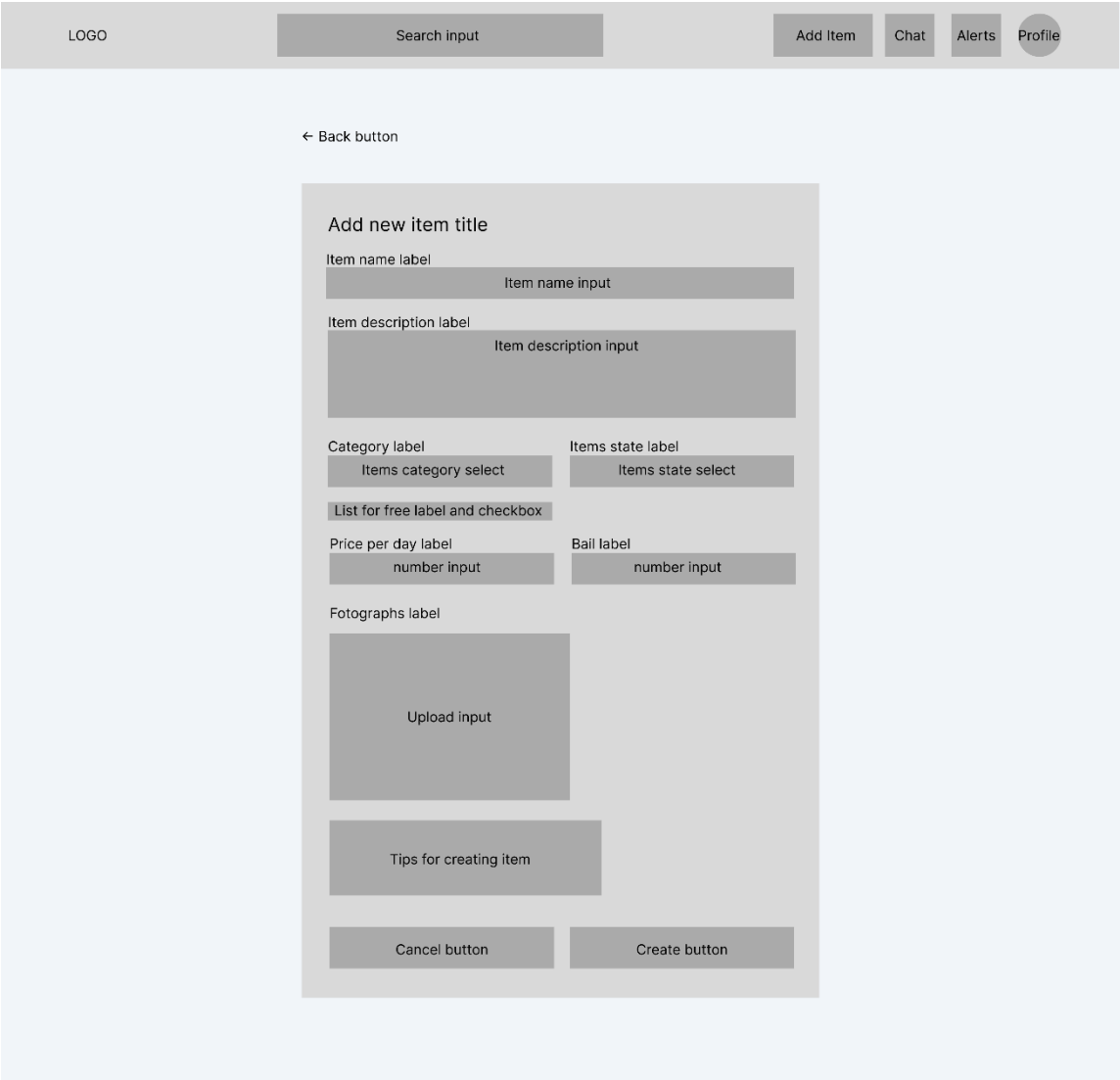
Autentizační rozhraní (Přihlášení/Registrace)

Vstupní bránu do zabezpečené části aplikace představuje autentizační rozhraní pro přihlášení a registraci. Tento drátěný model se opírá o naprostý minimalismus a obsahuje pouze nezbytná vstupní pole pro zadání e-mailové adresy a hesla, jež jsou logicky zarovnána do středového kontejneru. Formulář od počátku počítá s validací uživatelských vstupů přímo na straně klienta. V případě, že uživatel zadá chybné údaje nebo poruší požadovaný formát hesla, systém mu zobrazí srozumitelnou chybovou hlášku. Tento přehledný a bezpečný autentizační proces je nezbytným předpokladem pro hladké zapojení nových uživatelů do platformy.



Formulář pro vložení inzerátu

Posledním navrženým modelem je formulář pro vložení nového inzerátu, jehož primárním úkolem je předcházet vzniku nekompletních nebo datově nekvalitních záznamů. Prvky uživatelského rozhraní jsou zde uspořádány do bloků pokrývajících zásadní informace, cenotvorbu a vizuální dokumentaci. Systém vyžaduje po uživateli vyplnění všech povinných atributů, které se posléze bezpečně propisují do příslušné relační tabulky v databázi. Formulář zároveň obsahuje vestavěné mechanismy pro prevenci chyb, jako je například plošné omezení vkládání nečíselných znaků do pole pro zadání ceny nebo výrazná vizuální indikace chybějících údajů. Celý proces tvorby inzerátu je zakončen hlavním potvrzovacím tlačítkem, které po úspěšném odeslání dat přesměruje uživatele zpět na hlavní stránku.



Vzhledem k využití CSS frameworku Tailwind, který umožňuje rapidní vývoj uživatelského rozhraní, bylo od tvorby klasických interaktivních prototypů v grafických editorech upuštěno. Místo toho byl na základě schválených drátěných modelů vytvořen rovnou takzvaný funkční prototyp (coded prototype) přímo v prostředí frameworku Next.js. Tento moderní přístup umožnil testovat reálné chování aplikace, její responzivitu a logiku navigace přímo v prostředí webového prohlížeče, což výrazně zefektivnilo a urychlilo přechod do fáze implementace backendové logiky a napojení na databázi.

4.3 Technický návrh a databáze

Technický návrh aplikace SousedePůjč vychází z požadavků definovaných v analytické části a opírá se o technologický stack popsaný v teoretických východiscích. Jádrem celé architektury je cloudová platforma Supabase, která zastřešuje relační databázi PostgreSQL, autentizaci uživatelů, real-time komunikaci a souborové úložiště. Následující podkapitoly podrobně popisují návrh databázového modelu, včetně jednotlivých tabulek a jejich vzájemných relací, a celkovou architekturu projektu zahrnující strukturu zdrojového kódu, datový tok, autentizační mechanismy a zabezpečení dat na úrovni jednotlivých řádků.

4.3.1 Tabulky a relace

Databázový model aplikace byl navržen v souladu s principy relačního modelu. Schéma se skládá z jedenácti vzájemně provázaných tabulek, které pokrývají veškeré datové potřeby platformy – od správy uživatelských profilů přes katalog předmětů a rezervační systém až po real-time chat a reputační mechanismus. Jako primární klíč je u všech tabulek využit datový typ *UUID*, generovaný funkcí *gen_random_uuid()*, což zajišťuje globální unikátnost identifikátorů bez nutnosti centrální sekvence. Veškeré cizí klíče jsou definovány s klauzulí *ON DELETE CASCADE*, která automaticky odstraní závislé záznamy při smazání nadřazené entity a zajišťuje tak referenční integritu databáze. Celkový pohled na strukturu databázového schématu včetně všech relací mezi tabulkami zachycuje obrazek na následující stráně.

users
id uuid
email varchar NN
name varchar NN
phone varchar
address text
avatar_url text
is_verified boolean
is_admin boolean
reputation_score numeric
created_at timestamp
updated_at timestamp
show_phone boolean
show_address boolean
show_email boolean
privacy_settings jsonb
bio text
last_active timestamptz
last_seen timestamptz
categories
id uuid
name varchar NN
description text
icon varchar
created_at timestamp
items
id uuid
owner_id uuid
category_id uuid
title varchar NN
description text
condition varchar
daily_rate numeric
deposit_amount numeric
is_available boolean
location text
images text[]
created_at timestamp
updated_at timestamp
bookings
id uuid
item_id uuid
borrower_id uuid
start_date date NN
end_date date NN
status varchar
total_amount numeric
message text
created_at timestamp
updated_at timestamp
chat_rooms
id uuid
booking_id uuid
item_id uuid NN
owner_id uuid NN
borrower_id uuid NN
last_message text
last_message_time timestamptz
created_at timestamptz
updated_at timestamptz
chat_messages
id uuid
room_id uuid NN
sender_id uuid NN
message text NN
is_read boolean
created_at timestamptz
is_edited boolean
updated_at timestamptz
reactions jsonb
message_type varchar
metadata jsonb
attachment_url text
attachment_name text
attachment_type text
reply_to_id uuid
content jsonb
notifications
id uuid
user_id uuid
title varchar NN
message text NN
type varchar NN
is_read boolean
created_at timestamp
reviews
id uuid
booking_id uuid
reviewer_id uuid
reviewed_id uuid
rating integer
comment text
review_type varchar NN
created_at timestamp
message_reactions
id uuid
message_id uuid NN
user_id uuid NN
emoji varchar NN
created_at timestamptz
inappropriate_content_logs
id uuid
user_id uuid NN
original_content text NN
filtered_content text NN
created_at timestamptz
chat_room_id uuid
severity integer
content_moderation_logs
id uuid
user_id uuid NN
original_message text NN
filtered_message text NN
filtered_words text[] NN
context_type varchar NN
context_id uuid
ip_address inet
user_agent text
created_at timestamptz
dbdiagram.io

Tabulka users

Centrální entitou celého systému je tabulka *users*, která uchovává profily registrovaných uživatelů. Klíčovým návrhovým rozhodnutím je přímá vazba na interní autentizační systém platformy Supabase – primární klíč *id* v tabulce *users* musí být identický s identifikátorem uživatele v tabulce *auth.users*, která je spravována modulem Supabase Auth. Tato shoda je nezbytná pro správné fungování bezpečnostních politik Row Level Security, kde funkce *auth.uid()* extrahuje identifikátor přihlášeného uživatele z JWT tokenu a porovnává jej s hodnotami v databázi.

Tabulka obsahuje základní identifikační údaje (e-mail, jméno, telefon, adresa), odkaz na profilový obrázek (avatar_url) a textovou biografii. Pro řízení přístupu slouží booleovské příznaky *is_verified* (ověření e-mailové adresy) a *is_admin* (administrátorská role). Reputační skóre uživatele je uloženo jako numerická hodnota *reputation_score* s výchozí hodnotou 5,00 na škále 1-5. Nastavení soukromí je implementováno prostřednictvím sloupce *privacy_settings* s datovým typem *JSONB*, který umožňuje flexibilní uložení strukturovaného objektu s klíči *show_email, show_phone, show_address* a *show_bio*. Tento přístup byl zvolen namísto vytvoření samostatné relační tabulky, neboť nastavení soukromí je vždy načítáno společně s profilem uživatele a nepodléhá složitým dotazům. Pro sledování aktivity uživatelů slouží časové značky *last_active* a *last_seen*, které jsou automaticky aktualizovány databázovými triggery.

Tabulka categories

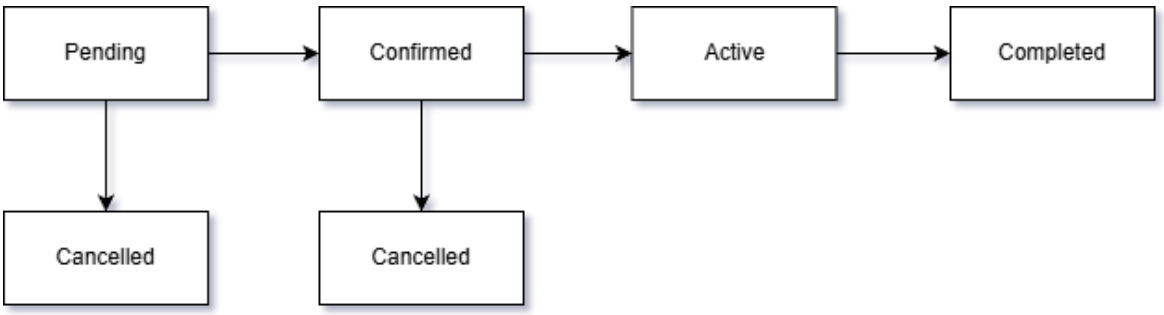
Tabulka *categories* představuje číselníkovou (lookup) tabulku, která definuje hierarchii kategorií předmětů nabízených k zapůjčení. Každý záznam obsahuje název kategorie, volitelný popis a identifikátor ikony pro vizuální reprezentaci v uživatelském rozhraní. Tabulka je naplněna počátečními daty (seed data) při inicializaci databáze a zahrnuje kategorie jako Nářadí, Sport a outdoor, Zahrada, Domácnost, Elektronika, Hudební nástroje a další. Z hlediska přístupových práv je tabulka určena výhradně ke čtení – běžní uživatelé nemohou kategorie vytvářet ani upravovat.

Tabulka items

Tabulka *items* tvoří jádro katalogového systému a uchovává inzeráty předmětů nabízených k zapůjčení. Každý záznam je prostřednictvím cizího klíče *owner_id* svázán s vlastníkem z tabulky *users* a prostřednictvím *category_id* přiřazen ke kategorii. Mezi klíčové atributy patří název předmětu, textový popis, stav předmětu (*condition*) s předdefinovanými hodnotami (excellent, very_good, good, fair, poor), denní sazba za zapůjčení (*daily_rate*) a výše kauce (*deposit_amount*), obě uložené jako numerické hodnoty s přesností na dvě desetinná místa. Příznak *is_available* určuje, zda je předmět aktuálně nabízen k zapůjčení. Fotografie předmětu jsou uloženy jako pole textových odkazů (*text[]*) směřujících na soubory v úložišti Supabase Storage. Volitelný atribut *location* uchovává lokaci předmětu pro účely geografického filtrování.

Tabulka bookings

Rezervační systém je implementován prostřednictvím tabulky *bookings*, která propojuje předmět (*item_id*) s výpůjčitelem (*borrower_id*). Každá rezervace obsahuje časový rozsah definovaný atributy *start_date* a *end_date* datového typu DATE. Celková částka za výpůjčku (*total_amount*) je kalkulována na klientské straně aplikace na základě počtu dnů a denní sazby předmětu. Klíčovým atributem je *status*, který řídí životní cyklus rezervace prostřednictvím stavového automatu s pěti stavy: *pending* (čekající na schválení), *confirmed* (potvrzená vlastníkem), *active* (probíhající výpůjčka), *completed* (úspěšně dokončená) a *cancelled* (zrušená). Volitelný atribut *message* slouží k uchování zprávy od výpůjčitele nebo důvodu zamítnutí.



Tabulka chat_rooms

Pro zajištění komunikace mezi vlastníkem předmětu a vypůjčitelem je navržena tabulka *chat_rooms*, která reprezentuje chatovací místnosti. Každá místnost je automaticky vytvořena při založení nové rezervace a je provázána s konkrétní rezervací (*booking_id),* předmětem (*item_id*) a oběma účastníky konverzace – vlastníkem (*owner_id*) a vypůjčitelem (*borrower_id*). Pro optimalizaci výkonu při zobrazení seznamu konverzací obsahuje tabulka denormalizované atributy last_message a last_message_time, které uchovávají text a čas poslední zprávy. Díky tomu není nutné při načtění přehledu chatů provádět nákladné poddotazy do tabulky zpráv.

Tabulka chat_messages

Jednotlivé zprávy v rámci chatovacích místností jsou uloženy v tabulce *chat_messages*. Každá zpráva je navázána na místnost (*room_id*) a odesílatele (*sender_id*). Tabulka podporuje pokročilé funkce moderního chatu: příznak *is_read* sleduje, zda byla zpráva přečtena příjemcem, příznak *is_edited* indikuje úpravu zprávy po odeslání a atributy *attachment_url*, *attachment_name* a *attachment_type* umožňují přikládání souborů ke zprávám. Funkce odpovídání na konkrétní zprávu je realizována prostřednictvím sebereferencujícího cizího klíče *reply_to_id*, který odkazuje na jinou zprávu ve stejné tabulce s klauzulí *ON DELETE SET NULL*, aby smazání původní zprávy nezpůsobilo kaskádové odstranění všech odpovědí. Pro optimalizaci dotazů jsou na tabulce vytvořeny indexy na sloupcích *room_id*, *sender_id*, *created_at*, *is_read* a *reply_to_id*. Tabulka je rovněž registrována v publikaci *supabase_realtime*, což umožňuje real-time doručování zpráv prostřednictvím WebSocket spojení.

Tabulka message_reactions

Systém emoji reakcí na chatové zprávy je implementován prostřednictvím samostatné tabulky *message_reactions*, která uchovává tři klíčové atributy: cizí klíč zprávy (*message_id*), cizí klíč uživatele (*user_id*) a textový řetězec emoji. Na této trojici je definováno unikátní omezení (*UNIQUE constraint*), které zabraňuje duplicitnímu přidání stejné reakce jedním uživatelem na jednu zprávu.

Tabulka notifications

Notifikační systém je realizován tabulkou *notifications*, kde každý záznam reprezentuje jedno upozornění adresované konkrétnímu uživateli (*user_id*). Notifikace obsahuje titulek, textovou zprávu, typ události (například *booking_request*, *booking_confirmed*, *booking_cancelled*, *new_review*) a příznak přečtení *is_read*. Typové označení umožňuje klientské aplikaci vizuálně odlišit jednotlivé druhy upozornění a přesměrovat uživatele na relevantní sekci systému. Tabulka je stejně jako *chat_messages* registrována v publikaci *supabase_realtime* pro okamžité doručování notifikací.

Tabulka reviews

Reputační mechanismus, identifikovaný v analytické části jako klíčový prvek v P2P systému, je implementován tabulkou *reviews*. Každé hodnocení je svázáno s konkrétní rezervací (*booking_id*) a obsahuje dva uživatelské cizí klíče: *reviewer_id* (autor hodnocení) a *reviewed_id* (hodnocený uživatel). Číselné hodnocení (*rating*) je omezeno databázovým omezením *CHECK* na rozsah 1 až 5. Atribut *review_type* rozlišuje, zda se jedná o hodnocení vypůjčitele (*borrower*) nebo půjčitele (*lender*), čímž je umožněno oboustranné hodnocení po dokončení každé transakce.

Tabulky pro moderaci obsahu

Pro zajištění bezpečnosti komunikace v rámci chatovacího systému jsou v databázi přítomny dvě auditní tabulky. Tabulka *content_moderation_logs* zaznamenává podrobné informace o filtrovaných zprávách včetně původního textu, upraveného textu, seznamu filtrovaných slov a kontextu (typ a identifikátor entity). Tabulka *inappropriate_content_logs* slouží jako zjednodušený záznam s přidanou úrovní závažnosti (severity). Obě tabulky jsou přístupné pouze administrátorům systému.

Přehled relací

Vztahy mezi tabulkami tvoří logický řetězec, který odráží hlavní uživatelský tok aplikace. Centrální entita *users* je propojena s tabulkou *items* vazbou vlastník-předmět (1:N). Na předměty navazují rezervace v tabulce *bookings* (1:N), přičemž každá rezervace odkazuje současně na předmět i na vypůjčitele. Vytvoření rezervace automaticky generuje chatovací místnost v tabulce *chat_rooms*, která propojuje všechny zúčastněné entity – rezervaci, předmět, vlastníka a vypůjčitele. V rámci místnosti vznikají zprávy (*chat_messages*) s možností reakcí (*message_reactions*). Paralelně s tímto hlavním tokem existují nezávislé vazby pro notifikace (*notifications*) a hodnocení (*reviews*), které jsou navázány přímo na uživatele, respektive na rezervace.

Zdrojová tabulka	Cizí klíč	Cílová tabulka	Akce při smazání
items	*owner_id*	*users*	CASCADE
items	*category_id*	*categories*	-
bookings	*item_id*	*items*	CASCADE
bookings	*borrower_id*	*users*	CASCADE
chat_rooms	*booking_id*	*bookings*	CASCADE
chat_rooms	*item_id*	*items*	CASCADE
chat_rooms	*owner_id*	*users*	CASCADE
chat_rooms	*borrower_id*	*users*	CASCADE
chat_messages	*room_id*	*chat_rooms*	CASCADE
chat_messages	*sender_id*	*users*	CASCADE
chat_messages	*reply_to_id*	*chat_messages*	SET NULL
message_reactions	*message_id*	*chat_messages*	CASCADE
message_reactions	*user_id*	*users*	CASCADE
notifications	*user_id*	*users*	CASCADE
reviews	*booking_id*	*bookings*	CASCADE
reviews	*reviewer_id*	*users*	CASCADE
reviews	*reviewed_id*	*users*	CASCADE

Tabulka 1- Přehled cizích klíčů a referenční integrity mezi tabulkami

Databázové triggery a indexy

Pro automatizaci opakujících se operací jsou v databázi definovány tři triggerové funkce, implementované v procedurálním jazyce PL/pgSQL. Funkce *update_updated_at_column()* je spouštěna před každou aktualizací záznamů v tabulkách *chat_messages* a *chat_rooms* a automaticky nastavuje časovou značku *updated_at* na aktuální čas. Funkce *update_user_last_active()* je aktivována po vložení nové chatové zprávy a aktualizuje sloupec *last_active* u odesílatele. Funkce *update_user_last_seen()* je volána při každé aktualizaci záznamu v tabulce *users* a nastavuje aktuální čas do sloupce *last_seen*. Využití triggerů zajišťuje konzistenci těchto metadat bez nutnosti implementace na aplikační vrstvě.

Pro optimalizaci výkonu databázových dotazů je na tabulkách definováno celkem 26 indexů. Indexovány jsou zejména sloupce cizích klíčů využívané při *JOIN* operacích (například *idx_bookings_item*, *idx_bookings_borrower*, *idx_items_owner*) a sloupce používané při řazení a filtrování (například *idx_chat_messages_created_at* se sestupným řazením, *idx_chat_rooms_updated_at*). Tabulka *chat_rooms* disponuje rovněž složenými indexy *idx_chat_room_item_;borrower* a *idx_chat_rooms_item_owner*, které optimalizují časté dotazy na vyhledání chatovací místnosti podle předmětů a účastníka.

NAME	DESCRIPTION	ROWS (ESTIMATED)	SIZE (ESTIMATED)	REALTIME ENABLED	
bookings	No description	2	64 kB	✗	10 columns
categories	No description	21	32 kB	✗	5 columns
content_moderation_logs	No description	1	80 kB	✗	10 columns
chat_messages	No description	9	112 kB	✓	16 columns
chat_rooms	No description	12	176 kB	✓	9 columns
inappropriate_content_logs	No description	0	24 kB	✗	7 columns
items	No description	80	136 kB	✗	13 columns
message_reactions	No description	0	88 kB	✓	5 columns
notifications	No description	40	80 kB	✓	7 columns
reviews	No description	0	24 kB	✗	8 columns
users	No description	8	64 kB	✗	18 columns

4.3.2 Architektura projektu

Architektura aplikace SousedePůjč je postavena na principu oddělení klientské prezentační vrstvy od datové vrstvy spravované cloudovou platformou. Na rozdíl od tradičních webových aplikací, které využívají vlastní backendový server s REST API endpointy, tato aplikace komunikuje s databází přímo prostřednictvím klientské knihovny Supabase. Tento přístup, označovaný jako BaaS, eliminuje nutnost vývoje a údržby vlastního serveru a přenáší zodpovědnost za zabezpečení dat na databázovou vrstvu.

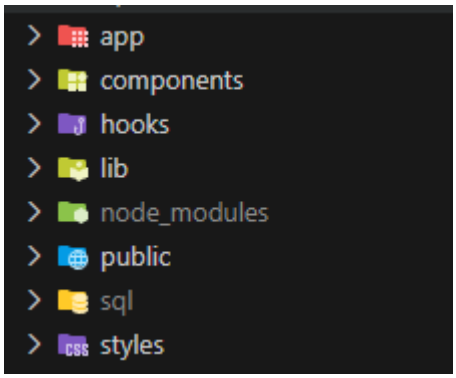
Adresářová struktura

Projekt využívá souborový systém směrování (file-system based routing) frameworku Next.js s architekturou App Router. Zdrojový kód je organizován do čtyř hlavních adresářů:

Adresář *app/* obsahuje stránky aplikace, kde každý podadresář odpovídá jedné URL cestě. Například *app/items/[id]/page.tsx* definuje stránku detailu předmětu s dynamickým parametrem identifikátoru. Soubory *layout.tsx* definují sdílené rozložení stránek a soubory *loading.tsx* zajišťují zobrazení indikátorů načítání prostřednictvím mechanismu React Suspense.

Adresář *lib/* soustřeďuje veškerou sdílenou logiku aplikace. Soubor *supabase.ts* konfiguruje a exportuje instanci klienta Supabase, *database.ts* obsahuje veškeré databázové operace (dotazy, vkládání, aktualizace, mazání), *auth.tsx* implementuje autentizační kontext pomocí React Context API, *types.ts* definuje TypeScript rozhraní pro všechny datové entity a *content-filter.ts* zajišťuje filtrování nevhodného obsahu v chatových zprávách.

Adresář *components/* obsahuje znovupoužitelné React komponenty organizované podle domény: *chat/* pro komponenty chatovacího systému, *items/* pro správu inzerátů, *bookings/* pro rezervační karty a *ui/* pro generické prvky uživatelského rozhraní.



Adresář *sql/* uchovává databázové schéma, migrační skripty a bezpečnostní politiky jako referenční dokumentaci.

Datový tok

Veškeré datové operace v aplikaci procházejí jednotnou cestou. Klientská komponenta volá příslušnou funkci z modulu *database.ts*, která sestaví dotaz pomocí Supabase JavaScript klientské knihovny. Tato knihovna transformuje volání do HTTP požadavku směřujícího na PostgREST – automaticky generované REST API, které platforma Supabase nasazuje nad databází PostgreSQL. PostgREST zpracuje požadavek, aplikuje bezpečnostní politiky Row Level Security a vrátí výsledek ve formátu JSON.

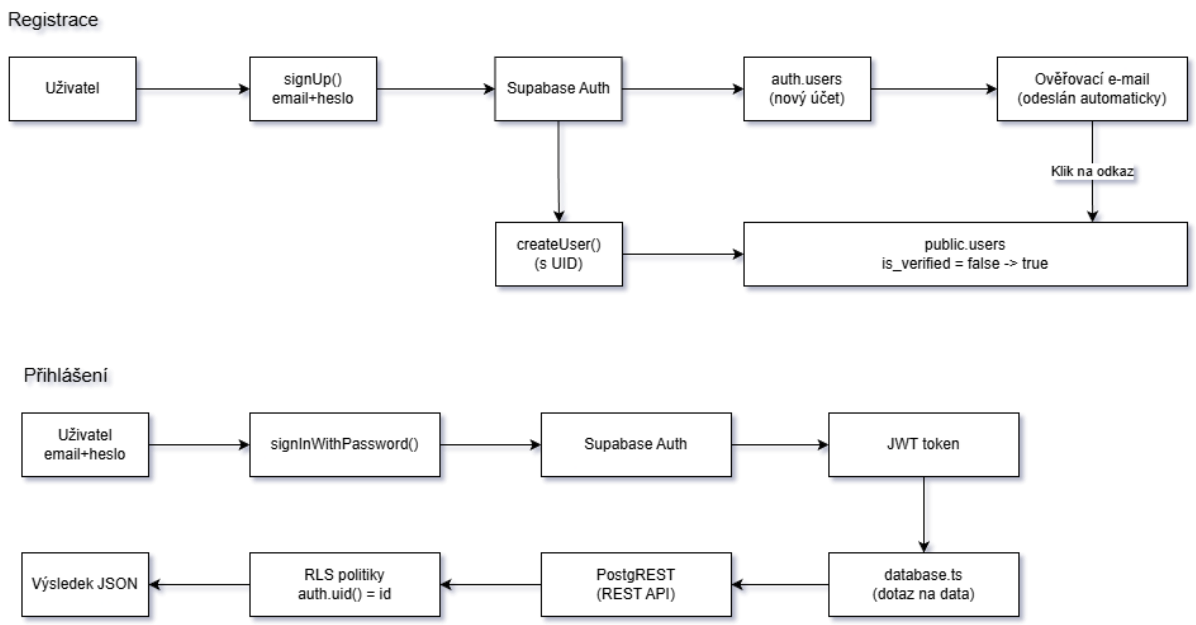
Pro optimalizaci výkonu implementuje modul *database.ts* klientské cachování s konfigurovatelnou dobou platnosti. Data předmětů jsou cachována po dobu 30 sekund, data kategorií po dobu 5 minut, neboť se jedná o statický číselník, který se mění zřídka. Cache je automaticky invalidována při každé mutační operaci (vytvoření, aktualizace nebo smazání předmětu).

Autentizace a správa relací

Autentizační systém je implementován prostřednictvím modulu Supabase Auth. Registrace nového uživatele probíhá ve dvou krocích: nejprve je vytvořen účet v autentizačním systému Supabase pomocí metody *signUp()*, která vrátí unikátní identifikátor uživatele. Tento identifikátor je následně použit jako primární klíč při vytvoření záznamu v tabulce *public.users*. Shoda identifikátorů mezi autentizačním systémem a aplikační databází je nezbytným předpokladem pro korektní vyhodnocování bezpečnostní politik.

Po úspěšném přihlášení metodou *signInWithPassword()* je uživateli vydán přístupový token JWT, který klientská knihovna Supabase automaticky přikládá ke každému následujícímu HTTP požadavku. Stav přihlášení je v aplikaci spravován prostřednictvím React Context API. Komponenta *AuthProvider* obaluje celou aplikaci a poskytuje hook *useAuth()*, který zpřístupňuje aktuální uživatelský objekt, funkce pro přihlášení, odhlášení a registraci všem podřízeným komponentám. Relace je ukládána do lokálního úložiště

prohlížeče (*localStorage*) s unikátním klíčem pro každou instanci prohlížeče, což umožňuje souběžné přihlášení ve více oknech.



Zabezpečení dat pomocí Row Level Security

Klíčovým bezpečnostním mechanismem aplikace je Row Level Security (RLS). Na rozdíl od přístupu, kdy je autorizace implementována v aplikačním kódu na straně serveru, jsou v tomto projektu přístupová pravidla definována přímo v databázi PostgreSQL formou SQL politik. Tento přístup zajišťuje, že bezpečnostní omezení jsou vynucována na úrovni databáze bez ohledu na to, jakým způsobem je požadavek odeslán – ať už z klientské aplikace, nebo prostřednictvím přímého volání API.

Pro každou tabulku jsou definovány samostatné politiky pro operace SELECT, INSERT, UPDATE a DELETE. Například tabulka *items* povoluje čtení všem návštěvníkům (včetně nepřihlášených), zatímco vytváření, úpravu a mazání předmětu povoluje pouze vlastníkovi, u kterého musí platit podmínka *auth.uid() = owner_id*. Tabulka *chat_messages* využívá poddotaz do tabulky *chat_rooms* k ověření, že přihlášený uživatel je skutečně účastníkem dané konverzace. Tabulka *notifications* povoluje čtení a aktualizaci pouze vlastníkovi notifikace, zatímco vytváření je povoleno libovolnému přihlášenému uživateli, neboť notifikace jsou vytvářeny napříč uživateli (například při žádosti o rezervaci).

Real-time komunikace

Pro zajištění okamžitého doručování chatových zpráv a notifikací využívá aplikace modul Supabase Realtime, který poskytuje obousměrné WebSocket spojení mezi klientem a serverem. Tabulky *chat_messages*, *chat_rooms*, *message_reactions* a *notifications* jsou registrovány v publikaci *supabase_realtime*, což umožňuje klientské aplikaci naslouchat změnám v těchto tabulkách v reálném čase. Při přijetí nové zprávy nebo notifikace je klientská komponenta automaticky aktualizována bez nutnosti opakovaného dotazování serveru (polling). Každé real-time připojení je při odpojení komponenty řádně ukončeno voláním *supabase.removeChannel()*, čímž se předchází únikům paměti a zbytečnému zatěžování serveru.

4.4 Implementace projektu

Tato kapitola popisuje konkrétní implementaci jednotlivých funkčních celků aplikace SousedePůjč. Zatímco předchozí kapitola 4.3 se věnovala technickému návrhu databázového modelu a architektuře projektu, následující podkapitoly zachycují postup realizace klíčových funkcionalit – od inicializace projektu a konfigurace cloudové platformy přes implementaci autentizačního systému, katalogu předmětů a rezervačního mechanismu až po chatovací systém s real-time komunikací. U každé funkcionality je popsán způsob implementace a znázorněn výsledný stav v uživatelském rozhraní.

4.4.1 Inicializace projektu a konfigurace Supabase

Projekt byl inicializován pomocí nástroje *create-next-app* s konfigurací pro framework Next.js verze 14 s architekturou App Router. Jako programovací jazyk byl zvolen TypeScript v přísném režimu (strict mode), který poskytuje statickou typovou kontrolu a snižuje riziko běhových chyb. Pro stylování uživatelského rozhraní byl nainstalován framework TailwindCSS spolu s knihovnou komponent Radix UI, která poskytuje přístupné a přizpůsobitelné primitivní komponenty (dialogy, rozbalovací nabídky, přepínače, tooltipy a další). Validace formulářových dat je zajištěna kombinací knihoven React Hook Form pro správu stavu formulářů a Zod pro definici validačních schémat.

Mezi další klíčové závislosti projektu patří knihovna *date-fns* pro práci s datumy, *lucide-react* pro ikony, *react-day-picker* pro výběr dat v kalendáři, *fuse.js* pro fuzzy

vyhledávání v katalogu předmětů, *leaflet* a *react-leaflet* pro zobrazení mapových podkladů, *recharts* pro vizualizaci statistik a *resend* pro odesílání e-mailových notifikací. Kompletní seznam závislostí je definován v souboru *package.json*.

Konfigurace platformy Supabase zahrnovala vytvoření nového projektu v prostředí Supabase Dashboard, kde byl automaticky přidělen unikátní URL endpoint a anonymní API klíč (anon key). Tyto přístupové údaje jsou v aplikaci uloženy jako proměnné prostředí *NEXT_PUBLIC_SUPABASE_URL* a *NEXT_PUBLIC_SUPABASE_ANON_KEY* v souboru *.env*. Klientská instance Supabase je inicializována v modulu *lib/supabase.ts* voláním funkce *createClient()* s vlastní konfigurací autentizačního úložiště – každá instance prohlížeče generuje unikátní identifikátor, který je použit jako klíč pro ukládání relace do *localStorage*. Toto řešení umožňuje souběžné přihlášení stejného uživatele ve více oknech prohlížeče bez vzájemné interference relací.

4.4.2 Autentizace a správa uživatelů

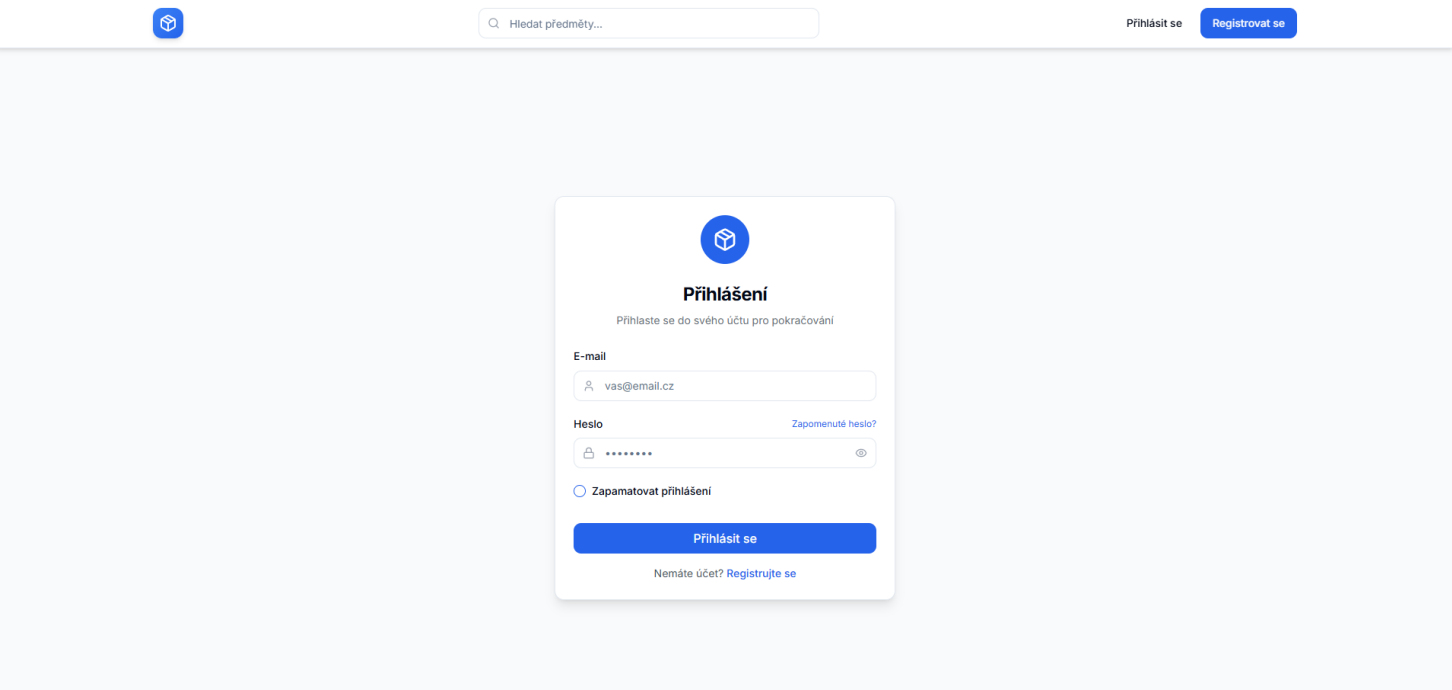
Autentizační modul aplikace je implementován jako klientská komponenta v souboru *lib/auth.tsx* s využitím React Context API. Komponenta *AuthProvider* obaluje celou aplikaci a spravuje stav přihlášeného uživatele prostřednictvím hooku *useState*. Při inicializaci aplikace je asynchronně ověřena existence platné relace voláním metody *supabase.auth.getSession()*. Pokud relace existuje, jsou načtena uživatelská data z tabulky *public.users* a uložena do kontextu. Pro sledování změn autentizačního stavu (přihlášení, odhlášení, obnovení tokenu) je registrován listener *supabase.auth.onAuthStateChange(),* který automaticky aktualizuje stav aplikace při každé změně. Veškeré autentizační funkce jsou zpřístupněny podřízeným komponentám prostřednictvím *useAuth()*.

Registrace nového uživatele probíhá ve dvou krocích popsaných v kapitole 4.3.2. Po odeslání registračního formuláře je nejprve vytvořen účet v autentizačním systému Supabase voláním metody *supabase.auth.signUp()*, která vrátí unikátní identifikátor uživatele. Tento identifikátor je následně použit jako primární klíč při vytvoření záznamu v tabulce *public.users* voláním funkce *db.createUser()*. Shoda identifikátorů je nezbytná pro korektní fungování bezpečnostních politik Row Level Security.

```
const register = async (userData: { email: string; name: string; password: string }): Promise<boolean> => {
  try {
    // Zkontrolujeme, zda uživatel již existuje v naší databázi
    const existingUser = await db.getUserByEmail(userData.email)
    if (existingUser) {
      return false
    }

    // Registrace pomocí Supabase Auth s povoleným e-mailovým ověřením
    const { data, error } = await supabase.auth.signUp({
      email: userData.email,
      password: userData.password,
      options: {
        data: {
          name: userData.name,
        },
        emailRedirectTo: `${window.location.origin}/auth/callback`,
      },
    })

    if (error) {
      console.error("Registration error:", error.message)
      return false
    }

    // Vytvoříme uživatele v naší databázi (neověřený)
    // Pass the Supabase Auth UID so public.users.id matches auth.uid()
    await db.createUser({
      id: data.user!.id,
      email: userData.email,
      name: userData.name,
      is_verified: false,
      is_admin: false,
      reputation_score: 5.0,
    })

    // Nebudeme uživatele automaticky přihlašovat
    return true
  } catch (error) {
    console.error("Registration error:", error)
    return false
  }
}
```

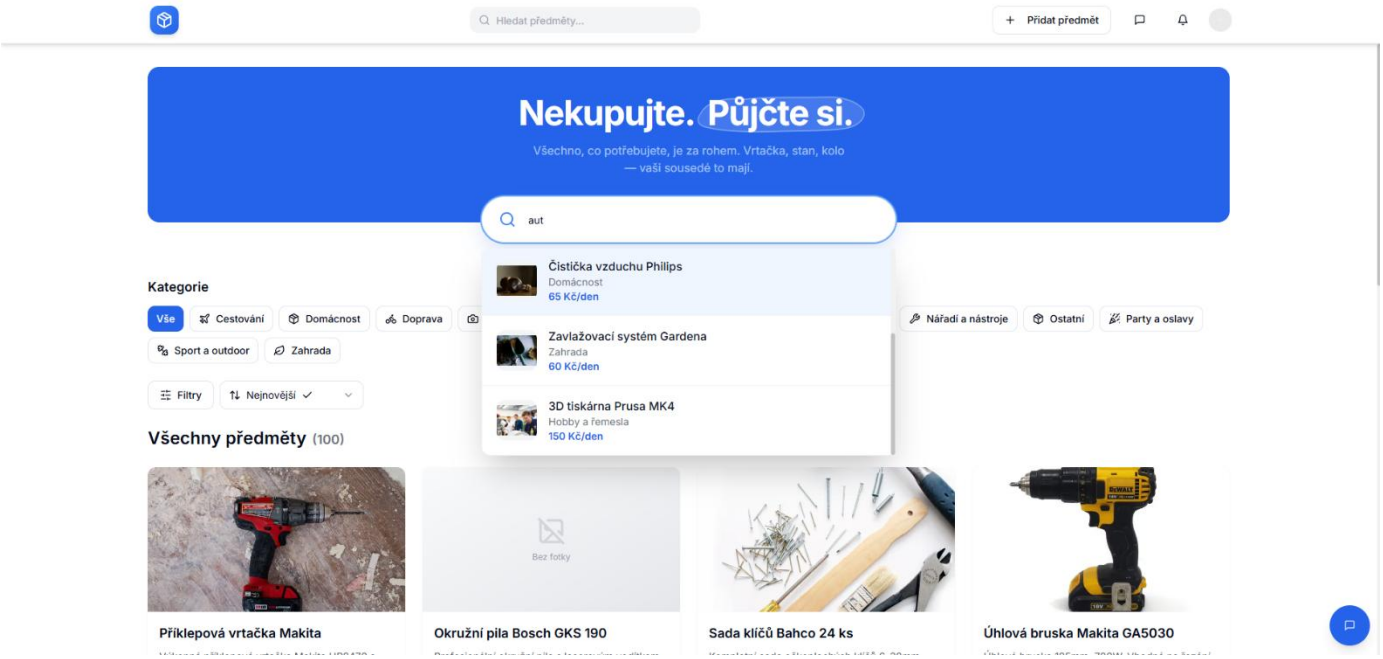
Registrační formulář, implementovaný v souboru *app/register/page.tsx*, vyžaduje od uživatele zadání jména, e-mailové adresy a hesla s minimální délkou šesti znaků. Součástí formuláře je vizuální indikátor síly hesla, potvrzení hesla a souhlas s podmínkami použití. Po úspěšné registraci je uživatel informován o nutnosti ověření e-mailové adresy a po pěti sekundách automaticky přesměrován na přihlašovací stránku.

Přihlašovací stránka (*app/login/page.tsx*) obsahuje formulář s poli pro e-mail a heslo, možností zobrazení hesla, zapamatování přihlášení a odkazem na obnovu zapomenutého hesla. Po úspěšném přihlášení metodou *signInWithPassword()* je uživateli vydán JWT token, který klientská knihovna Supabase automaticky přikládá ke všem následujícím požadavkům. Při pokusu o přihlášení s neověřeným e-mailem je uživateli zobrazena informační zpráva s možností opětovného odeslání ověřovacího e-mailu.



4.4.3 Katalog předmětů a filtrování

Hlavní stránka aplikace (*app/page.tsx*) zobrazuje katalog všech předmětů nabízených k zapůjčení. Data jsou načtena z databáze voláním funkce *db.getItems()* při prvním vykreslení komponenty a následně cachována po dobu 30 sekund pro optimalizaci výkonu. Každý předmět je zobrazen prostřednictvím komponenty *ItemCard*, která vykresluje náhledový obrázek, název, popis, denní sazbu, stav předmětu a hodnocení vlastníka. Jednotlivé karty jsou uspořádány do responzivní mřížky pomocí komponenty *ItemGrid*, která využívá memoizaci (*React.memo*) pro prevenci zbytečných překreslení při změně filtrů.



Pro vyhledávání v katalogu je implementováno Fuzzy vyhledávání pomocí knihovny *fuse.js*, které umožňuje nalezení předmětů i při nepřesném zadání dotazu (například překlepy nebo částečné shody). Vyhledávání je aktivováno od dvou znaků a prohledává název i popis předmětu. Pro dotazy kratší než dva znaky je použito základní filtrování pomocí metody *includes()*. Komponenta *SearchAutocomplete* poskytuje uživateli našeptávač s náhledem výsledků během psaní.

Filtrování předmětů je realizováno prostřednictvím komponenty *ItemFilters*, která nabízí filtrování podle cenového rozsahu pomocí posuvníku (*Slider*), dostupnosti předmětů a řazení podle šesti kritérií (nejnovější, nejstarší, cena vzestupně/sestupně, název A-Z/Z-A). Dále je implementován filtr podle kategorií prostřednictvím komponenty *CategoryFilter*, která zobrazuje dostupné kategorie s ikonami. Veškeré aktivní filtry jsou synchronizovány s URL parametry stránky, což umožňuje sdílení filtrovaného pohledu prostřednictvím odkazu a zachování stavu filtrů při navigaci zpět.

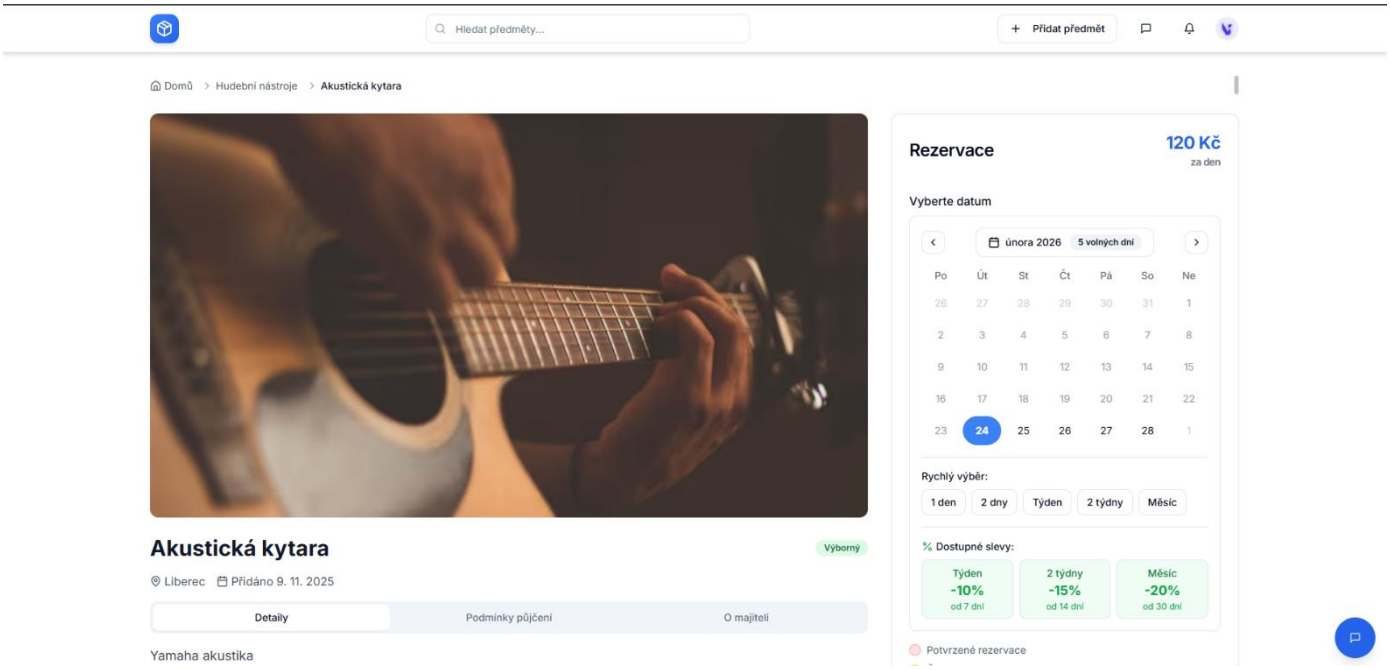
```
// Funkce pro filtrování a řazení předmětů - optimalizováno
const filteredAndSortedItems = useMemo(() => {
  let result = [...items]

  // Filtrování podle kategorie
  if (selectedCategory) {
    result = result.filter((item) => item.category_id === selectedCategory)
  }

  // Filtrování podle vyhledávání (fuzzy search)
  if (searchQuery && searchQuery.length >= 2) {
    // Use professional fuzzy search
    const fuzzyResults = fuzzyMatchItems(result, searchQuery)
    result = fuzzyResults
  } else if (searchQuery) {
    // Basic search for 1 char queries
    const query = searchQuery.toLowerCase()
    result = result.filter(
      (item) =>
        item.title.toLowerCase().includes(query) ||
        item.description?.toLowerCase().includes(query) ||
        item.category?.name.toLowerCase().includes(query),
    )
  }
```

4.4.4 Detail předmětu a správa inzerátu

Stránka detailu předmětu (*app/items/[id]/page.tsx*) využívá dynamické směrování frameworku Next.js, kdy segment *[id]* v cestě odpovídá identifikátoru předmětu v databázi. Při načtění stránky je z URL extrahován parametr *id* pomocí hooku *useParams()* a data předmětu jsou načtena voláním funkce *db.getItemById()*. Stránka je rozložena do dvou sloupců: levý sloupec zobrazuje galerii obrázků prostřednictvím komponenty *ImageGallery* a tabulkový systém se třemi záložkami (Detaily, Podmínky půjčení, O majiteli), pravý sloupec obsahuje rezervační panel s kalendářem, cenou a formulářem pro odeslání žádosti.

Záložka Detaily zobrazuje popis předmětu, specifikace (kategorie, stav, značka) a přehled slev při delším pronájmu. Záložka Podmínky půjčení obsahuje rozbalovací sekce (implementované komponentou *Accordion*) s informacemi o kauci, vyzvednutí a vrácení, platebních podmínkách a postupu při poškození. Záložka O majiteli zobrazuje profil vlastníka s hodnocením, počtem nabízených předmětů a nejnovějšími recenzemi. Pokud je přihlášený uživatel vlastníkem zobrazovaného předmětu, jsou v záhlaví stránky zobrazena

tlačítka pro úpravu a smazání inzerátu. Smazání vyžaduje potvrzení prostřednictvím dialogového okna (*Dialog*), které upozorňuje na nevratnost akce.

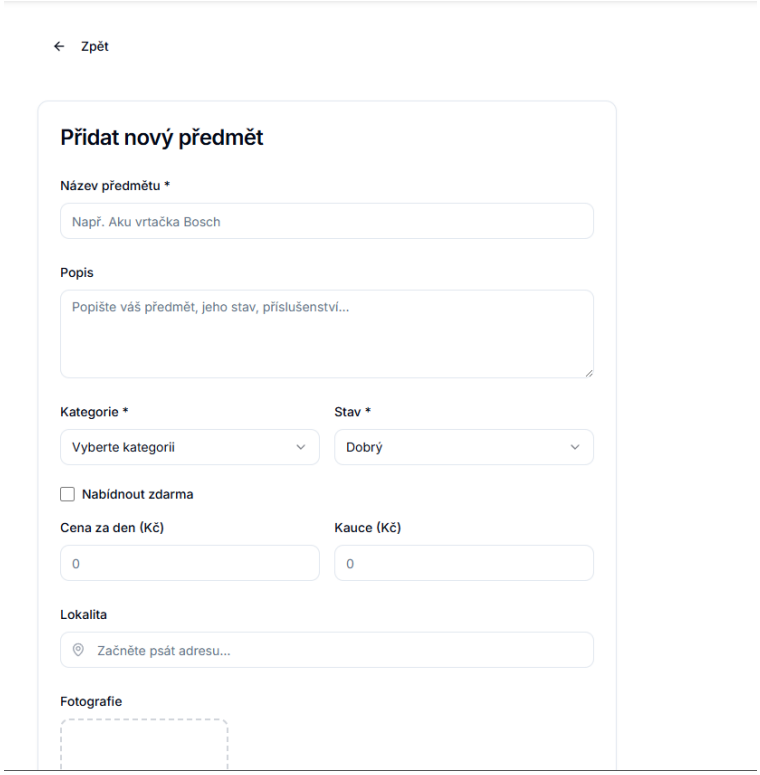


Vytvoření nového inzerátu je realizováno na stránce *app/items/new/page.tsx,* která je přístupná pouze přihlášeným uživatelům – nepřihlášení uživatelé jsou automaticky přesměrováni na přihlašovací stránku. Formulář obsahuje pole pro název, popis, výběr kategorie z číselníku, stav předmětu, denní sazbu, výši kauce a lokaci s našeptávačem adres (komponenta *AddressAutocomplete*). Nahrávání fotografií zajišťuje komponenta *ImageUpload*, která ukládá soubory do úložiště Supabase Storage a vrací veřejné URL adresy uložené následně v poli *images* tabulky *items*. Po úspěšném vytvoření je uživatel přesměrován na stránku nově vytvořeného předmětu. Editace existujícího inzerátu (*app/items/[id]/edit/page.tsx*) využívá stejný formulář s předvyplněnými hodnotami.

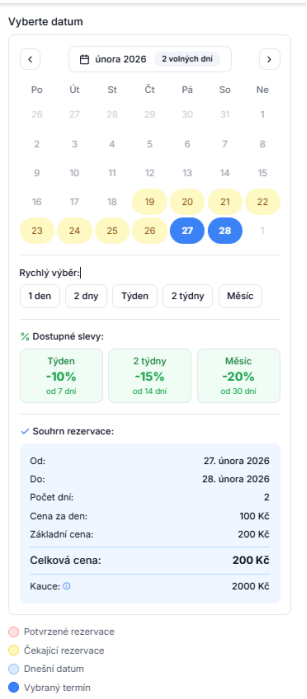
4.4.5 **Rezervační systém**

Rezervační proces je zahájen na stránce detailu předmětu, kde přihlášený uživatel vybírá požadovaný termín výpůjčky prostřednictvím komponenty *BookingCalendar*. Kalendář je implementován jako vlastní komponenta v souboru *components/calendar/booking-calendar.tsx* s využitím knihovny *date-fns* pro práci s datumy a české lokalizace. Při načtení komponenty jsou z databáze načteny existující

rezervace daného předmětu, na jejichž základě jsou obsazené dny vizuálně odlišeny barevným kódem – čekající rezervace žlutě, potvrzené červeně. Uživatel nemůže vybrat dny, které se překrývají s existující rezervací nebo které leží v minulosti.

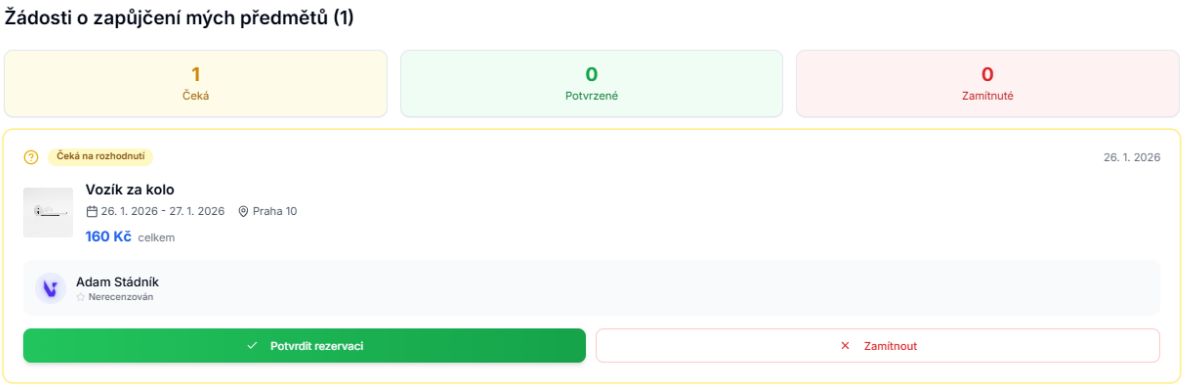


Po výběru termínu je automaticky kalkulována celková cena na základě počtu dnů a denní sazby předmětu. Systém rovněž implementuje množstevní slevy při delším pronájmu: 10 % při výpůjčce na 7 a více dnů, 15 % při 14 a více dnech a 20 % při měsíční výpůjčce. Uživatel může ke své žádosti připojit volitelnou zprávu pro vlastníka. Po odeslání žádosti je vytvořen záznam v tabulce *bookings* se stavem *pending*, vlastníkovi je odeslána notifikace a e-mailové upozornění a automaticky je založena chatovací místnost pro vzájemnou komunikaci obou stran.



Správa rezervací je realizována prostřednictvím komponenty *BookingCard*, která zobrazuje detaily rezervace včetně předmětu, termínu, celkové ceny a kontaktních údajů protistrany. Vlastník předmětu může přijatou žádost potvrdit nebo zamítnout, přičemž obě akce generují notifikaci a e-mailové upozornění pro vypůjčitele. Po potvrzení rezervace se stav změní na *confirmed*. Obě strany mají možnost rezervaci zrušit, což vyžaduje potvrzení prostřednictvím dialogového okna. Po dokončení výpůjčky (stav *completed*) je oběma stranám zpřístupněn formulář pro vzájemné hodnocení, čímž je propojen rezervační systém s reputačním mechanismem popsaným v kapitole 4.3.1.

```
109 const handleBooking = async () => {
126     // Format dates in local timezone to avoid UTC shift issues
127     const formatDateLocal = (date: Date) => {
128       const year = date.getFullYear()
129       const month = String(date.getMonth() + 1).padStart(2, '0')
130       const day = String(date.getDate()).padStart(2, '0')
131       return `${year}-${month}-${day}`
132     }
133
134     const bookingData: Omit<Booking, "id" | "created_at" | "updated_at"> = {      v0, 8 months ago • Initialized repository for project Webová api…
135       item_id: item.id,
136       borrower_id: user.id,
137       start_date: formatDateLocal(selectedDates.from),
138       end_date: formatDateLocal(selectedDates.to),
139       status: "pending",
140       total_amount: calculateTotalAmount(),
141       message: message.trim() || undefined,
142     }
143
144     const newBooking = await db.createBooking(bookingData)
145
146     // Vytvoření notifikace pro majitele
147     await db.createNotification({
148       user_id: item.owner_id,
149       title: "Nová žádost o půjčení",
150       message: `${user.name} má zájem o půjčení předmětu "${item.title}"`,
151       type: "booking_request",
152       is_read: false,
153     })
154
155     // Send email notification to owner (non-blocking)
156     fetch('/api/emails/booking-status', {
157       method: 'POST',
158       headers: { 'Content-Type': 'application/json' },
159       body: JSON.stringify({ bookingId: newBooking.id, status: 'pending' })
160     }).catch(console.error)
161
162     // Vytvoření chat roomu pro komunikaci - now returns just the ID
163     const chatRoomId = await db.createChatRoom({
164       item_id: item.id,
165       owner_id: item.owner_id,
166       borrower_id: user.id,
167       booking_id: ""
168     })
169
170     // Přidání první zprávy do chatu
171     if (chatRoomId) {
172       await db.sendChatMessage({
173         room_id: chatRoomId,
174         sender_id: user.id,
175         message: `Zdravím, mám zájem o půjčení předmětu "${item.title}" od ${selectedDates.from.toLocaleDateString("cs-CZ")} do ${selectedDates.to.toLocaleDateString("cs-CZ")}.${message ? ` ${message}` : ""}`,
176       })
177     }
```



4.4.6 Chatovací systém

Chatovací systém představuje jednu z nejkomplexnějších částí aplikace a zahrnuje real-time doručování zpráv, přikládání souborů, emoji reakce, odpovědi na zprávy, indikátor psaní a filtrování nevhodného obsahu. Systém je tvořen dvanácti specializovanými komponentami v adresáři *components/chat/* a vlastním hookem *useRealtimeChat* v adresáři *hooks/*.

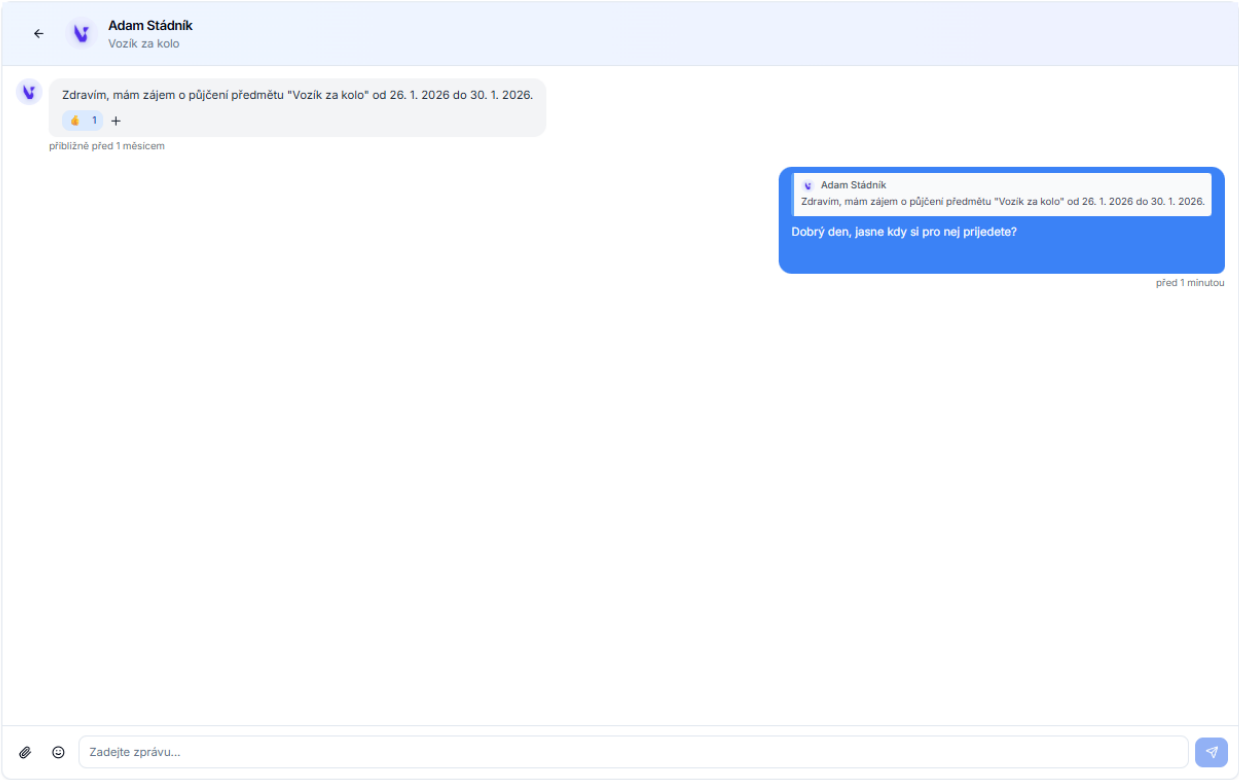
Jádrem implementace je hook *useRealtimeChat*, který zapouzdřuje veškerou logiku real-time komunikace. Při inicializaci je vytvořen kanál Supabase Realtime voláním *supabase.channel()* s konfigurací pro broadcast a presence. Kanál naslouchá změnám v tabulce *chat_messages* prostřednictvím události *postgres_changes* – při vložení nové zprávy je automaticky přidána do lokálního stavu, při úpravě je aktualizována a při smazání odebrána. Indikátor psaní je realizován prostřednictvím broadcastového kanálu, kde každý uživatel vysílá událost *typing* během psaní zprávy. Při odpojení komponenty je kanál řádně ukončen voláním *supabase.removeChannel()*, čímž se předchází únikům paměti.

```
41  // Inicializace realtime připojení
42  useEffect(() => {
43    if (!roomId || !currentUserId) return
44
45    isMountedRef.current = true
46    loadMessages()
47
48    // Vytvoření realtime kanálu
49    const channel = supabase.channel(`chat-room-${roomId}`, {
50      config: {
51        broadcast: { self: true },
52        presence: { key: currentUserId },
53      },
54    })
55
56    // Poslouchání změn v chat_messages
57    channel
58      .on(
```

Komponenta *ChatRoom* zobrazuje konverzaci mezi dvěma uživateli. Zprávy jsou vykresleny chronologicky s rozlišením vlastních a přijatých zpráv. U každé zprávy je k dispozici kontextové menu s možnostmi úpravy, smazání a odpovědi. Editace zprávy je provedena přímo v konverzaci prostřednictvím inline editačního pole. Smazání vyžaduje

potvrzení dialogovým oknem. Funkce odpovídání na konkrétní zprávu využívá komponentu *RepliedMessage*, která zobrazuje náhled původní zprávy nad odpovědí.

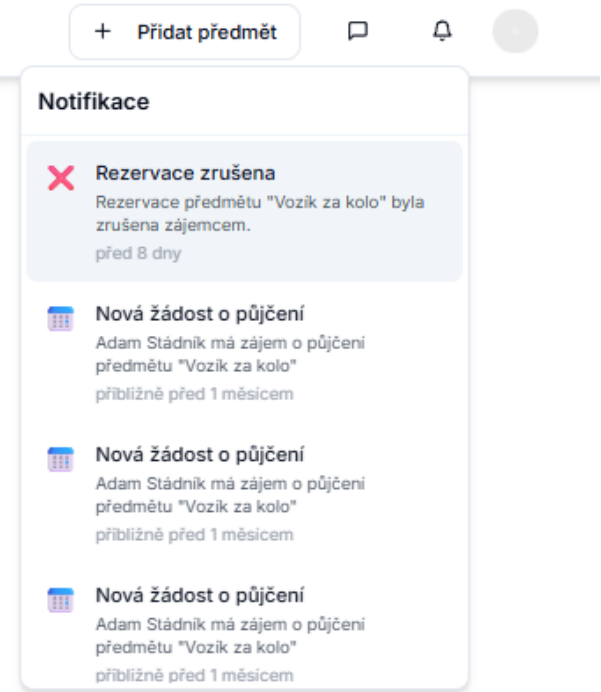
Odesílání zpráv je zajištěno komponentou *MessageInput*, která integruje textové pole, výběr emoji prostřednictvím komponenty *EmojiPicker* a nahrávání souborů komponentou *FileUpload*. Nahrané soubory jsou uloženy do bucketu *chat-files* v úložišti Supabase Storage a zobrazeny v konverzaci komponentou *MessageAttachment*. Před odesláním každé zprávy je text zpracován modulem *content-filter.ts*, který detekuje a filtruje nevhodný obsah. Pokud je ve zprávě detekován nevhodný obsah, je uživateli zobrazeno upozornění prostřednictvím komponenty *ContentWarning* a záznam je uložen do auditní tabulky.



4.4.7 Notifikace a hodnocení

Notifikační systém informuje uživatele o důležitých událostech v aplikaci prostřednictvím komponenty *NotificationDropdown*, která je integrována do navigačního panelu. Komponenta využívá vlastní hook *useNotifications*, který při inicializaci načte existující notifikace uživatele z databáze a následně naslouchá novým notifikacím prostřednictvím kanálu Supabase Realtime. Ikona zvonku v navigaci zobrazuje badge s počtem nepřečtených notifikací. Po kliknutí se rozbalí rozbalovací menu se seznamen posledních deseti upozornění, kde každá notifikace je vizuálně odlišena ikonou podle typu události – žádost o rezervaci, potvrzení, zrušení, nová zpráva nebo přijaté hodnocení. Kliknutí na konkrétní notifikaci ji označí jako přečtenou a přesměruje uživatele na příslušnou sekci aplikace (například záložku rezervací nebo zpráv v uživatelském profilu). K dispozici je rovněž tlačítko pro hromadné označení všech notifikací jako přečtených.

Notifikace jsou vytvářeny programově na klientské straně při klíčových akcích v systému. Při vytvoření nové rezervace je vlastníkovi odeslána notifikace typu *booking_request*, při potvrzení nebo zrušení rezervace je notifikován vypůjčitel notifikací typu *booking_confirmed* nebo *booking_cancelled*. Současně s databázovou notifikací je prostřednictvím API endpointu *api/emails/booking-status* odeslán email s využitím služby Resend.



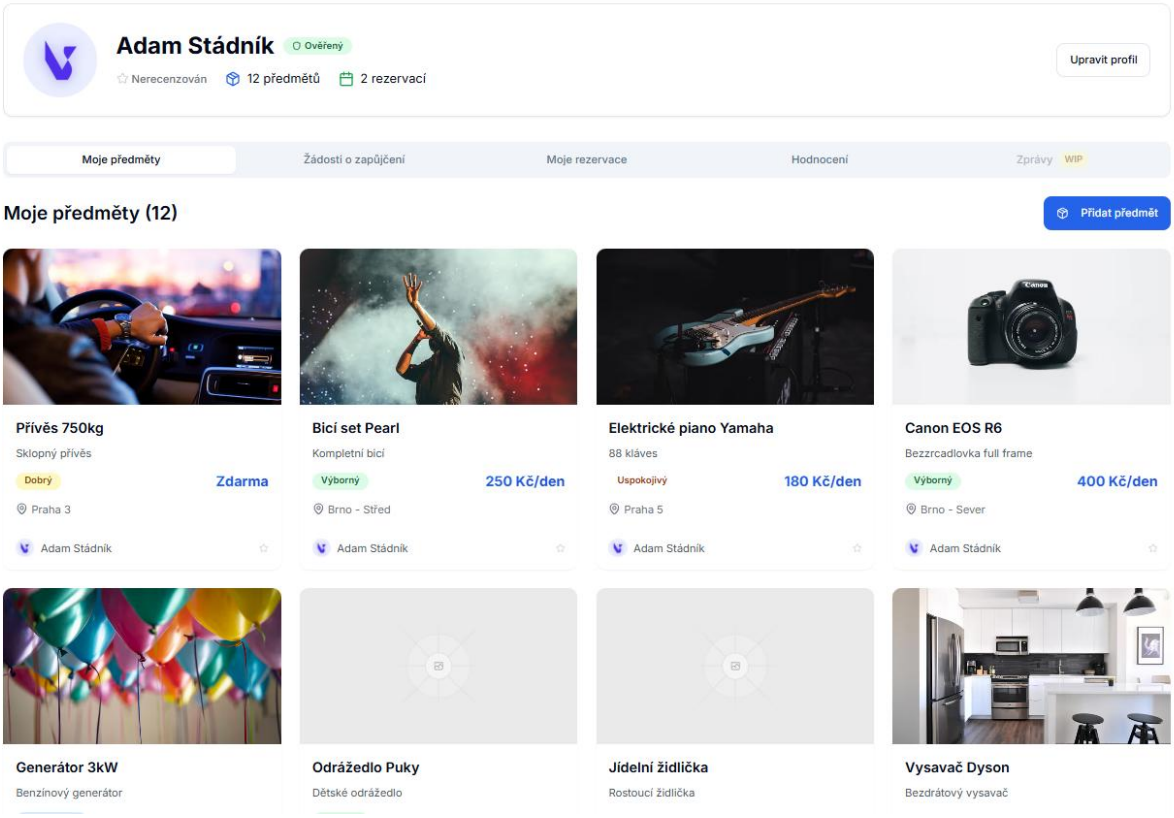
Systém hodnocení umožňuje oboustranné hodnoceni po dokončení výpůjčky. Po změně stavu rezervace na *completed* je v komponentě *BookingCard* zpřístupněn formulář pro vytvoření recenze. Formulář obsahuje hvězdičkové hodnocení na škále 1 až 5 a volitelný textový komentář. Hodnocení je uloženo do tabulky *reviews* voláním funkce *db.createReview()* s rozlišením typu hodnocení – vypůjčitel hodnotí půjčitele (*review_type: „lender“*) a naopak. Po odeslání hodnocení je hodnocenému uživateli vytvořena notifikace typu *review_received*.

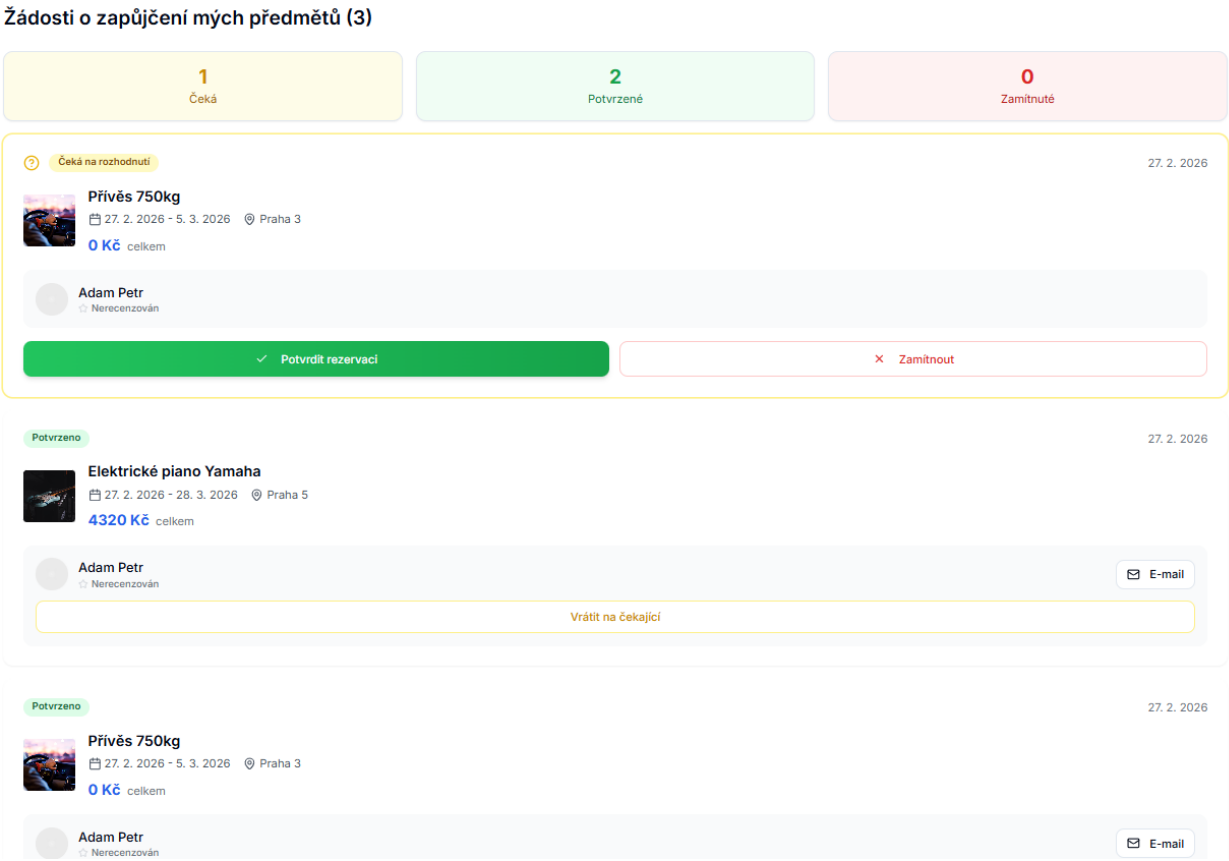
4.4.8 Uživatelský profil a správa výpůjček

Stránka uživatelského profilu (*app/profile/page.tsx*) slouží jako centrální přehled veškeré aktivity přihlášeného uživatele v aplikaci. Stránka je přístupná pouze přihlášeným uživatelům – nepřihlášení jsou automaticky přesměrováni na přihlašovací stránku. Při načtení jsou paralelně načtena data uživatele prostřednictvím metody *Promise.allSettled()*, která zajišťuje, že selhání jednoho dotazu neblokuje načtení ostatních dat. Načítají se předměty uživatele, přijaté hodnocení, chatovací místnosti a rezervace přijatých předmětů.

Profil je organizován do záložkového rozhraní prostřednictvím komponenty *Tabs* s pěti sekcemi. Záložka Moje předměty zobrazuje mřížku inzerátů vytvořených uživatelem s možností přidání nového předmětu. Záložka Moje rezervace obsahuje karty rezervací, kde uživatel vystupuje jako vypůjčitel, s přehledem stavu, termínu a kontaktních údajů majitele. Záložka Žádosti o vypůjčení zobrazuje rezervace předmětů, které uživatel vlastní, s tlačítky pro potvrzení nebo zamítnutí čekajících žádostí a počítadlem nevyřízených požadavků. Hodnoceni zobrazuje přijatá hodnocení od ostatních uživatelů.

V záhlaví stránky je zobrazen profil uživatele s avatarem, jménem, reputačním skórem, datem registrace a ověřovacím statusem. Uživatel má možnost upravit svůj profil včetně nahrání profilového obrázku, změny kontaktních údajů a nastavení soukromí. Aktivní záložka je synchronizována s URL parametrem *tab*, což umožňuje přímé odkazování na konkrétní sekci profilu – například notifikace přesměrovávají uživatele přímo na záložku *bookings* nebo *messages*.


Adam Stádník
Ověřený
Nerecenzován
12 předmětů
2 rezervací
Upravit profil
Moje předměty
Žádosti o zapůjčení
Moje rezervace
Hodnocení
Zprávy
WIP
Moje předměty (12)
Přidat předmět
Přívěs 750kg
Sklopný přívěs
Dobrý
Zdarma
Praha 3
Adam Stádník
Bicí set Pearl
Kompletní bicí
Výborný
250 Kč/den
Brno - Střed
Adam Stádník
Elektrické piano Yamaha
88 kláves
Uspokojivý
180 Kč/den
Praha 5
Adam Stádník
Canon EOS R6
Bezzrcadlovka full frame
Výborný
400 Kč/den
Brno - Sever
Adam Stádník
Generátor 3kW
Benzínový generátor
Odrážedlo Puky
Dětské odrážedlo
Jídelní židlička
Rostoucí židlička
Vysavač Dyson
Bezdrátový vysavač


Žádosti o zapůjčení mých předmětů (3)
1
Čeká
2
Potvrzené
0
Zamítnuté
Čeká na rozhodnutí
27. 2. 2026
Přívěs 750kg
27. 2. 2026 - 5. 3. 2026
Praha 3
0 Kč celkem
Adam Petr
Nerecenzován
Potvrdit rezervaci
Zamítnout
Potvrzeno
27. 2. 2026
Elektrické piano Yamaha
27. 2. 2026 - 28. 3. 2026
Praha 5
4320 Kč celkem
Adam Petr
Nerecenzován
E-mail
Vrátit na čekající
Potvrzeno
27. 2. 2026
Přívěs 750kg
27. 2. 2026 - 5. 3. 2026
Praha 3
0 Kč celkem
Adam Petr
Nerecenzován
E-mail

4.5 Zabezpečení a optimalizace

Tato kapitola se věnuje dvěma klíčovým aspektům, které doplňují implementaci popsanou v předchozí kapitole: zabezpečení dat na úrovni databáze a optimalizaci aplikace pro vyhledávače. Zatímco kapitola 4.3.2 představila koncept Row Level Security a jeho roli v architektuře projektu, následující podkapitola detailně rozebírá konkrétní bezpečnostní politiky implementované v databázi PostgreSQL. Podkapitola o SEO optimalizaci poté ukazuje praktickou aplikaci teoretických principů popsaných v kapitole 3.10.

4.5.1 Row Level Security

Bezpečnostní model aplikace je postaven na mechanismu Row Level Security popsaném v teoretické části. Veškerá přístupová pravidla jsou definována přímo v databázi formou SQL politik, čímž je zajištěno, že bezpečnostní omezení jsou vynucována nezávisle na klientské aplikaci. V praxi jsou pro každou tabulku definovány samostatné politiky pro jednotlivé databázové operace (*SELECT, INSERT, UPDATE, DELETE*), přičemž každá politika obsahuje podmínku *USING* nebo *WITH CHECK*, která je vyhodnocena při každém přístupu k datům.

V rámci implementace byly identifikovány čtyři základní vzory bezpečnostních politik, které pokrývají různé scénáře přístupu k datům:

Přímá kontrola vlastnictví

Nejjednodušší vzor ověřuje, zda je přihlášený uživatel vlastníkem daného záznamu. Funkce *auth.uid()* extrahuje identifikátor uživatele z JWT tokenu a porovnává jej s hodnotou cizího klíče v tabulce. Tento vzor je aplikován například v tabulce *chat_messages*, kde politika pro mazání zpráv obsahuje podmínku *sender_id = auth.uid()*, čímž je zajištěno, že uživatel může smazat pouze vlastní zprávy. Stejný princip je využit i pro úpravu zpráv.

Kontrola členství prostřednictvím poddotazu

Komplexnější vzor využívá poddotaz (subquery) pro ověření přístupu na základě relace mezi tabulkami. Tabulka *chat_messages* aplikuje tento vzor pro operace čtení a vkládání zpráv. Politika pro čtení obsahuje poddotaz do tabulky *chat_rooms*, který ověřuje, zda je přihlášený uživatel jedním z účastníků dané chatovací místnosti (*owner_id* nebo

borrower_id). Toto řešení zajišťuje, že uživatel může číst pouze zprávy v konverzacích, jichž se účastní, a to bez nutnosti ukládat oprávnění redundantně u každé zprávy.

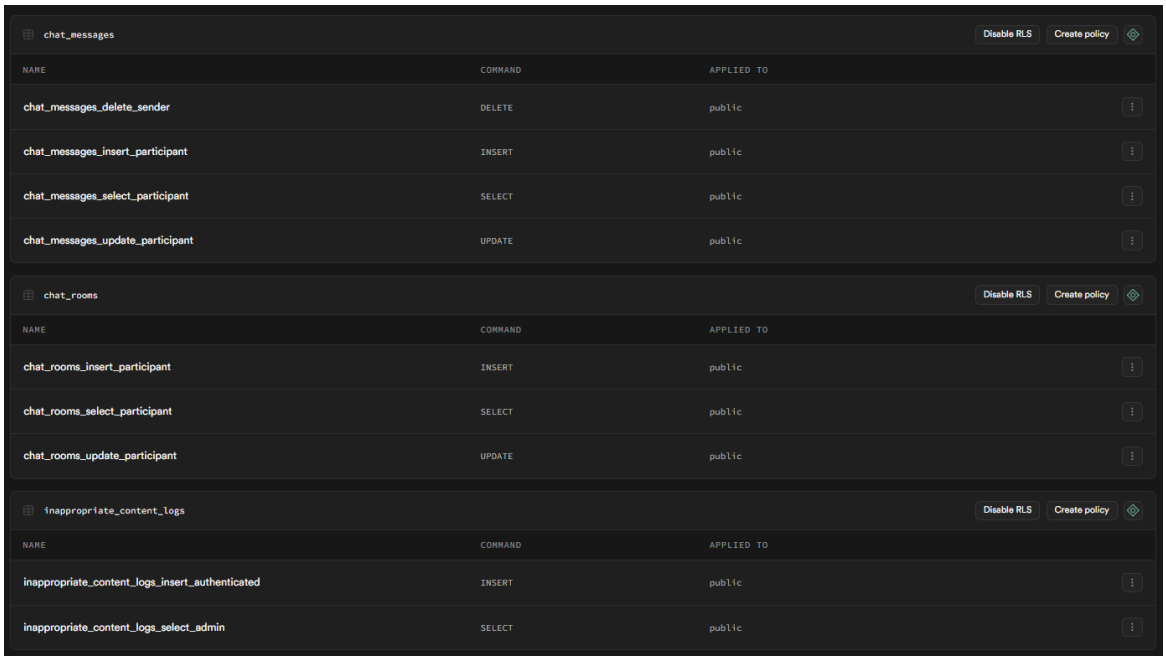
Kontrola administrátorské role

Pro auditní tabulky, jako je *content_moderation_logs*, je implementována politika, která povoluje čtení záznamů pouze administrátorům systému. Podmínka *USING* obsahuje poddotaz do tabulky *users*, který ověřuje, zda má přihlášený uživatel příznak *is_admin* nastaven na hodnotu *true*. Tím je zajištěno, že záznamy o filtrovaných zprávách a nevhodném obsahu jsou přístupné pouze oprávněným osobám.

Otevřený přístup pro čtení

Některé tabulky, jako jsou *message_reactions* nebo *categories*, povolují čtení všem uživatelům bez omezení (podmínka *USING* nastavena na true). Zápis je však omezen na ověřené uživatele, přičemž u reakcí je navíc vyžadována shoda *user_id* s identifikátorem přihlášeného uživatele, aby uživatel nemohl vkládat reakce pod cizím jménem.

Kombinace těchto vzorů zajišťuje víceúrovňové zabezpečení, kde každá tabulka má pravidla odpovídající její roli v systému. Klíčovou výhodou tohoto přístupu oproti implementaci autorizace v aplikačním kódu je skutečnost, že bezpečnostní pravidla jsou vynucována na úrovni databáze bez ohledu na způsob přístupu k datům, čímž je eliminováno riziko obejití kontroly prostřednictvím přímého volání API.



chat_messages — Disable RLS — Create policy

NAME	COMMAND	APPLIED TO
chat_messages_delete_sender	DELETE	public
chat_messages_insert_participant	INSERT	public
chat_messages_select_participant	SELECT	public
chat_messages_update_participant	UPDATE	public

chat_rooms — Disable RLS — Create policy

NAME	COMMAND	APPLIED TO
chat_rooms_insert_participant	INSERT	public
chat_rooms_select_participant	SELECT	public
chat_rooms_update_participant	UPDATE	public

inappropriate_content_logs — Disable RLS — Create policy

NAME	COMMAND	APPLIED TO
inappropriate_content_logs_insert_authenticated	INSERT	public
inappropriate_content_logs_select_admin	SELECT	public

4.5.2 SEO optimalizace

Optimalizace pro vyhledávače je v aplikaci realizována kombinací technik popsaných v teoretické části. Základním předpokladem je využití Server-Side Renderingu frameworku Next.js, díky kterému vyhledávací roboti obdrží při indexaci kompletní HTML obsah stránky namísto prázdného kontejneru s odkazem na JavaScript, jak by tomu bylo u čistě klientsky renderovaných aplikací.

Metadata a jazyková deklarace

Globální metadata aplikace jsou definována v kořenovém souboru *app/layout.tsx* prostřednictvím exportu objektu *metadata*, který je standardním rozhraním frameworku Next.js. Titulek stránky je nastaven na „SousedePůjč – Půjčování věcí mezi sousedy“ a popisek (*meta description*) stručně charakterizuje účel platformy. Tyto údaje jsou v souladu s doporučeními z kapitoly 3.10.1 klíčové pro zobrazení stránky ve výsledcích vyhledávání – titulek se zobrazuje jako hlavní nadpis snippetu a popisek ovlivňuje míru prokliku (CTR). Element *html* obsahuje atribut *lang* s hodnotou *cs*, který vyhledávačům a asistivním technologiím jednoznačně sděluje jazyk obsahu stránky.

Sémantická struktura a přístupnost

Aplikace využívá sémantické HTML5 elementy popsané v kapitole 3.3, jako jsou *header* pro navigační panel, *main* pro hlavní obsah stránky a *footer* pro patičku. Tato struktura umožňuje vyhledávacím robotům správně identifikovat hierarchii obsahu a zlepšuje přístupnost pro uživatele s asistivními technologiemi. Komponenty z knihovny Radix UI, která tvoří základ komponentové knihovny aplikace, implementují standardy WAI-ARIA a zajišťují správné atributy role a aria u interaktivních prvků, jako jsou dialogová okna, rozbalovací nabídky nebo záložkové rozhraní.

Souborový systém směrování a čisté URL adresy

Architektura App Router frameworku Next.js přináší inherentní výhodu v podobě čistých a popisných URL adres. Struktura adresáře *app/* se přímo promítá do URL cest – například stránka detailu předmětu je dostupná na adrese */items/[id],* stránka pro vytvoření nového inzerátu na */items/new* a uživatelský profil na */profile*. Oproti jednostránkovým aplikacím, kde je veškerý obsah často dostupný pod jedinou URL s fragmentovými identifikátory, umožňuje tento přístup vyhledávačům indexovat každou stránku samostatně.

4.6 Testování

Ověření funkčnosti a kvality aplikace je nedílnou součástí vývojového procesu. Tato kapitola popisuje testování aplikace SousedePůjč ze tří hledisek: funkčního testování prostřednictvím manuálních testovacích scénářů, uživatelského testování zaměřeného na ověření použitelnosti a měření výkonu pomocí nástroje Lighthouse. Zatímco funkční testy ověřují splnění požadavků definovaných v kapitole 4.1.2, uživatelské testování navazuje na principy použitelnosti popsané v kapitole 3.9.1 a měření výkonu na metriky Core Web Vitals z kapitoly 3.10.2.

4.6.1 Testovací scénáře

Funkční testování aplikace bylo provedeno formou manuálních testovacích scénářů, které systematicky ověřují splnění každého funkčního požadavku definovaného v analytické části. Každý scénář definuje výchozí podmínky, kroky provedení a očekávaný výsledek. Testování bylo prováděno na nasazené verzi aplikace dostupné na www.sousedepujc.vercel.app v prohlížeči Brave.

Scénář	Popis	Očekávaný výsledek	Výsledek
TS-01	Registrace nového uživatele	Účet vytvořen, e-mail k ověření, přesměrování	Splněno
TS-02	Přihlášení s ověřeným účtem	Uživatel přihlášen, přesmerování na home	Splněno
TS-03	Přihlášení s neověřeným e-mailem	Popup s opětovným odeslání ověř. e-mailu	Splněno
TS-04	Vyhledání předmětu	Odpovídající předměty, fuzzy vyhledávání	Splněno
TS-05	Filtrování podle kategorie	Odpovídající kategorie, URL parametry	Splněno
TS-06	Filtrování podle ceny a řazení	Daný rozsah ve správném pořadí	Splněno
TS-07	Vytvoření nového inzerátu	Inzerát vytvořen, přesměrování na detail	Splněno
TS-08	Úprava existujícího inzerátu	Údaje aktualizovaný v databázi	Splněno
TS-09	Smazání inzerátu	Odstraněn z databáze a katalogu	Splněno
TS-10	Odeslání žádosti o rezervaci	Rezervace vytvořena, vlastník notifikován	Splněno
TS-11	Ověření blokace termínů	Obsazené dny vizuálně odlišeny	Splněno
TS-12	Kalkulace ceny + sleva	Automatická sleva, přepočítaná cena	Splněno
TS-13	Potvrzení rezervace vlastníkem	Stav změněn na confirmed, notifikace e-mailem	Splněno
TS-14	Zamítnutí rezervace	Stav na cancelled, notifikace	Splněno
TS-15	Odeslání zprávy v chatu	Zpráva zobrazena v konverzaci	Splněno
TS-16	Úprava a smazání zprávy	Zpráva aktualizována a odstraněna	Splněno
TS-17	Přidání emoji reakce	Reakce zobrazena pod zprávou	Splněno
TS-18	Příjem notifikace	Ikona zvonku zobrazuje nepřečtené notifikace	Splněno
TS-19	Odeslání hodnocení	Hodnocení uloženo	Splněno
TS-20	Úprava uživatelského profilu	Profil aktualizován	Splněno

Tabulka 2 - Přehled testovacích scénářů a jejich výsledků

4.6.2 Uživatelské testování

Pro ověření použitelnosti aplikace bylo provedeno uživatelské testování s využitím standardizovaného dotazníku System Usability Scale (SUS). Dotazník byl distribuován prostřednictvím Google Forms a skládal se ze dvou částí: praktických úkolů ověřujících klíčové uživatelské toky a dotazníku použitelnosti. Testování se zúčastnilo 15 respondentů, kteří pracovali s nasazenou verzí aplikace.

Testovací úkoly

V první části dotazníku byli respondenti požádáni o splnění sedmi úkolů pokrývajících hlavní funkce aplikace: registrace a přihlášení, vyhledání předmětu, zobrazení detailu předmětu, vytvoření nového inzerátu, odeslání žádosti o rezervaci, odeslání zprávy v chatu a kontrolu notifikací. U každého úkolu respondenti uvedli, zda se jej podařilo splnit, a ohodnotili jeho obtížnost na škále 1 (velmi snadný) až 5 (velmi obtížný).

Všech 15 respondentů úspěšně splnilo všechny testovací úkoly, čímž byla dosažena stoprocentní míra úspěšnosti. Průměrná obtížnost se u všech úkolů pohybovala v rozmezí 1,00 až 1,40, což odpovídá hodnocení „velmi snadný“. Nejvyšší průměrnou obtížnost vykázaly úkoly vytvoření nového inzerátu a odeslání žádosti o rezervaci (shodně 1,40), které vyžadují interakci s formulářem a kalendářem. Úkoly vyhledání předmětu, odeslání zprávy v chatu a kontrola notifikací byly hodnoceny jako nejsnazší s průměrnou obtížností 1,00.

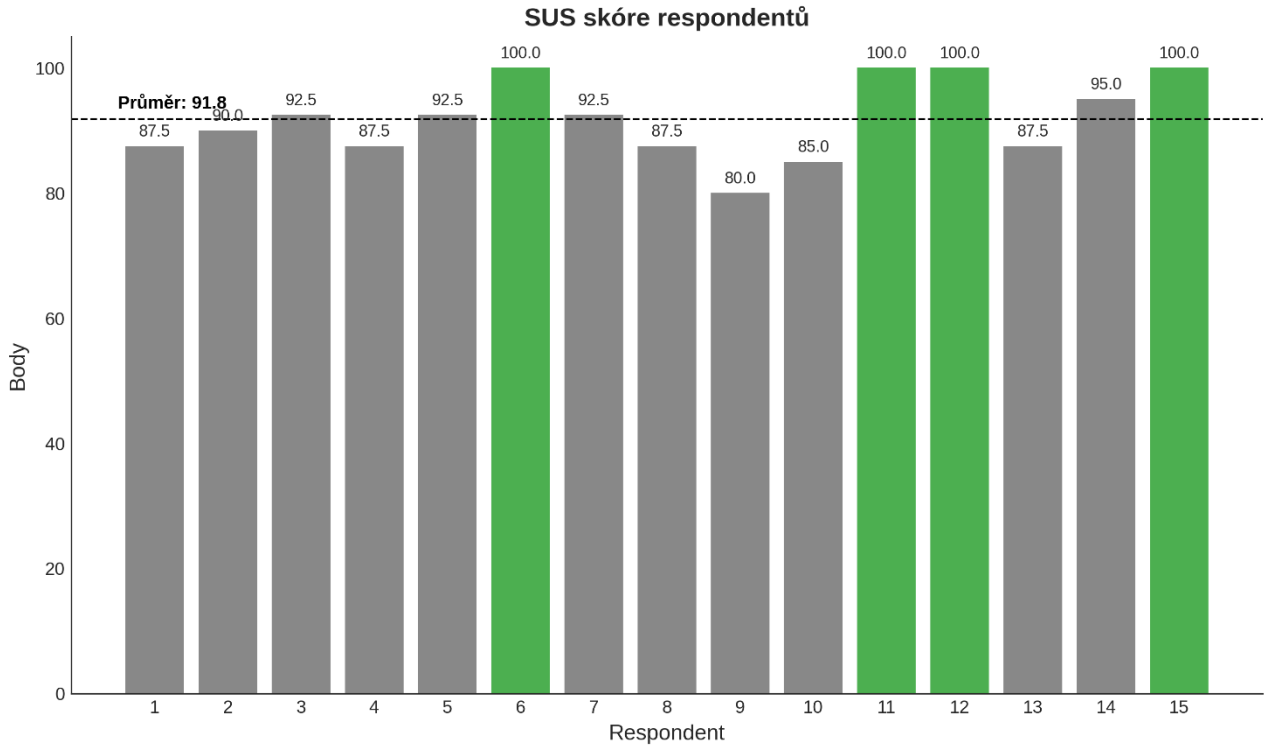
Úkol	Popis	Úspěšnost	Ø obtížnost
1	Registrace a přihlášení	100%	1,13
2	Vyhledání předmětu	100%	1,00
3	Zobrazení detailu předmětu	100%	1,07
4	Vytvoření nového předmětu	100%	1,40
5	Odeslání žádosti o rezervaci	100%	1,40
6	Odeslání zprávy v chatu	100%	1,00
7	Kontrola notifikací	100%	1,00

Tabulka 3 - Výsledky testovacích úkolů

Dotazník použitelnosti (SUS)

Druhá část dotazníku obsahovala deset standardizovaných výroků metodiky System Usability Scale, kde respondenti vyjadřovali míru souhlasu na škále 1 (zcela nesouhlasím) až 5 (zcela souhlasím). Metodika SUS střídá pozitivně a negativně formulované výroky, čímž minimalizuje riziko bezmyšlenkovitého vyplňování. Výsledné skóre je přepočteno na škálu 0-100 bodů, přičemž hodnoty nad 68 jsou považovány za nadprůměrné a hodnoty nad 80 za vynikající.

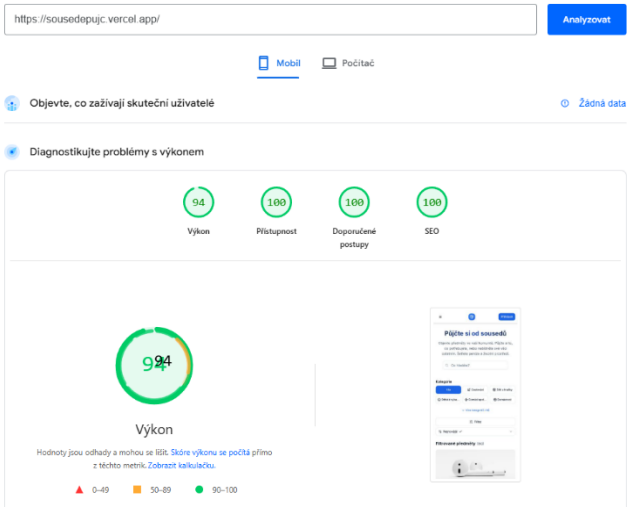
Průměrné SUS skóre aplikace dosáhlo hodnoty 91,8 bodů ze 100, což ji řadí do kategorie „vynikající“ použitelnosti. Nejnižší individuální skóre činilo 80,0 bodů a nejvyšší 100,0 bodů. Celkem čtyři respondenti ohodnotili aplikaci maximálním skórem 100 bodů.

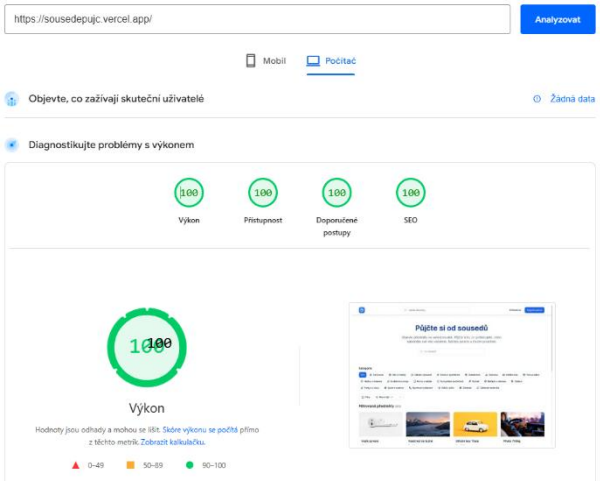


Tyto výsledky potvrzují, že uživatelské rozhraní aplikace je v souladu s principy použitelnosti definovanými v kapitole 3.9.1 – respondenti hodnotili aplikaci jako efektivní, snadno ovladatelnou a nevyžadující technickou asistenci.

4.6.3 Měření výkonu a SEO

Pro objektivní hodnocení výkonu a optimalizace aplikace byl využit nástroj Lighthouse, který je integrován přímo v prohlížeči Google Chrome. Tento nástroj provádí automatizovaný audit webové stránky a vyhodnocuje ji ve čtyřech kategoriích: výkon, přístupnost, osvědčené postupy a optimalizace pro vyhledávače (SEO). Výsledky jsou vyjádřeny na škále 0-100 bodů, přičemž hodnoty nad 90 jsou hodnoceny jako vynikající. Měření bylo provedeno na nasazené produkční verzi aplikace.





Metrika Largest Contentful Paint (LCP), popsaná v kapitole 3.10.2, měří dobu vykreslení největšího viditelného prvku na stránce. Dosažená hodnota je přímo ovlivněna využitím Server-Side Renderingu, díky kterému prohlížeč obdrží kompletní HTML obsah bez nutnosti čekat na stažení a spuštění JavaScriptu. Metrika Cumulative Layout Shift (CLS) měří vizuální stabilitu stránky a aplikace dosahuje nízkých hodnot díky definovaným rozměrům obrázků a konzistentnímu rozložení komponent.

Výkon aplikace je dále optimalizován kompresí HTTP odpovědí, minifikací kódu kompilátorem SWC, automatickou konverzí obrázků do moderních formátů AVIF a WebP s dobou platnosti cache nastavenou na jeden rok a direktivami *dns-prefetch* a *preconnect* pro minimalizaci latence při komunikaci s databází Supabase.

Výsledky měření prokázaly vysokou kvalitu aplikace napříč všemi hodnocenými kategoriemi. V režimu desktopového zařízení dosáhla aplikace maximální hodnoty 100 bodů ve všech čtyřech kategoriích. V režimu mobilního zařízení se zpomaleným připojením

4G dosáhla skóre přístupnosti, osvědčených postupů a SEO rovněž maximální hodnoty 100 bodů, skóre výkonu pak 94 bodů ze 100.

Z jednotlivých metrik výkonu dosáhla hodnota First Contentful Paint 0,9 sekundy a Speed Index rovněž 0,9 sekundy, což indikuje rychlé počáteční vykreslení obsahu. Metrika Cumulative Layout Shift dosáhla hodnoty 0,054, což je výrazně pod doporučeným limitem 0,1 a potvrzuje vysokou vizuální stabilitu stránky. Hodnota Total Blocking Time činila 50 milisekund, což značí minimální blokaci hlavního vlákna prohlížeče. Metrika Largest Contentful Paint dosáhla hodnoty 3,0 sekundy, což mírně překračuje doporučený limit 2,5 sekundy. Tato hodnota je ovlivněna testovacími podmínkami simulace mobilního zařízení se zpomaleným připojením 4G a načítáním dat z databáze Supabase při prvním vykreslení stránky.

4.7 Nasazení aplikace

Aplikace SousedePůjč je nasazena na cloudové platformě Vercel, která je popsána v teoretické části. Proces nasazení využívá přímou integraci platformy Vercel se systémem správy verzí GitHub, čímž je realizován plně automatizovaný cyklus průběžného nasazení (CI/CD). Zdrojový kód aplikace je hostován ve veřejném repozitáři na platformě GitHub a aplikace je veřejně dostupná na adrese www.sousedepujc.vercel.app.

Průběžné nasazení

Proces nasazení je zahájen automaticky při každém odeslání změn (*git push*) do hlavní větve repozitáře. Platforma Vercel detekuje nový commit, stáhne zdrojový kód, provede sestavení aplikace příkazem *next build* a výsledný artefakt nasadí na produkční prostředí. Celý proces probíhá bez nutnosti manuálního zásahu a trvá řádově desítky sekund. Při odeslání změn do vedlejší větve Vercel automaticky vytváří takzvané Preview Deployment – dočasné nasazení s unikátní URL adresou, které umožňuje otestovat změny v reálném prostředí před jejich sloučením do hlavní větve.

Konfigurace prostředí

Proměnné prostředí nezbytné pro komunikaci s platformou Supabase (*NEXT_PUBLIC_SUPABASE_URL* a *NEXT_PUBLIC_SUPABASE_ANON_KEY*) jsou nakonfigurovány v nastavení projektu na platformě Vercel. Tyto hodnoty nejsou součástí zdrojového kódu ani repozitáře, čímž je zajištěna bezpečnost přístupových údajů.

5 Zhodnocení výsledků

Cílem této kapitoly je zhodnotit dosažené výsledky práce ve vztahu ke stanoveným cílům definovaným v kapitole 2.1 a požadavkům specifikovaným v kapitole 4.1.

Splnění cílů práce

Hlavním cílem práce byl návrh a implementace webové aplikace pro komunitní půjčování předmětů. Tento cíl byl naplněn vytvořením aplikace SousedePůjč, která je nasazena na platformě Vercel a veřejně dostupná. Aplikace umožňuje uživatelům registraci, správu inzerátů, vyhledávání a filtrování předmětů, rezervaci prostřednictvím interaktivního kalendáře, real-time komunikaci a vzájemné hodnocení.

Dílčí cíl analýzy problematiky sdílené ekonomiky byl splněn v teoretické části (kapitola 3.1), kde byly prozkoumány principy P2P modelu, environmentální aspekty a analyzovány existující platformy na českém trhu. Dílčí cíl prozkoumání moderních technologií byl naplněn v kapitolách 3.6 a 3.7, kde byly podrobně popsány framework Next.js a platforma Supabase. Vytvoření UI/UX návrhu bylo realizováno v kapitole 4.2 formou design systému a wireframů. Ověření funkčnosti formou testování bylo provedeno v kapitole 4.6.

Splnění funkčních požadavků

V analytické části (kapitola 4.1.2) bylo definováno pět klíčových funkčních modulů tvořících minimální životaschopný produkt. Všechny tyto moduly byly úspěšně implementovány:

Modul autentizace a správy profilu je plně funkční – uživatelé se registrují pomocí e-mailové adresy s ověřením, disponují veřejným profilem s avatarem a reputačním skóre. Toto řešení přímo adresuje problém anonymity identifikovaný v teoretické rešerši jako hlavní bariéru sdílení.

Modul interaktivní mapy a geolokace byl implementován prostřednictvím knihovny Leaflet – při vytváření inzerátu uživatel zadává lokaci s podporou našeptávače adres. Toto

řešení reaguje na selhání platformy SharyGo, kde logistická náročnost představovala kritický faktor.

Strukturovaný katalog a správa inzerce je realizován systémem kategorií s fuzzy vyhledáváním a pokročilým filtrováním podle cenového rozsahu, dostupnosti a šesti kritérií řazení. Na rozdíl od nestrukturované inzerce na sociálních sítích nabízí aplikace jasnou kategorizaci se synchronizací filtrů do URL parametrů.

Rezervační systém představuje klíčovou funkci odlišující aplikaci od existujících řešení. Interaktivní kalendář vizuálně odlišuje obsazené termíny, automaticky kalkuluje cenu včetně množstevních slev a po odeslání žádosti vytváří notifikaci, e-mailové upozornění a chatovací místnost. Tento modul přímo řeší absenci rezervačního systému identifikovanou v analýze existujících platforem.

Systém hodnocení umožňuje oboustranné hodnocení po dokončení výpůjčky na škále 1 až 5 s volitelným komentářem, čímž implementuje reputační mechanismus popsaný v teoretické části jako klíčový prvek P2P platforem.

Nad rámec původních požadavků byly implementovány další funkce: real-time chatovací systém s emoji reakcemi, odpověďmi na zprávy, přikládáním souborů a filtrováním nevhodného obsahu, e-mailové notifikace prostřednictvím služby Resend a systém in-app notifikací.

Splnění nefunkčních požadavků

Výkon aplikace byl ověřen nástrojem Lighthouse (kapitola 4.6.3), kde aplikace dosáhla skóre 100 bodů ve všech kategoriích na desktopu a 94 bodů za výkon na mobilním zařízení. Responzivita je zajištěna důslednou aplikací Mobile-First přístupu prostřednictvím frameworku Tailwind CSS. Bezpečnost dat je realizována mechanismem Row Level Security na úrovni databáze (kapitola 4.5.1). Dostupnost a škálovatelnost jsou zajištěny serverless architekturou platformy Vercel a cloudovou databází Supabase.

Výsledky testování

Funkční testování prostřednictvím 20 testovacích scénářů (kapitola 4.6.1) potvrdilo stoprocentní splnění všech definovaných požadavků. Uživatelské testování s 15 respondenty (kapitola 4.6.2) prokázalo vynikající použitelnost aplikace – průměrné SUS skóre dosáhlo hodnoty 91,80 bodů ze 100, přičemž všichni respondenti úspěšně splnili všechny testovací úkoly s průměrnou obtížností v rozmezí 1,00 až 1,40 na pětibodové škále.

Omezení a náměty pro další rozvoj

Přestože aplikace splňuje všechny definované požadavky, existují oblasti pro další rozvoj. Platební systém v současné verzi nepodporuje online platby a předpokládá hotovostní vyrovnání při osobním předání – integrace platební brány by umožnila bezhotovostní transakce. Dále by bylo přínosné implementovat pokročilejší geolokační filtrování s automatickou detekcí polohy uživatele a filtrováním podle skutečné vzdálenosti. Rozšíření o mobilní aplikaci prostřednictvím frameworku React Native by mohlo zvýšit uživatelský komfort při použití v terénu.

6 Závěr

Tato bakalářská práce si kladla za cíl navrhnout a implementovat webovou aplikaci pro komunitní půjčování předmětů, která uživatelům v blízkém okolí usnadní vzájemné sdílení a zapůjčování vybavení. Na základě stanovených cílů a zvolené metodiky byl tento záměr úspěšně naplněn.

V teoretické části byla provedena rešerše problematiky sdílené ekonomiky a analýza existujících řešení na trhu, z níž vyplynuly klíčové nedostatky současných platforem – zejména absence strukturovaného katalogu, automatizovaného rezervačního systému a důrazu na lokální komunitu. Dále byl podrobně popsán zvolený technologický stack zahrnující framework Next.js, programovací jazyk TypeScript a cloudovou platformu Supabase.

Praktická část zahrnovala kompletní vývojový proces od analýzy požadavků a definice cílových skupin přes návrh wireframů s důrazem na Mobile-First přístup až po implementaci databázového modelu o jedenácti relačních tabulkách se zabezpečením prostřednictvím Row Level Security. Celkova architektura projektu je postavena na principu Backend-as-a-Service, který eliminuje nutnost vlastního backendového serveru.

Výsledná aplikace SousedePůjč implementuje všechny definované funkční požadavky: autentizaci, strukturovaný katalog, rezervační kalendář, chatovací systém a reputační mechanismus založený na vzájemném hodnoceni. Aplikace je nasazena na platformě Vercel a je veřejně dostupná.

Testování potvrdilo vysokou kvalitu řešení – funkční testy prokázaly splnění všech požadavků, uživatelské testování s 15 respondenty s průměrným SUS skóre 91,80 bodu a audit nástrojem Lighthouse potvrdil vynikající výkon a přístupnost.

Přínosem práce je vytváření funkční platformy. Která adresuje identifikované nedostatky existujících řešení a demonstruje možnosti moderních webových technologií pro vývoj komunitních aplikací. Výsledek může sloužit jako základ pro budoucí rozvoj v oblasti lokálního sdílení předmětů.

7 Seznam použitých zdrojů

[1] DELOITTE. *Sdílená ekonomika: Informace pro veřejnou správu*. Online. Ministerstvo pro místní rozvoj ČR, 2017. Dostupné z: https://mmr.gov.cz/getmedia/49c70327-7df4-49e9-b556-9d760f9f151e/CJ_Sdilena-ekonomika-Deloitte.pdf.aspx?ext=.pdf. [cit. 2025-12-09].

[2] SCHOLZOVÁ, Lucie. Dvě tváře sdílené ekonomiky. Online. 2020. Dostupné z: https://medium.com/edtech-kisk/dv%C4%9B-tv%C3%A1%C5%99e-sd%C3%ADlen%C3%A9-ekonomiky-8f83930765a. [cit. 2025-12-09].

[3] KUDLÁČEK, Patrik; KUBALOVÁ, Veronika a FINEX. P2P půjčky – Jak fungují a jak do nich investovat v roce 2025? Online. 2024. Dostupné z: https://finex.cz/rubrika/investice/p2p-pujcky/. [cit. 2025-12-09].

[4] SMPL. P2P (peer-to-peer). Online. 2024. Dostupné z: https://www.smpl.cz/glosar/p2p-peer-to-peer. [cit. 2025-12-09].

[5] NONDEK, Lubomír. Jak uchopit udržitelnost? Online. 2007. Dostupné z: https://www.cdk.cz/jak-uchopit-udrzitelnost. [cit. 2025-12-09].

[6] WOLF, Jiří. Vývoj internetu a webových technologií. Online. 2024. Dostupné z: https://www.jw.cz/vyvoj-internetu-webovych-technologii. [cit. 2025-12-10].

[7] ŽIELINSKÝ, Tomáš. Moderní webové technologie. Online. 2025. Dostupné z: https://zielinsky.cz/studijni-materialy/moderni-webove-technologie/. [cit. 2025-12-10].

[8] JANOVSKÝ, Dušan. HTTP protokol. Online. 2006. Dostupné z: https://www.jakpsatweb.cz/server/http-protokol.html. [cit. 2025-12-10].

[9] MINISTERSTVO VNITRA ČESKÉ REPUBLIKY. Popis API. Online. 2022. Dostupné z: Ministerstvo vnitra České republiky, https://opendata.mvcr.cz/popis-api/. [cit. 2025-12-10].

[10] ITNETWORK.CZ. Stopařův průvodce REST API. Online. Dostupné z: https://www.itnetwork.cz/programovani/nezarazene/stoparuv-pruvodce-rest-api. [cit. 2025-12-10].

[11] MDN_. HTML: HyperText Markup Language. Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/HTML. [cit. 2026-02-03].

[12] JAK PSÁT WEB. HTML příručka. Online. Dostupné z: https://www.jakpsatweb.cz/html/. [cit. 2026-02-03].

[13] MDN_. CSS: Cascading Style Sheets. Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/CSS. [cit. 2026-02-03].
[14] JAK PSÁT WEB. CSS Kaskádové styly. Online. Dostupné z: https://www.jakpsatweb.cz/css/. [cit. 2026-02-03].
[15] TailwindCSS. Online. Dostupné z: https://tailwindcss.com/. [cit. 2026-02-03].
[16] MICHÁLEK, Martin. Tailwind CSS: další evoluční krok pro CSS frameworky. Online. Dostupné z: https://www.vzhurudolu.cz/prirucka/tailwind-css. [cit. 2026-02-03].
[17] MICROSOFT. TypeScript is JavaScript with syntax for types. Online. Dostupné z: https://www.typescriptlang.org/. [cit. 2026-02-03].
[18] CZECHITAS. Úvod do Typescriptu. Online. Dostupné z: https://kodim.cz/czechitas/react-2/lekce/typescript/uvod-do-typescriptu. [cit. 2026-02-03].
[19] GEEKSFORGEEKS. Introduction to TypeScript. Online. Dostupné z: https://www.geeksforgeeks.org/typescript/introduction-to-typescript/. [cit. 2026-02-03].
[20] MDN_. JavaScript. Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/JavaScript#:~:text=JavaScript%20(JS)%20is%20a%20lightweight,Apache%20CouchDB%20and%20Adobe%20Acrobat.. [cit. 2026-02-03].
[21] KANTOR, Ilya. An Introduction to JavaScript. Online. Dostupné z: https://javascript.info/intro. [cit. 2026-02-03].
[22] ECMA INTERNATIONAL. ECMAScript. Online. Dostupné z: https://tc39.es/. [cit. 2026-02-03].
[23] VERCEL. The React Framework for the Web. Online. Dostupné z: https://nextjs.org/. [cit. 2026-02-03].
[24] VERCEL. Next.js Docs. Online. Dostupné z: https://nextjs.org/docs. [cit. 2026-02-03].
[25] SANITY. Next.js overview. Online. Dostupné z: https://www.sanity.io/glossary/next-js. [cit. 2026-02-03].
[26] META. React The library for web and native user interfaces. Online. Dostupné z: https://react.dev/. [cit. 2026-02-03].
[27] DODDS, Kent. What is React? Online. Dostupné z: https://www.epicreact.dev/what-is-react. [cit. 2026-02-03].
[28] MDN_. Server-side rendering (SSR). Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Glossary/SSR. [cit. 2026-02-03].

[29] MDN_. Client-side rendering (CSR). Online. Dostupné z: https://developer.mozilla.org/en-US/docs/Glossary/CSR. [cit. 2026-02-03].
[30] SUPABASE. The Open Source Firebase Alternative. Online. Dostupné z: https://supabase.com/. [cit. 2026-02-04]
[31] KOSISOCHUKWU, Melvin. What is Supabase. Online. Dostupné z: https://bejamas.com/hub/serverless-database/supabase. [cit. 2026-02-03].
[32] CLOUDFARE. What is BaaS? | Backend-as-a-Service vs. serverless. Online. Dostupné z: https://www.cloudflare.com/learning/serverless/glossary/backend-as-a-service-baas/. [cit. 2026-02-03].
[33] SUPABASE. Row Level Security. Online. Dostupné z: https://supabase.com/docs/guides/database/postgres/row-level-security. [cit. 2026-02-03].
[34] SUPABASE. Auth. Online. Dostupné z: https://supabase.com/docs/guides/auth. [cit. 2026-02-03].
[35] IBM. What is a relational database? Online. Dostupné z: https://www.ibm.com/think/topics/relational-databases. [cit. 2026-02-03].
[36] ORACLE. What Is a Relational Database? (RDBMS)? Online. Dostupné z: https://www.oracle.com/uk/database/what-is-a-relational-database/. [cit. 2026-02-03].
[37] SVOBODA, Martin. *Lecture: Relational Model*. Online. ČVUT. Dostupné z: https://cw.fel.cvut.cz/b212/_media/courses/b0b36dbs/lecture-relational-model.pdf. [cit. 2026-02-07].
[38] IBM. What is PostgreSQL? Online. Dostupné z: https://www.ibm.com/think/topics/postgresql. [cit. 2026-02-07].
[39] POSTGRESQL. PostgreSQL: The World's Most Advanced Open Source Relational Database. Online. Dostupné z: https://www.postgresql.org/. [cit. 2026-02-07].
[40] AMAZON, AWS. What is SQL (Structured Query Language)? Online. Dostupné z: https://aws.amazon.com/what-is/sql/. [cit. 2026-02-07].
[41] IBM. What is Structured Query Language (SQL)? Online. Dostupné z: https://www.ibm.com/think/topics/structured-query-language. [cit. 2026-02-07].
[42] IXDF, Interaction Design Foundation. What is User Interface (UI) Design? Online. Dostupné z: https://www.interaction-design.org/literature/topics/ui-design. [cit. 2026-02-07].

[43] IXDF, Interaction Design Foundation. What is User Experience Design? Online. Dostupné z: https://www.interaction-design.org/literature/topics/ux-design. [cit. 2026-02-07].
[44] FIGMA. What is the difference between UI and UX? Online. Dostupné z: https://www.figma.com/resource-library/difference-between-ui-and-ux/. [cit. 2026-02-07].
[45] STANFORD. Usability Principles. Online. Dostupné z: https://improvement.stanford.edu/resources/usability-principles. [cit. 2026-02-07].
[46] THE USABILITY BODY OF KNOWLEDGE. What is Usability? Online. Dostupné z: https://www.usabilitybok.org/what-is-usability. [cit. 2026-02-07].
[47] BALSAMIQ. What is a wireframe? A guide for non-designers. Online. Dostupné z: https://balsamiq.com/blog/what-are-wireframes/. [cit. 2026-02-07].
[48] UXPIN. What is a Prototype? A Guide to Functional UX. Online. Dostupné z: https://www.uxpin.com/studio/blog/what-is-a-prototype-a-guide-to-functional-ux/. [cit. 2026-02-07].
[49] OPTIMIZELY. Website optimization. Online. Dostupné z: https://www.optimizely.com/optimization-glossary/website-optimization/. [cit. 2026-02-10].
[50] GOOGLE. Search Engine Optimization (SEO) Starter Guide. Online. Dostupné z: https://developers.google.com/search/docs/fundamentals/seo-starter-guide. [cit. 2026-02-10].
[51] KOMERČNÍ BANKA. Co je SEO a jak pomůže vašemu podnikání. Online. Dostupné z: https://www.kb.cz/cs/podnikatele-a-firmy/kb-radce-pro-podnikatele/rady-podnikatelum/co-je-seo-a-jak-pomuze-vasemu-podnikani. [cit. 2026-02-10].
[52] GOOGLE. Meta tags and attributes that Google supports. Online. Dostupné z: https://developers.google.com/search/docs/crawling-indexing/special-tags. [cit. 2026-02-10].
[53] COLLABIM. Metadata a SEO: Vše, co o nich potřebujete vědět. Online. Dostupné z: https://www.collabim.cz/akademie/knihovna/metadata-a-seo-vse-co-o-nich-potrebujete-vedet/. [cit. 2026-02-10].
[54] GOOGLE. Understanding Core Web Vitals and Google search results. Online. Dostupné z: https://developers.google.com/search/docs/appearance/core-web-vitals. [cit. 2026-02-10].

[55] GITLAB. What is version control? Online. Dostupné z: https://about.gitlab.com/topics/version-control/. [cit. 2026-02-10].
[56] UMBRACO. What is Deployment? Online. Dostupné z: https://umbraco.com/knowledge-base/deployment/#:~:text=Deployment%20in%20software%20and%20web,live%20environment%20or%20production%20environment.. [cit. 2026-02-10].
[57] GIT. Git is lightning fast and has a huge ecosystem of GUIs, hosting services, and command-line tools. Online. Dostupné z: https://git-scm.com. [cit. 2026-02-10].
[58] GIT. What is Git? Online. Dostupné z: https://www.atlassian.com/git/tutorials/what-is-git. [cit. 2026-02-10].
[59] VERCEL. Build and deploy on the AI Cloud. Online. Dostupné z: https://vercel.com/. [cit. 2026-02-10].
[60] CRUICKSHANK, Dan. What is Vercel? Online. Dostupné z: https://www.getfishtank.com/insights/what-is-vercel. [cit. 2026-02-10].
[61] STÁDNÍK, Adam. *SousedePůjč*. Online. Dostupné z: https://sousedepujc.vercel.app/. [cit. 2026-02-18].

8 Seznam obrázků, tabulek, a zkratek

8.1 Seznam obrázků

Nenalezena položka seznamu obrázků.

8.2 Seznam tabulek

Tabulka 1- Přehled cizích klíčů a referenční integrity mezi tabulky 62
Tabulka 2 - Přehled testovacích scénářů a jejich výsledků.....86

8.3 Seznam zkratek

SQL – Structured Query Language
UI – User Interface
UX – User Experience
P2P – Peer to Peer
HTTP – Hypertext Transfer Protocol
HTML – Hypertext Markup Language
REST – Representational State Transfer
API – Application Programming Interface
RPC – Remote Procedure Call
SOAP – Simple Object Access Protocol
HATEOAS – Hypermedia as the Engine of Application State
URL – Uniform Resource Locator
CSS – Cascading Style Sheets
DOM – Document Object Model
SEO – Search Engine Optimization
JSON – JavaScript Object Notation
OOP – Object-Oriented Programming
SSR – Server-side Rendering
JIT – Just In Time
JSX – JavaScript XML
CSR – Client-side Rendering
BaaS – Backend-as-a-Service
SDK – Software Development Kit
CRUD – Create, Read, Update, Delete
JWT – JSON Web Token
PaaS – Platform-as-a-Service
RLS – Row Level Security
ACID – Atomicity, Consistency, Isolation, Durability
MVCC – Multiversion Concurrency Control
DML – Data Manipulation Language
DDL – Data Definition Language
DCL – Data Control Language
CTA – Call to Action
CTR – Click-Through Rate
CLS – Cumulative Layout Shift

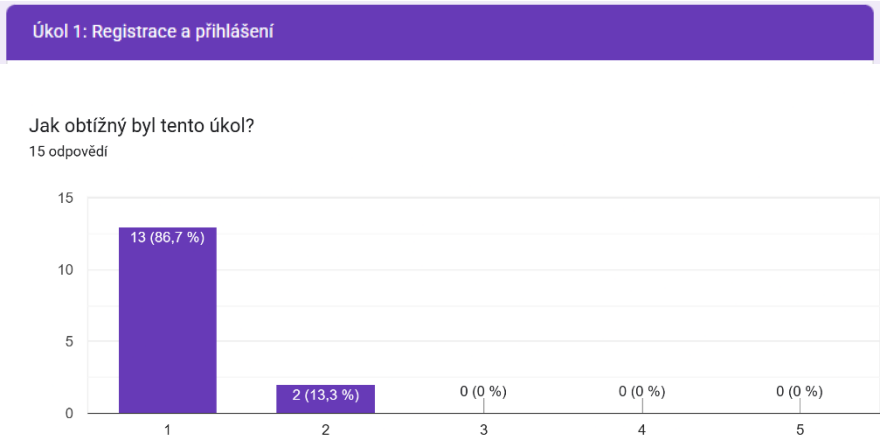
LCP – Largest Contentful Paint
INP – Interaction to Next Paint
CI/CD – Continuous Integration / Continuous Deployment
MVP – Minimum Viable Product
UUID – Universally Unique Identifier
JSONB – JavaScript Object Notation Binary
PL/pgSQL – Procedural Language/PostgreSQL
WAI-ARIA – Web Accessibility Initiative – Accessible Rich Internet Applications
SUS – System Usability Scale
SWC – Speedy Web Compiler

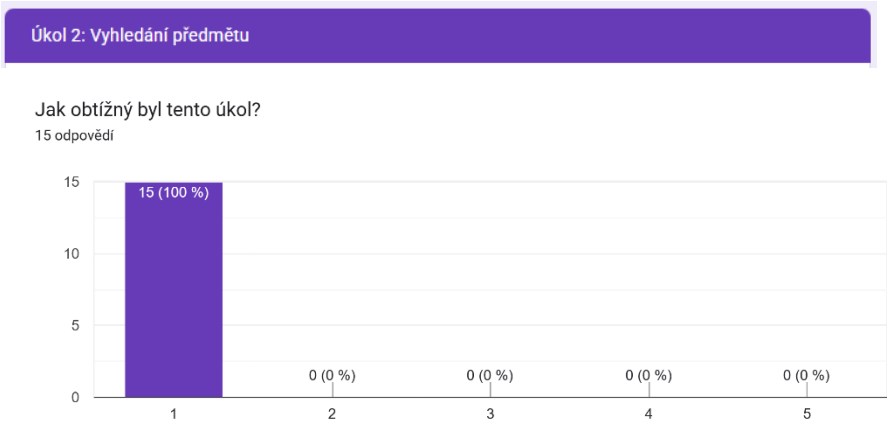
9 Přílohy

9.1 Umístění aplikace

9.2 Výsledky testování

Respondenti hodnotili obtížnost úkolů na škále 1 (velmi snadný) až 5 (velmi obtížný). U dotazníku SUS vyjadřovali míru souhlasu na škále 1 (zcela nesouhlasím) až 5 (zcela souhlasím)

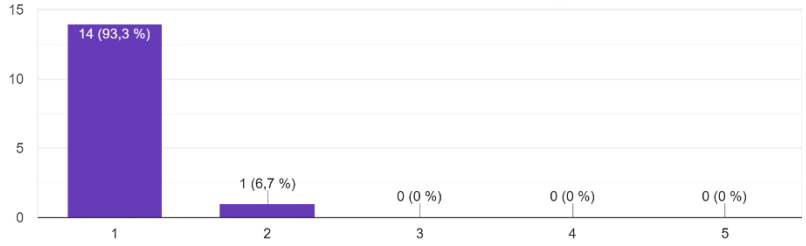




Úkol 3: Zobrazení detailu předmětu

Jak obtížný byl tento úkol?

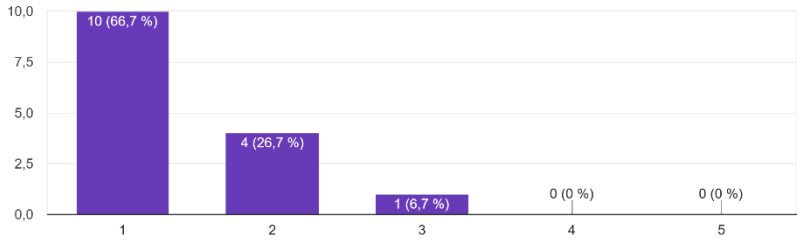
15 odpovědí



Úkol 4: Vytvoření nového inzerátu

Jak obtížný byl tento úkol?

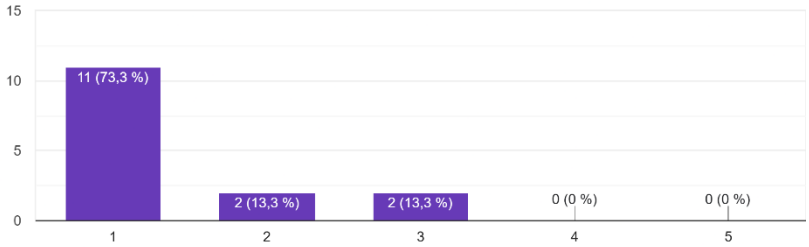
15 odpovědí



Úkol 5: Odeslání žádosti o rezervaci

Jak obtížný byl tento úkol?

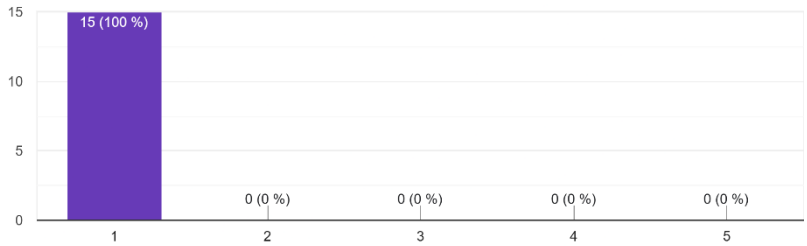
15 odpovědí



Úkol 6: Odeslání zprávy v chatu

Jak obtížný byl tento úkol?

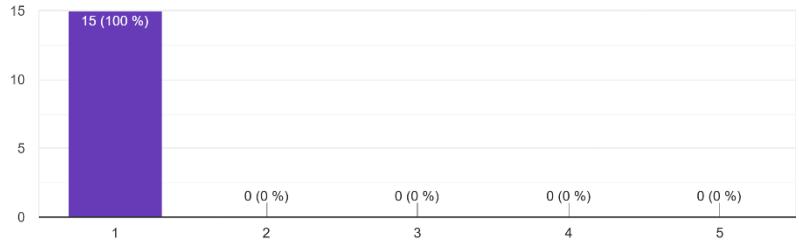
15 odpovědí



Úkol 7: Kontrola notifikací

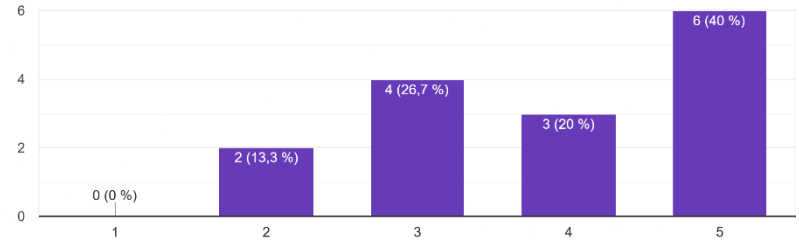
Jak obtížný byl tento úkol?

15 odpovědí



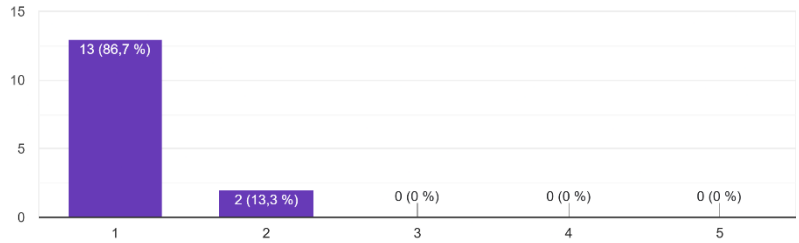
Myslím, že bych tuto aplikaci používal/a rád/a pravidelně.

15 odpovědí



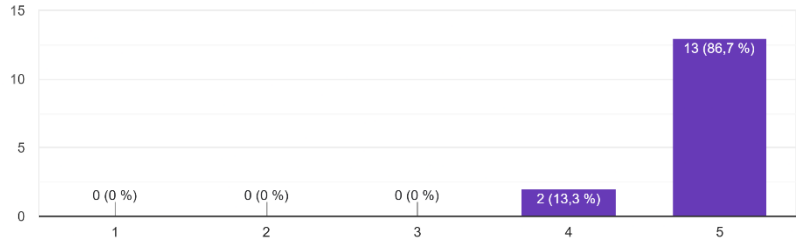
Aplikace mi přišla zbytečně složitá.

15 odpovědí

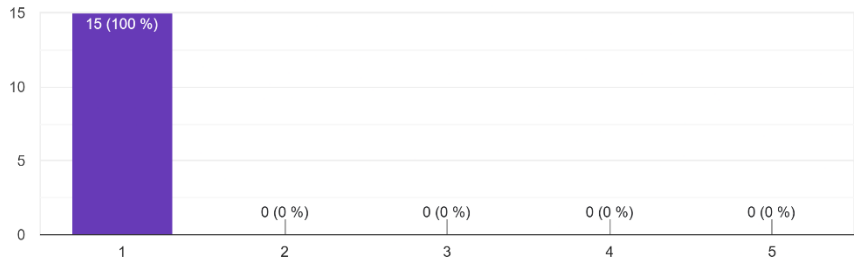


Myslím, že aplikace se snadno ovládá.

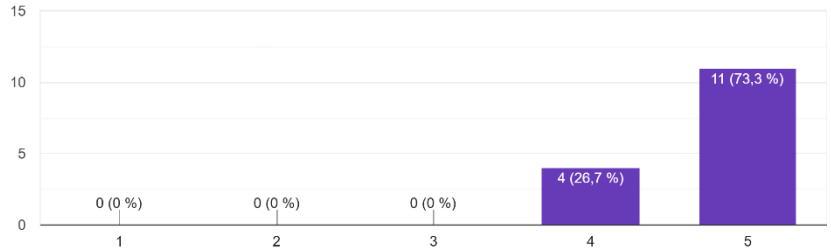
15 odpovědí



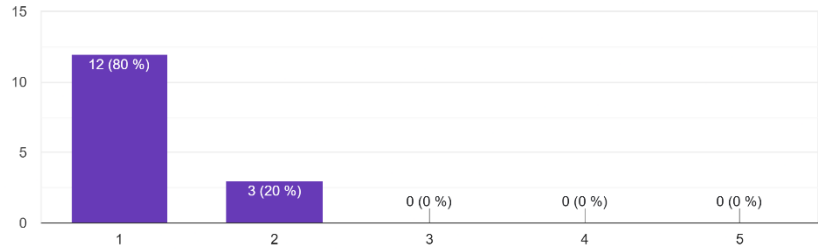
Myslím, že bych potřeboval/a pomoc technicky zdatné osoby, abych mohl/a aplikaci používat.
15 odpovědí



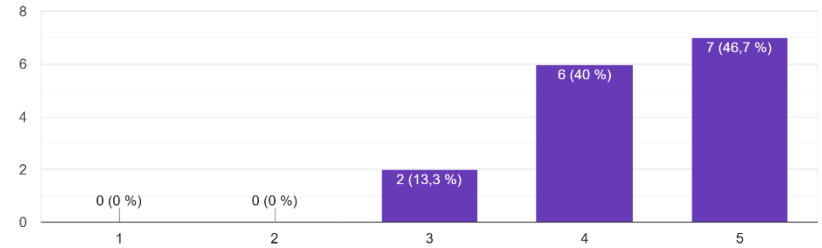
Jednotlivé funkce aplikace jsou dobře propojené.
15 odpovědí



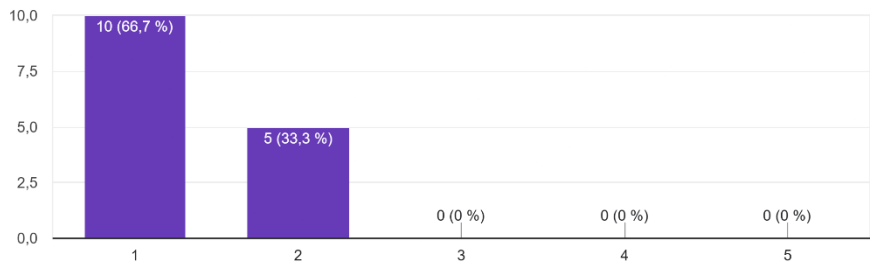
V aplikaci bylo příliš mnoho nekonzistencí (nesourodostí).
15 odpovědí



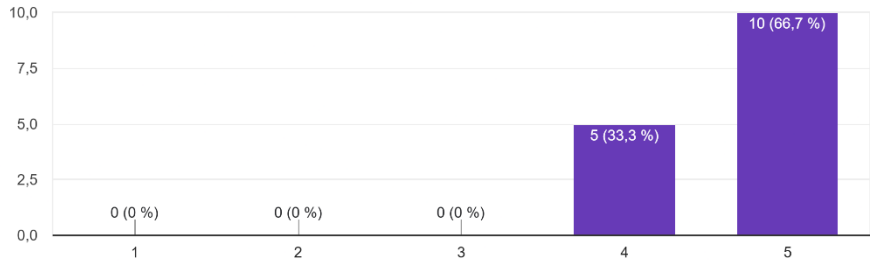
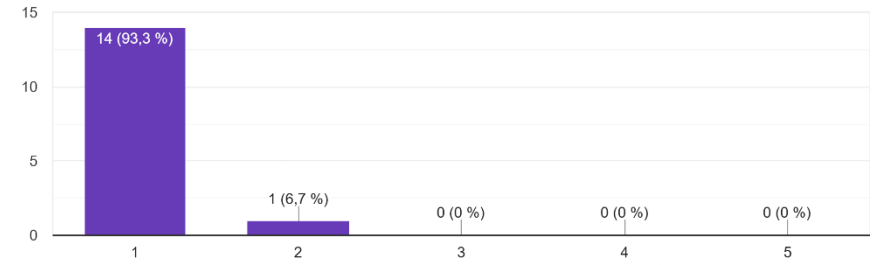
Dokážu si představit, že většina lidí se naučí tuto aplikaci ovládat velmi rychle.
15 odpovědí



Ovládání aplikace mi přišlo těžkopádné.
15 odpovědí



Při používání aplikace jsem se cítil/a jistě.
15 odpovědí



Musel/a jsem se naučit mnoho věcí, než jsem mohl/a aplikaci začít používat.
15 odpovědí